



Отчет о проверке

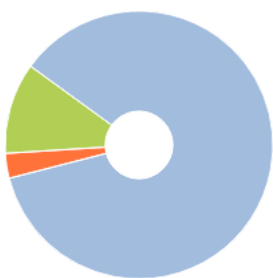
Автор: Острожных Денис Михайлович

Название документа: Цифровая платформа оптовых закупок товаров для организаций

Проверяющий: Шиманская Оксана Николаевна

Организация: Государственное учреждение «Национальная библиотека Беларуси»

РЕЗУЛЬТАТЫ ПРОВЕРКИ



Совпадения:
3,06%



Оригинальность:
85,51%



ИИ-контент:
10,28%



Цитирования:
11,43%



Самоцитирования:
0%



«Совпадения», «Цитирования», «Самоцитирования», «Оригинальность» являются отдельными показателями, отображаются в процентах и в сумме дают 100%, что соответствует проверенному тексту документа.



Есть подозрения на следующие группы маскировки заимствований: Сгенерированный текст на страницах: 19, 20, 21, 23, 24, 28, 41, 44, 49, 61... еще на 5 стр.



Проверено: 98,54% текста документа, исключено из проверки: 1,46% текста документа. Разделы и элементы, отключенные пользователем: Таблицы

- Совпадения** — фрагменты проверяемого текста, полностью или частично сходные с найденными источниками, за исключением фрагментов, которые система отнесла к цитированию или самоцитированию. Показатель «Совпадения» — это доля фрагментов проверяемого текста, отнесенных к совпадениям, в общем объеме текста.
- Самоцитирования** — фрагменты проверяемого текста, совпадающие или почти совпадающие с фрагментом текста источника, автором или соавтором которого является автор проверяемого документа. Показатель «Самоцитирования» — это доля фрагментов текста, отнесенных к самоцитированию, в общем объеме текста.
- Цитирования** — фрагменты проверяемого текста, которые не являются авторскими, но которые система отнесла к корректно оформленным. К цитированиям относятся также шаблонные фразы; библиография; фрагменты текста, найденные модулем поиска «СПС Гарант: нормативно-правовая документация». Показатель «Цитирования» — это доля фрагментов проверяемого текста, отнесенных к цитированию, в общем объеме текста.
- Текстовое пересечение** — фрагмент текста проверяемого документа, совпадающий или почти совпадающий с фрагментом текста источника.
- Источник** — документ, проиндексированный в системе и содержащийся в модуле поиска, по которому проводится проверка.
- Оригинальный текст** — фрагменты проверяемого текста, не обнаруженные ни в одном источнике и не отмеченные ни одним из модулей поиска. Показатель «Оригинальность» — это доля фрагментов проверяемого текста, отнесенных к оригинальному тексту, в общем объеме текста.

Обращаем Ваше внимание, что система находит текстовые совпадения проверяемого документа с проиндексированными в системе источниками. При этом система является вспомогательным инструментом, определение корректности и правомерности совпадений или цитирований, а также авторства текстовых фрагментов проверяемого документа остается в компетенции проверяющего.

ИНФОРМАЦИЯ О ДОКУМЕНТЕ

Номер документа: 2602

Тип документа: Дипломная работа

Дата проверки: 14.05.2026 18:58:58

Дата корректировки: 14.05.2026 19:02:17

Количество страниц: 84

Символов в тексте: 144173

Слов в тексте: 17177

Число предложений: 5293

Комментарий: не указано

ПАРАМЕТРЫ ПРОВЕРКИ

Выполнена проверка с учетом редактирования: Да

Выполнено распознавание текста (OCR): Нет

Выполнена проверка с учетом структуры: Нет

Разделы и элементы, отключенные пользователем: Таблицы

Модули поиска: Профессиональная лексика. АПК и биотех, Профессиональная лексика. Медицина, Переводные заимствования, Коллекция НБУ, Шаблонные фразы, Цитирование, PubMed, Публикации eLIBRARY, ИПС Адилет, СМИ России и СНГ, IEEE, Патенты СССР, РФ, СНГ, Сводная коллекция научных работ Беларуси, Коллекция открытых публикаций международных издательств, Перефразирования по коллекции IEEE, Перефразирования по базе публикаций открытого доступа PubMed, СПС ГАРАНТ: аналитика, Публикации РГБ, СПС ГАРАНТ: нормативно-правовая документация, Медицина, Перефразирования по Коллекции открытых публикаций международных издательств, Кольцо вузов, Сводная коллекция ЭБС, Переводные заимствования по коллекции Гарант: аналитика, Перефразированные заимствования по коллекции Интернет в английском сегменте, Перефразирования по СПС ГАРАНТ: аналитика, Публикации eLIBRARY (переводы и перефразирования), Переводные заимствования по коллекции Интернет в русском сегменте, Переводные заимствования по базе публикаций открытого доступа PubMed, Публикации РГБ (переводы и перефразирования), Переводные заимствования по коллекции Интернет в английском сегменте, Переводные заимствования по Коллекции открытых публикаций международных издательств, Переводные заимствования IEEE, Интернет Плюс, Перефразированные заимствования по коллекции Интернет в русском сегменте, Собственная коллекция компании

ИСТОЧНИКИ

№	Доля в тексте	Доля в отчете	Источник	Актуален на	Модуль поиска	Комментарий
[01]	10,61%	10,04%	не указано	13 Янв 2022	Цитирование	
[02]	2,16%	0%	https://studfile.net/preview/86390... https://studfile.net	07 Мар 2024	Интернет Плюс	
[03]	2,08%	0%	А. Г. Ивашко, Ю. Е. Карякин ; Росс... http://dlib.rsl.ru	01 Янв 2013	Публикации РГБ	
[04]	2,07%	0,01%	Metodicheskoe_posobie_po_discipl... https://sveden.utmn.ru	02 Июн 2025	Интернет Плюс	
[05]	1,98%	0%	Avdeev%202.pdf http://elib.kstu.kz	18 Янв 2025	Интернет Плюс	
[06]	1,78%	0,08%	Л. И. Алешин Проектирование би... http://dlib.rsl.ru	01 Янв 2008	Публикации РГБ	
[07]	1,66%	0,01%	Глоссарий терминов в сфере инф... https://book.ru	01 Янв 2018	Сводная коллекция ЭБС	
[08]	1,61%	0,01%	Нормативное обеспечение инфо... http://dlib.rsl.ru	01 Янв 2017	Публикации РГБ	
[09]	1,61%	0%	Нормативное обеспечение инфо... https://book.ru	01 Янв 2020	Сводная коллекция ЭБС	
[10]	1,61%	0%	Нормативное обеспечение инфо... https://book.ru	01 Янв 2017	Сводная коллекция ЭБС	
[11]	1,5%	0%	https://studfile.net/preview/164595... https://studfile.net	23 Мая 2021	Интернет Плюс	
[12]	1,49%	0%	А. С. Тимонин ; Федеральное аге... http://dlib.rsl.ru	01 Янв 2008	Публикации РГБ	
[13]	1,45%	0,1%	Руководство по использованию с... http://elibrary.ru	01 Янв 2018	Публикации eLIBRARY	
[14]	1,45%	0%	ГОСТ 34.602-89 Информационная... https://allgosts.ru	23 Мая 2019	Интернет Плюс	
[15]	1,43%	0%	АВТОМАТИЗИРОВАННАЯ СИСТЕМ...	10 Июн 2016	Кольцо вузов	
[16]	1,42%	0%	https://libeldoc.bsuir.by/bitstream/... https://libeldoc.bsuir.by	19 Ноя 2024	Интернет Плюс	
[17]	1,36%	0,08%	Технология проектирования авт...	20 Дек 2016	Медицина	
[18]	1,36%	0%	ISBN9785991201483.txt	26 Окт 2017	Кольцо вузов	
[19]	1,36%	0%	Технология проектирования авт...	27 Ноя 2017	Сводная коллекция ЭБС	
[20]	1,36%	0%	Технология проектирования авт... https://geotar.ru	19 Авг 2016	Сводная коллекция ЭБС	
[21]	1,34%	0%	Диплом	11 Янв 2024	Кольцо вузов	

[22]	1,33%	0,55%	Приказ Министерства науки, инф... http://ivo.garant.ru	23 Апр 2011	СПС ГАРАНТ: нормативно-правовая документация
[23]	1,33%	0%	Приказ Министерства науки, инф... http://ivo.garant.ru	23 Апр 2011	СПС ГАРАНТ: нормативно-правовая документация
[24]	1,31%	0,45%	Приказ Федеральной налоговой ... http://ivo.garant.ru	04 Мар 2014	СПС ГАРАНТ: нормативно-правовая документация
[25]	1,25%	0%	Ю. И. Коваленко Защита информ... http://dlib.rsl.ru	01 Янв 2016	Публикации РГБ
[26]	1,24%	0%	Организация и сопровождение э... http://elibrary.ru	01 Янв 2020	Публикации eLIBRARY
[27]	1,24%	0,19%	Politika19crp.pdf https://19crp.by	10 Ноя 2024	Интернет Плюс
[28]	1,22%	0%	ИНФОРМАЦИОННЫЕ СИСТЕМЫ ... http://elibrary.ru	01 Янв 2023	Публикации eLIBRARY (переводы и перефразирования)
[29]	1,21%	0,08%	ПОКАЗАТЕЛИ БЕЗОПАСНОСТИ КА... http://elibrary.ru	01 Янв 2022	Публикации eLIBRARY
[30]	1,2%	0,08%	Межгосударственный стандарт Г... http://ivo.garant.ru	07 Фев 2009	СПС ГАРАНТ: нормативно-правовая документация
[31]	1,2%	0%	Учебное пособие ПИС 16 12 22 н...	12 Янв 2024	Кольцо вузов
[32]	1,19%	0%	Диплом_Ткачев_151	30 Мая 2025	Кольцо вузов
[33]	1,17%	0%	Ф.С. Воройский Основы проекти...	12 Окт 2017	Публикации РГБ
[34]	1,16%	0,12%	Закон Республики Беларусь от 7 ... http://ivo.garant.ru	07 Мая 2021	СПС ГАРАНТ: нормативно-правовая документация
[35]	1,05%	0,01%	СИСТЕМЫ АВТОМАТИЗИРОВАНН... http://elibrary.ru	01 Янв 2018	Публикации eLIBRARY (переводы и перефразирования)
[36]	1,04%	0%	ISBN9785893499780.txt	26 Окт 2017	Кольцо вузов
[37]	1,01%	0,18%	Предварительный национальны... http://ivo.garant.ru	30 Мая 2021	СПС ГАРАНТ: нормативно-правовая документация
[38]	0,99%	0%	ВКР Веретенников А.С.	20 Дек 2024	Кольцо вузов
[39]	0,98%	0,05%	%D0%9F%D0%BE%D0%BB%D0%B... https://beurer-belarus.by	14 Апр 2026	Интернет Плюс
[40]	0,96%	0%	Информационная безопасность ... https://geotar.ru	14 Янв 2020	Сводная коллекция ЭБС
[41]	0,96%	0%	Инжиниринг объектов интеллект...	19 Дек 2016	Медицина
[42]	0,95%	0%	Л. И. Алешин Проектирование би... http://dlib.rsl.ru	01 Янв 2008	Публикации РГБ (переводы и перефразирования)
[43]	0,95%	0%	Сборник примеров заключений э... http://elibrary.ru	01 Янв 2016	Публикации eLIBRARY
[44]	0,9%	0%	СИСТЕМЫ АВТОМАТИЗИРОВАНН... http://elibrary.ru	01 Янв 2018	Публикации eLIBRARY
[45]	0,89%	0%	Основы проектирования автомат...	19 Дек 2016	Медицина
[46]	0,88%	0%	Приказ Департамента государств... http://ivo.garant.ru	22 Авг 2015	СПС ГАРАНТ: нормативно-правовая документация
[47]	0,86%	0%	Руководство по использованию с... http://elibrary.ru	01 Янв 2018	Публикации eLIBRARY (переводы и перефразирования)
[48]	0,86%	0%	Персональные данные. Образцы ... http://ivo.garant.ru	08 Апр 2023	СПС ГАРАНТ: аналитика
[49]	0,85%	0,05%	ИНФОРМАЦИОННЫЕ ТЕХНОЛОГ...	21 Фев 2017	Сводная коллекция ЭБС
[50]	0,84%	0%	Как разработать техническое зад... http://e-xecutive.ru	16 Апр 2012	СМИ России и СНГ
[51]	0,83%	0%	Ершов, Александр Александрови... http://dlib.rsl.ru	01 Янв 2013	Публикации РГБ
[52]	0,82%	0,03%	Problematic Issues of Legalisation ... https://openalex.org	10 Апр 2023	Коллекция открытых публикаций международных издательств

[53]	0,82%	0%	А. Г. Ивашко, Ю. Е. Карякин ; Росс... http://dlib.rsl.ru	01 Янв 2013	Публикации РГБ (переводы и перефразирования)	
[54]	0,8%	0%	Проектирование автоматизиров... https://geotar.ru	27 Апр 2023	Сводная коллекция ЭБС	
[55]	0,79%	0,02%	Сборник примеров заключений э... http://elibrary.ru	01 Янв 2016	Публикации eLIBRARY (переводы и перефразирования)	
[56]	0,78%	0%	Теплоэнергетика и теплотехника...	19 Дек 2016	Медицина	
[57]	0,76%	0%	Федеральный закон от 27.07.200... http://elibrary.ru	17 Июнь 2020	Публикации eLIBRARY	
[58]	0,73%	0%	Автоматизированная обработка ...	19 Дек 2016	Медицина	
[59]	0,72%	0%	Уланский, Александр Борисович ... http://dlib.rsl.ru	01 Янв 2006	Публикации РГБ (переводы и перефразирования)	
[60]	0,72%	0%	Требования к информационным ...	30 Окт 2024	СПС ГАРАНТ: аналитика	
[61]	0,72%	0%	Разработка Технического задани... https://pcnews.ru	16 Авг 2019	СМИ России и СНГ	Источник исключен. Причина: Маленький процент пересечения.
[62]	0,71%	0,13%	Требования к информационным ...	30 Окт 2024	Перефразирования по СПС ГАРАНТ: аналитика	
[63]	0,69%	0%	Персональные данные. Бизнес р...	19 Мар 2025	СПС ГАРАНТ: аналитика	
[64]	0,66%	0%	Уголовно-правовая охрана персо... https://elib.nlb.by	01 Янв 2024	Сводная коллекция научных работ Беларуси	Источник исключен. Причина: Маленький процент пересечения.
[65]	0,66%	0,66%	Состав и структура ВКР, методич... https://pandia.ru	17 Фев 2025	Переводные заимствования по коллекции Интернет в русском сегменте	
[66]	0,65%	0%	Modeling a Program with Vulnerabi... https://openalex.org	13 Мар 2023	Коллекция открытых публикаций международных издательств	Источник исключен. Причина: Маленький процент пересечения.
[67]	0,63%	0%	Куликова С.А., Пешкова (Белогор... http://ivo.garant.ru	29 Авг 2020	СПС ГАРАНТ: аналитика	
[68]	0,63%	0,04%	Роль и место интеллектуальной с...	09 Окт 2025	СПС ГАРАНТ: аналитика	
[69]	0,63%	0%	Комментарий к Федеральному за... http://ivo.garant.ru	06 Мар 2021	СПС ГАРАНТ: аналитика	
[70]	0,61%	0%	не указано	13 Янв 2022	Шаблонные фразы	Источник исключен. Причина: Маленький процент пересечения.
[71]	0,61%	0,04%	К ВОПРОСУ О ПРАВЕ СУБЪЕКТА П... http://elibrary.ru	01 Янв 2024	Публикации eLIBRARY	
[72]	0,6%	0%	Коржов В.Ю., Великанов А.П., Ив... http://ivo.garant.ru	22 Авг 2015	СПС ГАРАНТ: аналитика	Источник исключен. Причина: Маленький процент пересечения.
[73]	0,6%	0%	Правовые основы обеспечения и... http://journal.ib-bank.ru	19 Июнь 2013	СМИ России и СНГ	Источник исключен. Причина: Маленький процент пересечения.
[74]	0,59%	0%	Арестова О.Н., Ковалева Н.Н. Ком... http://ivo.garant.ru	07 Авг 2010	СПС ГАРАНТ: аналитика	
[75]	0,57%	0%	Распоряжение Правительства РФ... http://ivo.garant.ru	05 Апр 2019	СПС ГАРАНТ: нормативно-правовая документация	
[76]	0,56%	0,06%	К ВОПРОСУ О ФОРМИРОВАНИИ ... http://elibrary.ru	01 Янв 2024	Публикации eLIBRARY	
[77]	0,55%	0%	Об утверждении плана-графика ... https://adilet.zan.kz	12 Дек 2025	ИПС Адилет	Источник исключен. Причина: Маленький процент пересечения.
[78]	0,53%	0%	Делопроизводство и архивное де...	20 Дек 2016	Медицина	
[79]	0,53%	0%	ISBN9785976507845.txt	26 Окт 2017	Кольцо вузов	
[80]	0,53%	0%	[Рассолов И. М., Чубукова С. Г., Су... http://dlib.rsl.ru	30 Ноя 2014	Публикации РГБ	
[81]	0,52%	0%	Правосудие и правоохранительн... https://openalex.org	01 Янв 2025	Коллекция открытых публикаций международных издательств	Источник исключен. Причина: Маленький процент пересечения.
[82]	0,51%	0%	Роль и место интеллектуальной с...	09 Окт 2025	Перефразирования по СПС ГАРАНТ: аналитика	
[83]	0,51%	0%	К ВОПРОСУ О ТЕХНИЧЕСКОМ ЗАД... http://elibrary.ru	31 Авг 2017	Публикации eLIBRARY	

[84]	0,48%	0%	Национальный стандарт РФ ГОС... http://ivo.garant.ru	24 Ноя 2014	СПС ГАРАНТ: нормативно-правовая документация	Источник исключен. Причина: Маленький процент пересечения.
[85]	0,45%	0,01%	К ВОПРОСУ О ПРАВЕ СУБЪЕКТА П... http://elibrary.ru	01 Янв 2024	Публикации eLIBRARY (переводы и перефразирования)	
[86]	0,44%	0%	Технологии изготовления детале... https://elib.nlb.by	01 Янв 2017	Сводная коллекция научных работ Беларуси	Источник исключен. Причина: Маленький процент пересечения.
[87]	0,44%	0%	Информатика	19 Дек 2016	Медицина	Источник исключен. Причина: Маленький процент пересечения.
[88]	0,43%	0%	О перечне стандартов и рекомен... https://adilet.zan.kz	12 Дек 2025	ИПС Адилет	Источник исключен. Причина: Маленький процент пересечения.
[89]	0,43%	0%	Методы и средства разграничен... https://elib.nlb.by	01 Янв 2003	Сводная коллекция научных работ Беларуси	Источник исключен. Причина: Маленький процент пересечения.
[90]	0,42%	0%	Внедрение СЭД vs требования к ... https://habr.com	04 Июл 2021	СМИ России и СНГ	Источник исключен. Причина: Маленький процент пересечения.
[91]	0,41%	0%	Моделирование и оптимизация ... https://elib.nlb.by	раньше 2011	Сводная коллекция научных работ Беларуси	Источник исключен. Причина: Маленький процент пересечения.
[92]	0,41%	0%	ВКР Тюрина ЭБ-18-2 (1)	10 Янв 2024	Кольцо вузов	
[93]	0,38%	0%	Методические рекомендации ко... http://ivo.garant.ru	02 Июн 2012	СПС ГАРАНТ: аналитика	Источник исключен. Причина: Маленький процент пересечения.
[94]	0,37%	0%	Руководство по выполнению вып... http://biblioclub.ru	21 Янв 2020	Сводная коллекция ЭБС	Источник исключен. Причина: Маленький процент пересечения.
[95]	0,36%	0%	Национальный стандарт РФ ГОС... http://ivo.garant.ru	24 Мар 2012	СПС ГАРАНТ: нормативно-правовая документация	Источник исключен. Причина: Маленький процент пересечения.
[96]	0,35%	0,17%	Формирование аппаратного и пр...	07 Ноя 2024	Переводные заимствования по коллекции Гарант: аналитика	
[97]	0,34%	0,12%	Формирование аппаратного и пр...	07 Ноя 2024	Перефразирования по СПС ГАРАНТ: аналитика	
[98]	0,32%	0,06%	Ручьев, Анатолий Геннадьевич ... http://dlib.rsl.ru	01 Янв 2021	Публикации РГБ (переводы и перефразирования)	
[99]	0,31%	0%	не указано https://openalex.org	01 Авг 2023	Коллекция открытых публикаций международных издательств	Источник исключен. Причина: Маленький процент пересечения.
[100]	0,28%	0%	Conceptual foundations for assessi... https://openalex.org	01 Мая 2023	Коллекция открытых публикаций международных издательств	Источник исключен. Причина: Маленький процент пересечения.
[101]	0,28%	0%	Нарышкин, Константин Викторов... http://dlib.rsl.ru	01 Янв 2025	Публикации РГБ	Источник исключен. Причина: Маленький процент пересечения.
[102]	0,27%	0%	[Из песочницы] Порядок подгото... http://pcnews.ru	29 Июн 2015	СМИ России и СНГ	
[103]	0,26%	0%	В МЭСИ состоялось рабочее сове... http://moskva.bezformata.ru	10 Сен 2013	СМИ России и СНГ	Источник исключен. Причина: Маленький процент пересечения.
[104]	0,26%	0,05%	Основы PostgreSQL для начинаю... https://tproger.ru	03 Дек 2024	Перефразированные заимствования по коллекции Интернет в английском сегменте	
[105]	0,26%	0%	Как просто удалить дубликаты из... https://tproger.ru	01 Мар 2025	Перефразированные заимствования по коллекции Интернет в английском сегменте	
[106]	0,26%	0%	Нестеров, Максим Игоревич Упр... http://dlib.rsl.ru	01 Янв 2013	Публикации РГБ (переводы и перефразирования)	Источник исключен. Причина: Маленький процент пересечения.
[107]	0,25%	0%	Имя, должность, имущество: что ... https://zautra.by	22 Апр 2021	СМИ России и СНГ	Источник исключен. Причина: Маленький процент пересечения.
[108]	0,25%	0%	Готовые дипломные работы, кур... https://superinf.ru	13 Янв 2022	Интернет Плюс	
[109]	0,24%	0,24%	Веб-приложение для продажи ав...	18 Окт 2025	Публикации eLIBRARY (переводы и перефразирования)	
[110]	0,24%	0%	Израилов, Константин Евгеньевич... http://dlib.rsl.ru	01 Янв 2025	Публикации РГБ (переводы и перефразирования)	Источник исключен. Причина: Маленький процент пересечения.

[111]	0,23%	0%	Диплом Король - финал	23 Мая 2024	Кольцо вузов	Источник исключен. Причина: Маленький процент пересечения.
[112]	0,23%	0%	Отдельные вопросы получения с...	04 Июл 2025	Публикации eLIBRARY (переводы и перефразирования)	
[113]	0,23%	0,01%	ПОНЯТИЕ "БЛОК-СХЕМА" В ХРОН... http://elibrary.ru	01 Янв 2014	Публикации eLIBRARY	
[114]	0,22%	0%	М. В. Якунина ; Российская Федер...	11 Окт 2017	Публикации РГБ	Источник исключен. Причина: Маленький процент пересечения.
[115]	0,21%	0%	О Порядке взаимодействия межд... https://adilet.zan.kz	14 Дек 2025	ИПС Адилет	Источник исключен. Причина: Маленький процент пересечения.
[116]	0,21%	0%	Инженерно-техническая поддер... https://book.ru	01 Янв 2024	Сводная коллекция ЭБС	Источник исключен. Причина: Маленький процент пересечения.
[117]	0,19%	0%	Александров, Никита Дмитриевич... http://dlib.rsl.ru	04 Ноя 2025	Публикации РГБ (переводы и перефразирования)	
[118]	0,19%	0%	Об утверждении технического за... https://adilet.zan.kz	15 Дек 2025	ИПС Адилет	Источник исключен. Причина: Маленький процент пересечения.
[119]	0,19%	0%	%D0%9F%D0%BE%D0%BB%D0%B... https://beurer-belarus.by	14 Апр 2026	Переводные заимствования по коллекции Интернет в русском сегменте	
[120]	0,19%	0,01%	А. М. Афонин [и др.] Теоретическ... http://dlib.rsl.ru	01 Янв 2011	Публикации РГБ (переводы и перефразирования)	
[121]	0,19%	0%	Морозов, Сергей Александрович ... http://dlib.rsl.ru	01 Янв 2018	Публикации РГБ (переводы и перефразирования)	
[122]	0,18%	0,18%	https://elib.sfu-kras.ru/bitstream/h... https://elib.sfu-kras.ru	13 Дек 2023	Переводные заимствования по коллекции Интернет в русском сегменте	
[123]	0,18%	0,18%	Преддипломная практика и дипл... http://elibrary.ru	01 Янв 2013	Публикации eLIBRARY (переводы и перефразирования)	
[124]	0,18%	0,05%	Способ мониторинга информаци... http://findpatent.ru	24 Июн 2015	Патенты СССР, РФ, СНГ	
[125]	0,18%	0%	Система и способ защиты автома... https://fips.ru	04 Апр 2026	Патенты СССР, РФ, СНГ	
[126]	0,17%	0,17%	Муссель К.М. Платёжные техноло... http://ivo.garant.ru	21 Фев 2015	Перефразирования по СПС ГАРАНТ: аналитика	
[127]	0,17%	0%	Модель выбора шаблона програ... https://openalex.org	21 Июн 2019	Коллекция открытых публикаций международных издательств	Источник исключен. Причина: Маленький процент пересечения.
[128]	0,16%	0%	Вопрос: Принятие к учету автома... http://ivo.garant.ru	08 Апр 2023	СПС ГАРАНТ: аналитика	
[129]	0,16%	0,16%	Защита прав потребителей: в по...	18 Сен 2024	Переводные заимствования по коллекции Гарант: аналитика	
[130]	0,16%	0%	Вопрос: Принятие к учету автома... http://ivo.garant.ru	08 Апр 2023	Перефразирования по СПС ГАРАНТ: аналитика	Источник исключен. Причина: Маленький процент пересечения.
[131]	0,15%	0%	The Study of Metabolic By-Product... https://openalex.org	01 Янв 2022	Коллекция открытых публикаций международных издательств	Источник исключен. Причина: Маленький процент пересечения.
[132]	0,14%	0%	Информационное моделировани... http://ivo.garant.ru	26 Фев 2022	Перефразирования по СПС ГАРАНТ: аналитика	Источник исключен. Причина: Маленький процент пересечения.
[133]	0,14%	0%	Алтухова Н.Ф., Дзюбенко А.Л., Ло... http://ivo.garant.ru	04 Мая 2019	Перефразирования по СПС ГАРАНТ: аналитика	Источник исключен. Причина: Маленький процент пересечения.
[134]	0,13%	0%	АНАЛИЗ МЕТОДОВ ОЦЕНКИ ДЛЯ ... http://elibrary.ru	01 Янв 2025	Публикации eLIBRARY (переводы и перефразирования)	Источник исключен. Причина: Маленький процент пересечения.
[135]	0,13%	0%	Об утверждении Правил эксплуа... https://adilet.zan.kz	12 Дек 2025	ИПС Адилет	Источник исключен. Причина: Маленький процент пересечения.
[136]	0,13%	0%	Способ мониторинга безопаснос... http://findpatent.ru	24 Июн 2015	Патенты СССР, РФ, СНГ	Источник исключен. Причина: Маленький процент пересечения.
[137]	0,12%	0%	https://cpd.by/storage/2023/02/NP... https://cpd.by	16 Янв 2024	Переводные заимствования по коллекции Интернет в русском сегменте	Источник исключен. Причина: Маленький процент пересечения.

[138]	0,11%	0%	СПОСОБ ПОДГОТОВКИ ДОКУМЕН... https://fips.ru	17 Апр 2026	Патенты СССР, РФ, СНГ	Источник исключен. Причина: Маленький процент пересечения.
[139]	0,11%	0%	Способ обнаружения удаленных ... http://findpatent.ru	24 Июн 2015	Патенты СССР, РФ, СНГ	Источник исключен. Причина: Маленький процент пересечения.
[140]	0,1%	0%	ЗАЩИТА ПЕРСОНАЛЬНЫХ ДАНН... http://bis-expert.ru	02 Сен 2014	СМИ России и СНГ	Источник исключен. Причина: Маленький процент пересечения.

Учреждение образования Федерации профсоюзов Беларуси
«Международный университет «МИТСО»

Экономический факультет

Кафедра информационных технологий

Допущен к защите
Заведующий кафедрой
_____ О.Б. Хорошко
« ____ » _____ 2026

ДИПЛОМНЫЙ ПРОЕКТ

Цифровая платформа оптовых закупок товаров для организаций

Автор: _____
Острожных Денис Михайлович
Курс 4, группа № 2221
Специальность: информационные
системы и технологии

Научный руководитель: _____
Старший преподаватель:
Гардейчик Сергей Михайлович

Нормоконтролер _____ Пархемович Анна Игоревна

Минск 2026

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ ФЕДЕРАЦИИ ПРОФСОЮЗОВ БЕЛАРУСИ
«МЕЖДУНАРОДНЫЙ УНИВЕРСИТЕТ «МИТСО»

Экономический факультет
Специальность 1-40 05 01-02 «Информационные системы и технологии»
Кафедра информационных технологий

УТВЕРЖДАЮ
Заведующий кафедрой
информационных технологий
_____ О.Б.Хорошко
«__» _____ 2026 г

ЗАДАНИЕ
на дипломный проект

Обучающемуся Острожных Денису Михайловичу

Курс 4 Учебная группа 2221

Специальность 1-40 05 01-02 «Информационные системы и технологии»

1. Тема дипломного проекта: Платформа для корпоративного обучения.
2. Утверждена руководителем УВО от «27» февраля 2026 г. № 144 К (ст)
3. Исходные данные к дипломному проекту: нормативные законодательные акты, бухгалтерская и статистическая отчетность предприятия, научная литература по теме работы, статистические данные, интернет-ресурсы, результаты собственных исследований.
4. Перечень подлежащих разработке вопросов или краткое содержание пояснительной записки:

Дипломный проект – это самостоятельно выполненная проектно- конструкторская разработка. Актуальная, по крайней мере, для предприятия на котором проходила преддипломная практика студента, содержащая элемент новизны и практической значимости. Состоит из следующих взаимосвязанных частей.

Введение. Во введении необходимо отразить актуальность выбранной темы, цель и задачи, решаемые в проекте, используемые методики, практическую значимость полученных результатов, привести общие сведения о проекте (структура, краткое содержание разделов).

Аналитическая часть, целью которой является рассмотрение существующего состояния, разрабатываемой в дипломе информационной системы («КАК ЕСТЬ») предприятия (организации, фирмы, их подразделений), или создаваемой им для заказчика. Дать характеристику задач информационного обеспечения, решаемых существующей информационной системой (далее – ИС), рассмотреть бизнес-процессы, реализуемые в выделенной предметной области, выявить проблемы и недостатки в функционировании ИС и обоснование предложений по их устранению, внедрению новых подходов, новых технологий и т. д., обеспечивающих более эффективные решения («КАК ДОЛЖНО БЫТЬ»).

Проектная часть, целью которой является описание решений, принятых и реализованных по всей вертикали создания системы. Проектная часть должна быть основана на информации, представленной в аналитической части, с обобщением ее на языке информационных технологий. **123**

Расчет экономической эффективности проекта.

В обязательном порядке должен быть приведен анализ основных экономических показателей деятельности организации. Основной целью данной части дипломного проекта является разработка и обоснование конкретных направлений и мероприятий по повышению эффективности производственно-хозяйственной деятельности организации в соответствии с темой исследования.

Заключение. В заключении следует определить, какие задачи были решены, определить пути внедрения проекта и направления его дальнейшего совершенствования.

5. Перечень графического материала: **Диаграммы бизнес-процесса «Управление корпоративным обучением» в нотации IDEF0 (AS-IS и TO-BE), Ролевая модель доступа (RBAC — Role-Based Access Control), Диаграмма вариантов использования (UML Use Case Diagram), Логическая модель базы данных (ER-диаграмма), Диаграмма последовательности (UML Sequence Diagram), Схема шаблона проектирования Model-View-Controller (MVC) в контексте платформы обучения, Пример организации пользовательского интерфейса (UI/UX-макеты).** 65

6. Консультанты по дипломному проекту с указанием относящихся к ним разделов: нет

7. Примерный календарный график выполнения дипломного проекта

Наименование раздела, подраздела	Объем работы, %	Дата выполнения	Подпись руководителя
Введение	5	Февраль 2026	
Глава 1 Аналитическая часть	30	Февраль 2026	
Глава 2 Проектная часть	30	Март 2026	
Глава 3 Расчет экономической эффективности проекта	30	Апрель 2026	
Заключение	5	Май 2026	

8. Дата выдачи задания «27» февраля 2026 г.

9. Срок сдачи законченного дипломного проекта «04» мая 2026 г.

Руководитель _____ / _____ /

Обучающийся _____ / _____ /

Дата «27» февраля 2026 г.

РЕФЕРАТ

Пояснительная записка дипломного проекта содержит 81 страниц, 4 таблиц, 26 формул, 21 иллюстраций, 30 источников литературы.

ЦИФРОВАЯ ПЛАТФОРМА ОПТОВЫХ ЗАКУПОК, ВЕБ-ПРИЛОЖЕНИЕ, REACT.JS, NODE.JS, EXPRESS, POSTGRESQL, DRIZZLE ORM, REST API, JWT, REPLIT, АВТОМАТИЗАЦИЯ В2В-ПРОЦЕССОВ, ООО «СЕМЬ ДРАКОНОВ».

Объект дипломного проекта – цифровая платформа оптовых закупок товаров ООО «Семь драконов» и процессы взаимодействия предприятия с клиентами через цифровые каналы.

Предмет дипломного проекта – методы и инструменты проектирования и разработки корпоративных веб-платформ для автоматизации В2В-процессов оптовой торговли.

Цель дипломного проекта – разработка цифровой платформы оптовых закупок товаров для ООО «Семь драконов» с внедрением онлайн-каталога продукции, личного кабинета клиента, системы онлайн-заказов, чата поддержки и аналитической отчетности.

Методология работы. При выполнении проекта применял системный анализ деятельности предприятия, функциональное моделирование бизнес-процессов, объектно-ориентированное проектирование с ER- и UML-диаграммами, сравнение существующих программных решений и технико-экономическое обоснование принятых решений.

Результаты. Разобрана деятельность ООО «Семь драконов» – оптовая торговля детскими игрушками. Найдены слабые места: нет онлайн-каталога с фильтрами, нет личного кабинета клиента, нет онлайн-оформления заказов. Сравнил конкурентные решения, обосновал необходимость собственной разработки. Спроектировал клиент-серверную архитектуру на React.js + Node.js/Express + PostgreSQL. БД включает связанные таблицы пользователей, товаров, заказов, чатов и сообщений. Реализованы модули: каталог с поиском и фильтрацией, личный кабинет клиента, оформление и трекинг заказов, чат поддержки, админ-панель. Расчёты экономики: затраты на разработку – 23 927 BYN, годовая экономия для заказчика – 18 628 BYN, окупаемость – около 1,3 года.

Область применения. Платформа предназначена для использования в ООО «Семь драконов». Она автоматизирует работу с В2В-клиентами: оптовый каталог с ценовыми уровнями, оформление и сопровождение заказов, чат с менеджером, аналитика по продажам. Архитектура допускает рост – новые модули можно подключать без переписывания серверной части.

Подтверждаю: расчётно-аналитический материал, приведённый в работе, объективно отражает состояние объекта. Все заимствованные теоретические и методологические положения сопровождаются ссылками на авторов.

(подпись студента)

ОГЛАВЛЕНИЕ

СПИСОК ИСПОЛЬЗУЕМЫХ АББРЕВИАТУР И СОКРАЩЕНИЙ.....	8
ВВЕДЕНИЕ.....	9
ГЛАВА 1. АНАЛИТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ.....	13
1.1 Техничко-экономическая характеристика предметной области.....	13
1.1.1 Характеристика предприятия и подразделения.....	13
1.1.2 Экономическая сущность разрабатываемой системы.....	15
1.2 Анализ существующих решений и аналогов.....	16
1.2.1 Сравнительный анализ ключевых аналогов.....	16
1.2.2 Выводы по недостаткам существующих решений.....	17
1.3 Обоснование необходимости разработки.....	18
1.4 Постановка задач разработки.....	19
1.4.1 Цель и назначение системы.....	19
1.4.2 Функциональные и нефункциональные требования.....	20
1.5 Обоснование проектных решений.....	21
1.5.1 Техническое обеспечение.....	21
1.5.2 Программное обеспечение.....	23
1.5.3 Информационное обеспечение.....	25
1.5.4 Технологическое обеспечение.....	28
ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СИСТЕМЫ.....	31
2.1 Информационное и функциональное моделирование.....	31
2.1.1 Диаграмма вариантов использования.....	31
2.1.2 Диаграмма потоков данных.....	32
2.1.3 Диаграмма взаимодействия Frontend и Backend.....	33
2.2 Проектирование данных.....	39
2.2.1 Информационная модель.....	39
2.2.2 Характеристика базы данных.....	40
2.2.3 Описание нормативно-справочной, входной и результатной информации.....	42
2.3 Архитектура системы.....	43
2.3.1 Диаграмма компонентов.....	43

2.3.2 Диаграмма развертывания.....	45
2.4 Разработка программных модулей.....	46
2.4.1 Клиентский модуль.....	46
2.4.2 Серверный модуль.....	48
2.4.3 Модуль работы с данными.....	50
2.5 Технологическое обеспечение разработки.....	50
2.5.1 Процесс тестирования.....	50
2.5.2 Процесс развертывания.....	51
2.5.3 Мониторинг и логирование.....	52
ГЛАВА 3. РЕАЛИЗАЦИЯ, ТЕСТИРОВАНИЕ И ЭКСПЛУАТАЦИЯ.....	54
3.1 Руководство по установке и настройке.....	54
3.1.1 Системные требования.....	54
3.1.2 Пошаговая инструкция развертывания.....	55
3.1.3 Настройка окружения и интеграций.....	56
3.2 Описание контрольного примера.....	57
3.2.1 Сценарий использования системы.....	57
3.2.2 Демонстрация работы ключевых функций.....	59
3.3 Результаты тестирования.....	66
3.3.1 Покрытие и виды тестов.....	66
3.3.2 Выявленные и устраненные ошибки.....	70
ГЛАВА 4. ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ.....	73
4.1 Расчет затрат на разработку.....	73
4.1.1 Трудоемкость и стоимость работ.....	73
4.1.2 Затраты на ПО, оборудование, услуги.....	74
4.2 Оценка экономического эффекта.....	75
4.2.1 Для разработчика.....	75
4.2.2 Для заказчика	77
4.3 Показатели экономической эффективности.....	79
ЗАКЛЮЧЕНИЕ.....	80
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	83

СПИСОК ИСПОЛЬЗУЕМЫХ АББРЕВИАТУР И СОКРАЩЕНИЙ

ПО	–	программное обеспечение
СУБД	–	система управления базами данных
ТО	–	техническое обеспечение
ТехнО	–	технологическое обеспечение
ИО	–	информационное обеспечение
НДС	–	налог на добавленную стоимость
PL/SQL	–	процедурное расширение языка структурированных запросов (от англ. ProceduralLanguage/StructuredQueryLanguage)
TCP	–	протокол управления передачей (от англ. Transmission Control Protocol)
ПС	–	программное средство
ПП	–	программный продукт
PostgreSQL	–	открытая реляционная система управления базами данных

ВВЕДЕНИЕ

Согласно 68 СТ 34.003-90 «Информационная технология. Комплекс стандартов на автоматизированные системы. Термины и определения», 30 «автоматизированная система – система, состоящая из персонала и комплекса средств автоматизации его деятельности, реализующая информационную технологию выполнения установленных функций» 1]. Применительно к настоящей работе под автоматизированной системой 124 мается цифровая платформа оптовых закупок, обеспечивающая взаимодействие сотрудников ООО «Семь драконов» с клиентами и автоматизирующая операции приёма, обработки и сопровождения заказов.

Согласно 55 СТ 34.601-90 «Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания», 22 процесс создания автоматизированной системы включает следующие стадии: формирование требований к АС, разработка концепции АС, техническое задание, эскизный проект, технический проект, рабочая документация, ввод в действие, сопровождение АС» 1, с. 4]. Настоящий дипломный проект охватывает стадии формирования требований, разработки концепции, технического 17 рабочего проектирования платформы оптовых закупок ООО «Семь драконов».

Тема выбрана не случайно. ООО «Семь драконов» – оптовик в сегменте детских игрушек. Большая часть рутины (подбор позиций, согласование условий поставки, проверка остатков, оформление заявок) до сих пор крутится вокруг менеджеров и Excel. Это медленно, увеличивает нагрузку на персонал и периодически приводит к арифметическим ошибкам в счетах. Цифровой канал нужен – без него удерживать корпоративных клиентов в 2026 году становится всё сложнее [3, с. 24].

В соответствии с ГОСТ 34.003-90, «пользователь АС – лицо, участвующее в функционировании автоматизированной системы или использующее результаты её функционирования» 1]. В рамках разрабатываемой платформы пользователями выступают три категории: корпоративные клиенты (B2B-партнёры), менеджеры по работе с клиентами и администратор системы.

Согласно ГОСТ 34.602-89, 55 требования к функциям (задачам), выполняемым системой, должны включать перечень функций, задач или их комплексов, подлежащих автоматизации; временной регламент реализации каждой функции, задачи или их комплекса; требования к качеству реализации каждой функции (задачи), к форме представления выходной информации, характеристики необходимой точности и времени выполнения, требования 1

одновременности выполнения группы функций» 1 35. Указанные требования конкретизированы в разделе 1.4.2 настоящей работы.

Конкуренция на рынке детских товаров высокая. Оптовому клиенту важно за пару минут увидеть остатки, подобрать аналог, собрать корзину и получить счёт. Без личного кабинета, ценовых уровней под объём (5+/20+/50+) и быстрой связи с менеджером удержать партнёра трудно. Поэтому собственная платформа: индивидуальные условия для постоянных клиентов, объёмные скидки, контроль статуса заказа, аналитика для руководства.

Согласно 68 СТ 34.602-89 «Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы», «техническое задание на АС является основным документом, определяющим требования и порядок создания (развития или модернизации) автоматизированной системы, в соответствии с которым проводится её разработка и приёмка при вводе в действие» 22. 3]. Настоящий дипломный проект выполнен в соответствии с требованиями указанного стандарта к составу и содержанию проектной документации.

В ГОСТ 34.602-89 указано: «техническое задание на АС должно содержать следующие разделы: общие сведения; назначение и цели создания (развития) системы; характеристика объектов автоматизации; требования к системе; состав и содержание работ по созданию системы; порядок контроля и приёмки системы; требования к составу и содержанию работ по подготовке объекта автоматизации к вводу системы в действие; требования к документированию; источники разработки» 1, 62 4]. Перечисленные разделы учтены при формировании требований к разрабатываемой цифровой платформе.

Цель работы – разработать цифровую платформу оптовых закупок для организаций ООО «Семь драконов». Она должна закрыть весь клиентский цикл: коммуникация, оформление заказов, ведение каталога, поддержка пользователей, аналитика по продажам.

Для достижения поставленной цели необходимо решить следующие задачи:

- изучить деятельность ООО «Семь драконов» и выявить недостатки существующего порядка обработки оптовых заказов;
- провести анализ существующих цифровых решений и определить требования к разрабатываемой платформе;
- обосновать проектные решения по техническому, программному, информационному и технологическому обеспечению; 65
- спроектировать архитектуру системы, структуру базы данных и основные функциональные модули;
- реализовать клиентскую и серверную части веб-приложения с использованием React.js, Node.js/Express, PostgreSQL и Drizzle ORM;

- подготовить руководство по установке, настройке и эксплуатации системы;
- выполнить тестирование разработанной платформы и описать результаты контрольного примера;
- рассчитать экономическую эффективность внедрения системы для разработчика и заказчика.

Объектом дипломного проекта являются процессы взаимодействия ООО «Семь драконов» с корпоративными клиентами при оформлении оптовых закупок товаров.

Предметом дипломного проекта являются методы, модели и программные средства проектирования и разработки веб-платформы для автоматизации B2B-процессов оптовой торговли.

В соответствии с ГОСТ 34.602-89, «требования к АС – совокупность функциональных, технических, организационных и иных требований, которым должна соответствовать создаваемая автоматизированная система» [1, с. 5]. Требования к проектируемой платформе разделены на функциональные (состав реализуемых сценариев работы пользователей), нефункциональные (производительность, надёжность, безопасность) и интерфейсные (адаптивность, доступность с мобильных устройств).

Практическая значимость. Готовое решение можно сразу применить в компании: оно снижает трудоёмкость обработки заявок, делает работу с клиентами прозрачной, уменьшает количество ошибок и формирует единое информационное пространство для менеджеров, клиентов и администратора.

Согласно ГОСТ Р ИСО/МЭК 12207-2010, «обоснование выбора программных средств и среды разработки осуществляется на основе анализа функциональных и нефункциональных требований, а также с учётом ограничений по стоимости, срокам и квалификации исполнителей» [1, с. 18]. Выбор стека технологий для разрабатываемой платформы (React 18, TypeScript, Node.js/Express, PostgreSQL, Drizzle ORM, Vite, Tailwind CSS) обоснован указанным подходом и зафиксирован в техническом задании.

Согласно ГОСТ 34.003-90, «техническое обеспечение АС – совокупность всех технических средств, используемых при функционировании АС» [1]. В состав технического обеспечения разрабатываемой платформы входят серверы приложений, серверы баз данных, рабочие станции пользователей, активное сетевое оборудование и средства резервного копирования.

В соответствии с ГОСТ 34.201-89 «Информационная технология. Комплекс стандартов на автоматизированные системы. Виды, комплектность и обозначение документов при создании автоматизированных систем», состав документов на АС определяется в зависимости от стадии создания и видов обеспечения и включает документы общесистемные, организационные, функциональные, по информационному, программному, техническому, [1]

математическому, методическому и организационному обеспечениям» [12, с. 3]. Состав проектной документации настоящей работы соответствует указанному перечню.

Структура работы. Введение, четыре главы, заключение, список источников и приложение. Глава 1 – анализ предметной области и обоснование разработки. Глава 2 – проектные решения, архитектура, модель данных, программные модули. Глава 3 – руководство по установке, контрольный пример, итоги тестирования. Глава 4 – экономическое обоснование.

ГЛАВА 1

АНАЛИТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ

1.1 Техничко-экономическая характеристика предметной области

1.1.1 Характеристика предприятия и подразделения

Как указано в ГОСТ Р ИСО/МЭК 15288-2005 «Информационная технология. Системная инженерия. Процессы жизненного цикла систем», «организация – группа людей и средств с распределением ответственности, полномочий и взаимоотношений» [1 с. 5]. Данное определение применимо к структуре управления ООО «Семь драконов»: подразделения компании имеют чётко определённые функциональные обязанности и взаимодействуют по регламентированным бизнес-процессам.

В соответствии с ГОСТ 34.003-90, программное обеспечение АС – совокупность программ на носителях данных и программных документов, предназначенная для отладки, функционирования и проверки работоспособности АС» [1]. Программное обеспечение проектируемой платформы включает прикладное программное обеспечение, ответственное за разработку, общесистемное программное обеспечение (операционная система, СУБД, PostgreSQL, среда выполнения Node.js) и библиотеки сторонних разработчиков, распространяемые на условиях открытых лицензий.

ООО «Семь драконов» – белорусская оптовая компания. Основной профиль – детские игрушки и сопутствующие товары. Базируется в Минске. Поставки идут по всей республике: розничные магазины, торговые сети, региональные точки. С момента основания компания делает ставку на проверенные бренды и долгие отношения с поставщиками – это позволило закрепиться в оптовой дистрибуции.

В соответствии с ГОСТ 34.601-90, стадии формирования требований к АС выполняются следующие этапы: обследование объекта и обоснование необходимости создания АС, формирование требований пользователя к АС, оформление отчёта о выполненной работе и заявки на разработку АС» [1 35]. Обследование деятельности ООО «Семь драконов» проведено в ходе преддипломной практики и положено в основу аналитической части настоящей работы.

Структура «Семи драконов» покрывает весь цикл работы с клиентом: приём заявки, контроль остатков, комплектация, логистика, сопровождение

сделки. Схема подразделений, отвечающих за ключевые процессы, приведена на рисунке 1.1.

Основные направления деятельности

- Оптовые поставки игрушек и детских товаров от отечественных и зарубежных производителей.
- Формирование и поддержание широкого ассортимента: классические игрушки, развивающие игры, настольные комплекты, товары для творчества.
- Комплексное складское обслуживание: хранение, учёт, комплектация партий, подготовка заказов к отгрузке.
- Организация доставки по Минску и регионам, включая взаимодействие с транспортными компаниями и контроль сроков.
- Информационная и консультационная поддержка партнёров, помощь в подборе товарных позиций и формировании заказов.

Клиентура – розничные магазины, интернет-ритейлеры, торговые сети, индивидуальные предприниматели. Компания работает по гибкому ценообразованию и готова формировать индивидуальные коммерческие предложения для долгосрочных партнёров. За счёт этого удерживается конкурентоспособность и лояльность заказчиков.

Конкурентные преимущества

- Отлаженные бизнес-процессы, минимизирующие сроки обработки заказов и отгрузки.
- Современная складская инфраструктура с возможностью внедрения ИО и автоматизированных систем учёта.
- Ассортимент, адаптированный под потребности различных сегментов рынка.
- Надёжные деловые связи с производителями, что гарантирует качество поставляемой продукции.

Структура ООО «Семь драконов» представлена на рисунке 1.1.



Рисунок 1.1 – Структура организации ООО «Семь драконов»

Примечание – Источник – Собственная разработка.

1.1.2 Экономическая сущность разрабатываемой системы

Согласно 4 СТ Р ИСО/МЭК 12207-2010 «Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств», 24 процесс жизненного цикла программных средств – совокупность взаимосвязанных или взаимодействующих видов деятельности, связанных с заказом, поставкой, разработкой, эксплуатацией и сопровождением программного продукта» 1, с. 7]. Цифровая платформа оптовых закупок проходит полный цикл – от формирования концепции и анализа требований до разработки, ввода в эксплуатацию и сопровождения.

Платформа автоматизирует приём заявок, ведение каталога и трекинг отгрузок. Главная задача – снизить операционные затраты и поднять выручку за счёт ускорения обработки заявок и прозрачной коммуникации с клиентами.

В соответствии с РД 50-34.698-90 «Автоматизированные системы. Требования к содержанию документов», 22писание автоматизируемых функций должно содержать перечень функций, задач или их комплексов (в том числе обеспечивающих взаимодействие частей АС), подлежащих автоматизации, и при создании АС в нескольких очередях – сведения об их распределении по очередям» 1. В разрабатываемой платформе автоматизации подлежат функции ведения каталога, оформления заказа, расчёта стоимости с учётом оптовых скидок, обмена сообщениями с менеджером и формирования аналитических отчётов.

По экономическому смыслу решение ближе к нишевому B2B-маркетплейсу или к конфигуратору наборов: клиент собирает партию из нужных позиций и сразу получает актуальную цену. В интерфейсе защиты бизнес-правила (ценовые уровни, минимальные партии, остатки на складе) – благодаря этому коммерческие предложения формируются быстро и без расхождений.

Экономический эффект достигается за счёт:

- сокращения трудозатрат на ввод заказов и ручную обработку сопроводительной документации
- уменьшения числа ошибок при расчёте цен и проверке наличия товара
- ускорения цикла «заказ – сборка – отгрузка», что положительно влияет на движение денежных средств
- роста клиентской лояльности и увеличения объёма кросс-продаж благодаря персонализированным рекомендациям
- получения централизованной аналитики для оптимизации маркетинговых и складских расходов

Ожидаемый эффект – рост выручки за счёт более высокой конверсии заказов, сокращение операционных издержек и улучшение финансовых показателей «Семи драконов» как в краткосрочной, так и в долгосрочной перспективе.

1.2 Анализ существующих решений и аналогов

1.2.1 Сравнительный анализ ключевых аналогов

Таблица 1.1 – Сравнительный анализ ключевых аналогов

Сеть	Формат	Каналы продаж	Цифровые функции	Лояльность и USP	Примечание для «Семь драконов»
Buslik	Оффлайн + онлайн	Сайт, мобильное приложение, 80+ магазинов	Click&Collect, курьерская доставка, личный кабинет, push-уведомления	Бонусная программа, персональные подборки	Пример мультимодальной логистики и WMS-интеграции
KariKids	Оффлайн + онлайн	Сайт, 5 магазинов в Минске	Онлайн-заказ с учётом примерки, отображение остатков	Накопительные скидки, стилистические консультации	Модель локального ритейлера с CRM-коннекторами
FUNtastik	Оффлайн + онлайн	Сайт, офлайн-бутики	Электронный каталог, самовывоз, доставка курьером	Промо-коды, сезонные подборки	Аналог управления ассортиментом и логистикой
Детский мир	Онлайн	Маркетплейс, собственный веб-магазин	Конфигуратор подарков, мгновенный расчёт стоимости	Эксклюзивные брендовые коллекции, быстрая доставка по Беларуси	Демонстрация сложной прайс-модели и SLA-контроля
Mothercare	Оффлайн + онлайн	Сайт, флагманский магазин в Минске	Онлайн-заказ, pick-up point	Бонусные рубли, мероприятия и семинары для родителей	Полезный кейс по управлению сезонными остатками
Платформа ООО «Семь драконов»	B2B / оптовая торговля	Веб-портал, API-интеграция с ERP, WMS	Автооформление заказов, конфигуратор наборов, SLA-мониторинг	Персонализированные B2B-коммерческие предложения, контроль SLA	Фокус на wholesale-логистике игрушек и отчетности

Примечание: Источник – Собственная разработка.

При сравнении ключевых цифровых решений детских магазинов Беларуси раскрываются следующие преимущества у каждого игрока рынка.

Buslik работает по омниканальной модели: 80+ офлайн-точек дополнены сайтом и мобильным приложением с Click&Collect, курьерской доставкой и личным кабинетом. Клиент выбирает удобную схему получения, оператор управляет запасами через WMS и мобильные сканеры на складе.

Kari Kids делает ставку на UX вокруг примерки. В онлайн можно заранее проверить размеры в конкретном магазине, выбрать «примерку перед оплатой» и оформить заказ под этот сценарий. Механика снижает возвраты и поднимает конверсию в офлайне.

FUNtastik – большой электронный каталог, разбитый по темам, наборам и подарочным акциям. На сайте – самовывоз и курьерская доставка, удобный фильтр аксессуаров, система промо-кодов и сезонные подборки.

«Детский мир» – конфигуратор подарков: клиент собирает набор из разных категорий, видит стоимость и доступность по регионам в режиме реального времени. Ускоренная доставка по Беларуси и SLA-контроль доставки «до двери» формируют ощущение надёжного сервиса.

Mothercare сочетает офлайн-экспертизу с программой лояльности – бонусные рубли, тематические события, консультации для родителей. Пик-ап-поинты и онлайн-заказ дополняют формат флагмана, грамотное управление сезонными остатками снижает уценку.

1.2.2 Выводы по недостаткам существующих решений

Анализ цифровых платформ конкурентов показал: ни одно из решений полностью не закрывает потребности оптового бизнеса «Семи драконов». Большинство сервисов – про B2C либо про упрощённый B2B, а крупному дистрибьютору нужны более глубокие возможности кастомизации и интеграции.

Основные недостатки конкурирующих решений:

- Ограниченная B2B-поддержка Большинство платформ не умеют формировать индивидуальные коммерческие предложения, не предлагают гибкий учёт объёмных заказов и отсутствуют механизмы автоматического расчёта скидок по уровням партнёров.

- Отсутствие сквозного SLA-мониторинга Решения не дают возможности отслеживать этапы «заказ – сборка – отгрузка» в реальном времени и не уведомляют о нарушении целевых сроков на каждом этапе.

– Недостаточная интеграция с WMS/ERP/CRM Зачастую функционал складывается из базовых API или сторонних модулей, что приводит к точечным доработкам с длительным сроком реализации.

– Слабая аналитическая отчётность Аналитические инструменты ограничены отчётами по продажам и клиентам, не покрывают аналитику остатков, рентабельности по сегментам и прогнозирование спроса.

– Низкая гибкость ценового конфигуратора У большинства аналогов нет удобных инструментов для быстрого формирования комплектов товаров с учётом остатка на складе и оптовых условий.

– Высокие затраты на внедрение и поддержку Решения с развитым BPM-движком или богатой API требуют значительных инвестиций в лицензии и участие сертифицированных консультантов.

– Ограниченный мобильный и офлайн-функционал Отсутствие мобильных приложений для менеджеров и нерешённые задачи офлайн-продаж усложняют оперативную обработку заказов на выезде и врозницу.

По итогу, для «Семь драконов» критически важно разработать платформу с полной поддержкой, сквозным мониторингом, модульной интеграцией с ключевыми системами, гибким конфигуратором наборов и встроенной аналитикой.

1.3 Обоснование необходимости разработки

В современных условиях оптовой торговли игрушками ООО «Семь драконов» сталкивается с расширяющимся портфелем клиентов, усложнённым ассортиментом и возросшими требованиями к скорости обслуживания. Типовые решения, ориентированные на B2C или упрощённое B2B, не позволяют обеспечить необходимый уровень гибкости, прозрачности и контроля, что приводит к:

- высоким трудозатратам на ручную обработку заказов и документов;
- рискам ошибок при расчёте цен и проверке остатков;
- замедлению цикла «заказ – сборка – отгрузка»;
- недостаточной скорости реакции на нестандартные запросы крупных партнёров.

Одновременно усиливается конкурентное давление со стороны как локальных, так и международных дистрибьюторов, внедряющих собственные цифровые платформы. Для сохранения и укрепления позиций на рынке необходимо:

– Автоматизировать полноценный B2B-цикл с возможностью формирования индивидуальных коммерческих предложений и гибким SLA-мониторингом.

– Интегрировать веб-портал с существующими ERP, WMS и CRM-системами без точечных доработок, обеспечив единое информационное пространство.

– Внедрить конфигуратор товарных комплектов, учитывающий остатки на складах и оптовые скидки, для мгновенного расчёта и оформления заказов.

– Обеспечить централизованную BI-аналитику по продажам, рентабельности и прогнозированию спроса, чтобы оперативно корректировать маркетинговые и операционные стратегии.

Разработка собственной платформы позволит ООО «Семь драконов» отказаться от компромиссных решений, устранить узкие места в процессах, минимизировать зависимость от сторонних вендоров и гибко масштабировать функциональность в соответствии с растущими потребностями бизнеса.

1.4 Постановка задач разработки

1.4.1 Цель и назначение системы

Система призвана стать центральным цифровым ядром, объединяющим все ключевые процессы оптовой торговли ООО «Семь драконов». Она обеспечит единый интерфейс для оформления заказов, управления ассортиментом и контроля логистических операций, что снимет необходимость переключаться между разрозненными приложениями и значительно упростит работу как внутренних пользователей, так и клиентов компании. В основе платформы лежит гибкая архитектура, способная адаптироваться под растущие потребности бизнеса и новые сценарии взаимодействия с партнёрами.

Одной из главных функциональных задач станет автоматизированный конфигуратор товарных комплектов. Клиенты получают возможность собирать заказы из различных категорий продукции, сразу видя актуальные остатки на складах, стоимость с учётом оптовых скидок и варианты доставки. Такой подход не только ускорит процесс формирования коммерческих предложений, но и снизит количество ошибок, присущих ручному подбору и многоступенчатым согласованиям.

Полный цикл «заказ – сборка – отгрузка» будет полностью прозрачен: система будет фиксировать каждый этап выполнения заявки, автоматически рассчитывать SLA-показатели и оповещать ответственных лиц о возможных

нарушениях сроков. Это позволит менеджерам оперативно вмешиваться в процесс при возникновении задержек и обеспечит соблюдение обязательств перед ключевыми партнёрами компании. Переход от ручного контроля к автоматизированному мониторингу статусов сделает работу более предсказуемой и сократит риски возникновения «узких мест».

Реализация данной платформы обеспечит ощутимое снижение операционных расходов за счёт минимизации ручного ввода и рутинных согласований, повысит скорость денежного цикла за счёт ускоренного обслуживания заказов и укрепит лояльность ключевых клиентов благодаря соблюдению прозрачным условиям работы. При этом масштабируемая архитектура и поддержка современных веб-технологий создадут фундамент для дальнейшего развития системы без значительных дополнительных затрат на IT-ресурсы.

1.4.2 Функциональные и нефункциональные требования

Функциональные и нефункциональные требования к будущей платформе для ООО «Семь драконов» можно развернуть в более широкое, комплексное описание, чтобы подчеркнуть масштаб проекта и важность каждого элемента.

Платформа должна стать единым цифровым центром, охватывающим все этапы взаимодействия с клиентами и внутренними подразделениями. Её функциональность начинается с гибкого управления пользователями: система будет поддерживать регистрацию, аутентификацию, назначение ролей и настройку прав доступа таким образом, чтобы каждая категория пользователей – от менеджера по продажам до кладовщика и клиента – имела чётко определённый набор инструментов. Это позволит исключить избыточный доступ, усилить безопасность и ускорить работу за счёт персонализированных интерфейсов.

Процесс оформления и обработки заказов станет полностью автоматизированным. Менеджер сможет создать заказ вручную или через интуитивный конфигуратор товарных наборов, который в реальном времени покажет остатки, рассчитает стоимость с учётом скидок, валютных курсов и акций, а также предложит альтернативы при дефиците товаров. Многоуровневая система согласований обеспечит контроль ценовых условий, доступности и сроков поставки, а история заказов позволит быстро анализировать прошлые сделки и повторять успешные конфигурации.

Отдельный модуль будет отвечать за управление прайс-листами и скидками, включая сезонные акции, персональные условия для ключевых клиентов и автоматическое пересчитывание цен при изменении условий

поставок или рыночных факторов. SLA-мониторинг станет важной частью платформы: он не только проконтролирует соблюдение сроков на каждом этапе цикла «заказ – сборка – отгрузка», но и вовремя предупредит о рисках срыва обязательств через встроенную систему уведомлений.

С точки зрения интеграций, система будет «общаться» с ERP для обмена данными о номенклатуре и остатках, с WMS – для передачи заданий на сборку и контроля перемещений на складе, и с CRM – для управления клиентской базой и отслеживания истории взаимодействий. Все эти интеграции будут реализованы через стандартизированные API, что упростит подключение новых модулей и снизит стоимость поддержки. BI-подсистема соберёт данные из всех точек и предоставит руководству аналитические отчёты и прогнозы, позволяя мгновенно оценивать эффективность продаж и корректировать стратегию.

В нефункциональной части платформа должна быть высокой производительности, способной обрабатывать тысячи заказов в сутки без потери скорости отклика. Уровень доступности планируется не ниже 99,9 %, при этом будет реализовано горячее резервирование и регулярное бэкапирование данных. Безопасность обеспечат шифрование трафика, защищённое хранение конфиденциальных данных и регулярные тесты на уязвимости. Интерфейс будет адаптивным, многоязычным и соответствующим стандартам доступности, что позволит использовать систему на разных устройствах и в разных условиях.

Модульная архитектура и современный технологический стек обеспечат лёгкое масштабирование и развитие платформы по мере роста бизнеса. Таким образом, это решение не просто автоматизирует существующие процессы, но и создаст основу для дальнейших цифровых инициатив «Семь драконов», обеспечивая компании конкурентное преимущество на долгие годы.

1.5 Обоснование проектных решений

1.5.1 Техническое обеспечение

Техническое обеспечение (ТО) платформы учёта и автоматизации для ООО «Семь драконов» – это комплекс аппаратных средств, сетевого оборудования и инженерной инфраструктуры, обеспечивающих стабильную и безопасную работу ПО, СУБД и сервисных модулей системы. Основная задача ТО – гарантировать требуемый уровень производительности,

отказоустойчивости и информационной безопасности при работе в режиме 24/7.

Серверная инфраструктура

– Веб-сервер

- CPU: Intel Xeon E5-2680 v4 (14 ядер, 2.4 ГГц) или аналог • ОЗУ: 32 ГБ DDR4

- Накопитель: SSD ≥ 1 ТБ Обоснование: высокая многопоточность и большой объём памяти позволяют обрабатывать сотни параллельных запросов, кэшировать данные и снижать время отклика платформы.

– Сервер СУБД

- CPU: Intel Xeon Gold 6248 (20 ядер, 2.5 ГГц) или аналог

- ОЗУ: 64 ГБ DDR4

- Накопитель: SSD ≥ 2 ТБ в RAID 1 Обоснование: высокая многопоточность и объём памяти критичны для сложных транзакций, формирования аналитических отчётов и работы с большими наборами данных; RAID 1 обеспечивает отказоустойчивость.

– Канал связи

- Выделенная линия ≥ 1 Гбит/с с резервированием Обоснование: стабильный высокоскоростной канал исключает узкие места в интеграциях с WMS/CRM, ускоряет синхронизацию складских операций и обеспечивает пиковую нагрузку без деградации.

– Система бесперебойного питания (ИБП)

- Мощность ≥ 10 кВА Обоснование: предотвращает аварийную остановку систем и потерю данных при перебоях электроснабжения.

– Требования к клиентским устройствам

– Процессор

- ОЗУ: Intel Core i3 или AMD Ryzen 3 и выше, ОЗУ ≥ 4 ГБ Обоснование: обеспечивает быструю работу интерфейса WMS-модуля, личного кабинета и мобильных приложений для склада.

– Дисковое пространство: ≥ 100 МБ свободного места

- Обоснование: для хранения временных файлов, офлайн-кэша и локальных логов.

– Сетевое подключение: широкополосный Интернет

- Обоснование: гарантирует синхронность данных с сервером.

– Поддерживаемые браузеры

- Chrome 90+

- Firefox 88+

- Safari 14+

- Edge 90+

- Сетевое оборудование
- Коммутаторы уровня L3 с пропускной способностью ≥ 48 Гбит/с
 - Обоснование: агрегация и сегментация трафика, поддержка VLAN для изоляции складской, офисной и гостевой сетей.
- Маршрутизаторы с поддержкой BGP/OSPF
 - Обоснование: динамическая маршрутизация и балансировка каналов при сбоях.
- Межсетевые экраны с производительностью ≥ 10 Гбит/с
 - Обоснование: глубокая инспекция пакетов, защита от DDoS и атак на периметр.
- Точки доступа Wi-Fi стандарта 802.11ac/ax
 - Обоснование: высокоскоростной доступ на складе и в офисе, поддержка большого количества одновременных подключений.

1.5.2 Программное обеспечение

Программное обеспечение (ПО) разрабатываемой системы представляет собой взаимосвязанную совокупность системных, прикладных, интеграционных и сервисных компонентов, ориентированных на автоматизацию ключевых бизнес-процессов оптовой торговли и логистики. Оно реализует предметную область проекта, поддерживает внутренние и внешние интеграции, а также обеспечивает удобный, безопасный и производительный доступ пользователей к функционалу системы.

Серверный сегмент формирует ядро бизнес-логики и отвечает за обработку клиентских запросов, интеграцию с внешними модулями и управление данными. Архитектура выстроена на основе асинхронной среды выполнения, что позволяет обслуживать сотни и тысячи одновременных соединений с минимальными задержками. Это особенно важно для задач реального времени: синхронизации складских операций, обработки заказов, обновления аналитики.

Ключевые особенности серверной части:

- Статическая типизация обеспечивает строгий контроль структуры данных на всех этапах разработки, снижает риск логических ошибок и упрощает масштабирование кода.
- Модульная архитектура позволяет разрабатывать, тестировать и внедрять отдельные функциональные блоки независимо, что ускоряет внедрение новых возможностей.

– Слой маршрутизации и middleware служит для централизованной обработки запросов – валидация данных, аутентификация, ведение логов, сжатие и кэширование ответов.

– ORM-уровень обеспечивает безопасную работу с БД: автоматическую генерацию миграций, контроль типов, оптимизацию SQL-запросов и защиту от инъекций.

– Управление сессиями реализовано с использованием устойчивого к сбоям хранилища, что позволяет поддерживать авторизацию в кластере серверов и сохранять состояние пользователей при балансировке нагрузки.

– Интеграционный слой обрабатывает двусторонний обмен с бухгалтерскими системами, сервисами доставки, маркетплейсами и модулями мониторинга.

Пользовательский интерфейс реализован как одностраничное приложение, что обеспечивает мгновенную навигацию и динамическое обновление данных без полной перезагрузки страниц. Декларативный подход и виртуальный DOM гарантируют высокую отзывчивость даже при больших объёмах отображаемой информации.

Отличительные черты клиентской части:

– Единая система типов между сервером и клиентом исключает несогласованность форматов данных.

– Утилитарная система стилей и библиотека готовых компонентов позволяют быстро собирать сложные интерфейсы, сохраняя при этом единый фирменный стиль.

– Минимальный и оптимизированный роутер обеспечивает плавную работу навигации без нагруженности приложения лишними зависимостями.

– Адаптивный дизайн гарантирует корректное отображение на стационарных ПК, ноутбуках, планшетах и мобильных устройствах.

В качестве хранилища применяется промышленная реляционная СУБД с полной поддержкой ACID, оптимизированной системой индексов, механизмами репликации и восстановления после сбоев.

Функциональные преимущества:

– Поддержка сложных транзакций с высокой степенью параллелизма.

– Гибридные структуры данных – реляционные таблицы и JSON-поля для хранения полуструктурированной информации.

– Горизонтальное масштабирование за счёт шардинга и репликации.

– Механизмы резервного копирования для обеспечения целостности и восстановления данных.

Ключевые функциональные модули клиентской стороны:

– Аутентификация, авторизация.

- Формирование, редактирование и отслеживание заказов.
- Просмотр и анализ показателей продаж, остатков.
- Поддержка клиентов.
- Интерактивное отображение статусов заказов.

Ключевые функциональные модули серверной стороны:

- RESTful API для всех операций управления данными и взаимодействия с внешними системами.

- Ролевое разграничение доступа и централизованная авторизация.
- Система логирования и мониторинга для проактивного выявления проблем и оптимизации производительности.

Обеспечение безопасности:

- Шифрование трафика между всеми компонентами.
- Хранение конфиденциальных данных с использованием алгоритмов симметричного шифрования.

- Валидация и фильтрация данных на всех уровнях (сервер и клиент).

Эксплуатационные и архитектурные аспекты:

- Микросервисный подход облегчает масштабирование отдельных частей системы под нагрузкой.

- Контейнеризация обеспечивает идентичность сред разработки, тестирования и эксплуатации.

- Мониторинг (метрики, логи, алерты) обеспечивает предсказуемость работы и высокую доступность.

1.5.3 Информационное обеспечение

Согласно статье 1 Закона Республики Беларусь от 7 мая 2021 г. № 99-З «О защите **27** ональных данных», «персональные данные – любая информация, относящаяся к идентифицированному физическому лицу или физическому лицу, которое может быть идентифицировано» **34** с. 2]. К таким данным в системе ООО «Семь драконов» относятся фамилия, имя и отчество контактного лица, адрес электронной почты, номер телефона и реквизиты компании-клиента, обработка которых осуществляется в строгом соответствии с требованиями указанного закона.

В статье 6 Закона Республики Беларусь от 7 мая 2021 г. № 99-З закреплено: **27** обработка персональных данных должна осуществляться с соблюдением принципов законности, справедливости и прозрачности; объём обрабатываемых персональных данных должен быть минимально необходимым для достижения заявленных целей обработки» **1**, с. 5]. В

платформе данный принцип реализован через механизм осознанного согласия пользователя на обработку данных при регистрации и минимизацию состава хранимых полей профиля.

Согласно статье 12 Закона Республики Беларусь от 7 мая 2021 г. № 99-З, 71
«субъект персональных данных имеет право на получение информации, касающейся обработки его персональных данных, содержащей наименование (фамилию, собственное имя, отчество – если таковое имеется) и место нахождения (адрес места жительства) оператора, подтверждение факта обработки персональных данных оператором, его персональные данные и источник их получения, правовые основания и цели обработки персональных данных, срок, на который дано его согласие» 1 85 Платформа обеспечивает реализацию указанных прав через интерфейс личного кабинета пользователя.

Информационное обеспечение корпоративной платформы ООО «Семь драконов» – это комплексная совокупность данных, методов и регламентов, обеспечивающих поддержку всех бизнес-процессов предприятия в цифровой среде. Оно охватывает как содержательную сторону (состав и структуру данных), так и организационную (правила их формирования, обновления, хранения, передачи и защиты). По сути, это «информационный фундамент» компании, позволяющий автоматизировать ключевые функции и выстроить единое информационное пространство между подразделениями.

– Пользовательские данные

○ Состав данных: В систему заносятся и системно обрабатываются сведения о клиентах и партнёрах, прошедших регистрацию. В их числе:

- E-mail (основной идентификатор для входа и коммуникаций)
- Название компании (юридическое и/или коммерческое)
- ФИО директора
- Телефон (рабочий или мобильный)
- Почтовый адрес (для выставления счетов и доставки документов)

○ Назначение: Эти данные выполняют роль базы идентификации и аутентификации участников системы, лежат в основе документооборота.

○ Защита и разграничение прав: Реализована двухуровневая авторизация:

- Пользователи – имеют доступ к личному кабинету, заказам, коммуникационному модулю.
- Администраторы – управляют ассортиментом, заказами, учетными записями, настройками и интеграциями.

– Товарная номенклатура

- Состав данных: Каталог продукции содержит полный набор характеристик по каждой позиции:
 - Наименование товара
 - Подробное описание
 - Категория
 - Возрастная группа
 - Материал
 - Страна производства
 - Изображение (фотография)
 - Трёхуровневая модель ценообразования:
 - 5+ единиц
 - 20+ единиц
 - 50+ единиц
 - Статус наличия (в наличии, под заказ, отсутствует)
- Назначение: Товарная база служит центральным информационным ресурсом, который используют:
 - Клиенты – для выбора и оформления заказов
 - Менеджеры – для ведения каталога и расчёта цен
 - Склад – для синхронизации остатков
 - Отдел маркетинга – для подготовки рекламных материалов
 - Особенности: Синхронизация с системой управления складом (WMS) обеспечивает актуальность статусов наличия. При изменении остатков обновление отражается в каталоге в режиме реального времени. Алгоритмы ценообразования позволяют автоматически пересчитывать стоимость в корзине, стимулируя оптовые закупки.
- Заказная информация
 - Состав данных по каждому заказу:
 - Перечень товаров с указанием количества
 - Общая стоимость (с учётом ценового уровня и возможных скидок)
 - Адрес доставки
 - Статус обработки заказа:
 - Новый
 - В обработке
 - Отправлен
 - Доставлен
 - Планируемая дата доставки
 - Назначение и обработка: Эти сведения обеспечивают прозрачность всего цикла обработки заказа. На каждом этапе обновляется статус, что

позволяет клиенту отслеживать движение своей заявки, а менеджерам – планировать загрузку склада и логистики.

1.5.4 Технологическое обеспечение

Технологическое обеспечение разрабатываемой веб-платформы для ООО «Семь драконов» представляет собой комплекс современных технологий, инструментов и библиотек, которые формируют основу клиентской и серверной части системы, обеспечивают её надёжность, производительность, расширяемость и удобство использования. 109 вильно подобранный технологический стек позволяет создать продукт, отвечающий как текущим, так и перспективным потребностям бизнеса, а также упрощает последующее сопровождение и развитие.

В основе пользовательского интерфейса лежит современная библиотека для построения веб-интерфейсов с декларативным подходом к описанию компонентов. Использование актуальной версии этой библиотеки в сочетании с языком программирования, поддерживающим строгую статическую типизацию, позволяет:

- минимизировать количество ошибок на этапе разработки;
- обеспечить предсказуемость поведения кода;
- упростить совместную работу команды над крупными модулями;
- реализовать повторно используемые, модульные интерфейсные компоненты.

Сборка и запуск фронтенд-приложения выполняются с помощью высокопроизводительного инструмента разработки, обеспечивающего мгновенный запуск проекта (coldstart), поддержку «горячей» перезагрузки модулей (HMR) и минимальное время отклика во время изменения кода. Это ускоряет цикл разработки, тестирования и внедрения изменений, что особенно важно для динамично развивающегося проекта.

Для управления серверным состоянием и синхронизации данных между клиентом и API используется специализированная библиотека, оптимизирующая запросы, кэширование и повторное использование данных. Это позволяет:

- уменьшить количество избыточных обращений к серверу;
- моментально отображать актуальную информацию при повторных визитах пользователя;
- эффективно обрабатывать ошибки и состояния загрузки.

Маршрутизация внутри одностраничного приложения (SPA) реализована при помощи лёгкого клиентского роутера, что даёт возможность перемещаться между разделами без полной перезагрузки страницы и снижает нагрузку на сервер.

Для унификации внешнего вида и ускорения разработки используется утилитарный CSS-фреймворк, позволяющий создавать адаптивную и согласованную дизайн-систему без написания избыточных каскадных стилей. В связке с ним применяется библиотека готовых компонентов на основе Radix UI, что обеспечивает:

- доступность интерфейса (соответствие стандартам WCAG);
- предсказуемое поведение элементов управления;
- современный внешний вид и поддержку тёмной/светлой тем.

Это решение позволяет не только экономить время разработчиков, но и гарантирует единообразие пользовательского опыта на всех устройствах.

Серверная логика реализована на основе минималистичного веб-фреймворка, работающего в связке с TypeScript. Такой выбор обеспечивает:

- гибкую маршрутизацию HTTP-запросов;
- поддержку middleware-архитектуры для централизованной обработки ошибок, логирования, аутентификации и авторизации;
- простоту интеграции с внешними API и микросервисами.

Для работы с загружаемыми пользователями файлами (изображения товаров, документы и т.д.) используется специализированная библиотека, позволяющая эффективно обрабатывать файлы в памяти или на диске, а также накладывать ограничения на их размер и формат.

Вся бизнес-логика сервера модульно структурирована, что облегчает масштабирование приложения и внедрение новых функций без затрагивания существующих модулей.

В качестве основной системы управления базами данных используется надёжная реляционная СУБД с полной поддержкой транзакций, оптимизированными индексами и возможностью горизонтального масштабирования. Для доступа к БД применяется ORM-слой, который:

- обеспечивает полную типобезопасность запросов;
- автоматически генерирует миграции схемы данных;
- снижает риск SQL-инъекций;
- упрощает рефакторинг структуры данных.

Подключение к базе реализовано через облачный serverless-провайдер с пулом соединений, что позволяет:

- автоматически масштабировать ресурсы под нагрузку;
- оптимизировать время отклика запросов;

- снижать затраты на инфраструктуру за счёт динамического распределения вычислительных мощностей.

Технические и архитектурные преимущества выбранного стека:

- Производительность и масштабируемость – благодаря асинхронной обработке запросов на сервере, оптимизированной работе с БД и эффективному управлению состоянием на клиенте.

- Кроссплатформенность – интерфейс корректно работает в современных браузерах на любых устройствах, включая мобильные.

- Расширяемость – модульная архитектура позволяет легко добавлять новые компоненты и модули.

- Поддержка командной разработки – строгая типизация и единые стандарты кодирования ускоряют взаимодействие между разработчиками.

- Доступность и UX-качество – за счёт использования UI-фреймворка, ориентированного на доступность и адаптивный дизайн.

- Интеграционная готовность – API и архитектура бэкенда позволяют подключать внешние сервисы и корпоративные системы без кардинальной переработки кода.

ГЛАВА 2

ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СИСТЕМЫ

2.1 Информационное и функциональное моделирование

2.1.1 Диаграмма вариантов использования (Use Case)

Диаграмма вариантов использования – это один из видов поведенческих диаграмм UML, который демонстрирует функциональные требования к системе через “варианты использования” (UseCases) **65** внешних участников (Actors). Представлена она на рисунке 2.1

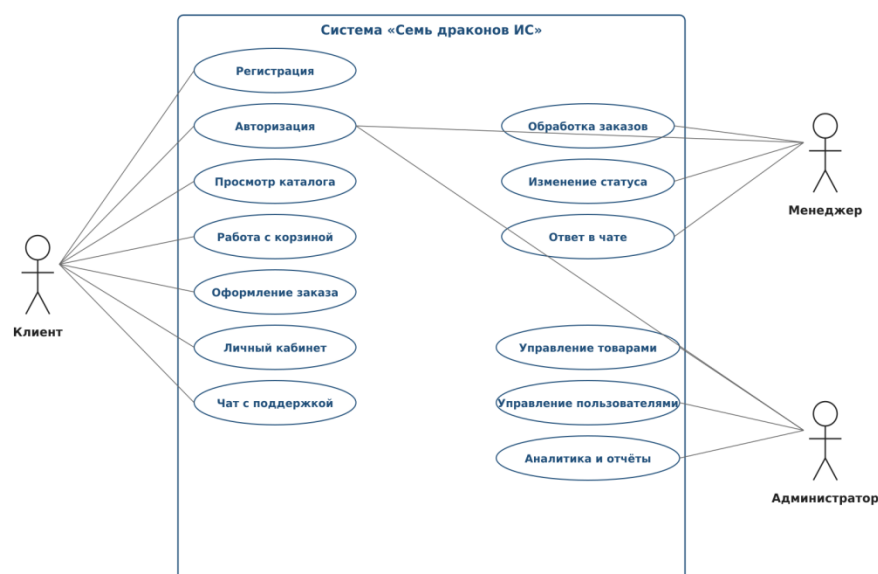


Рисунок 2.1 – Диаграмма вариантов использования

Примечание – Источник – Собственная разработка.

В соответствии с ГОСТ 34.003-90, «функция АС – совокупность действий автоматизированной системы, направленная на достижение определённой цели» **1** [1]. Каждый описываемый ниже вариант использования соответствует отдельной функции АС в трактовке стандарта и реализуется конкретным набором программных компонентов клиентской и серверной частей системы.

В ГОСТ Р ИСО/МЭК 12207-2010 указано: **6** процесс анализа требований к программным средствам – процесс, цель которого состоит в установлении требований к программным компонентам системы. В ходе процесса анализируются и документируются функциональные и нефункциональные **1**

требования, требования к интерфейсам, требования к качеству, требования к безопасности и иные требования, существенные для разработки программного средства» **1** с. 24]. Описанная далее модель вариантов использования является результатом анализа требований к платформе оптовых закупок.

Сценарий покупателя такой. Сначала – регистрация личного кабинета. Далее доступен поиск по каталогу, фильтрация по категориям и возрасту, просмотр карточек товара, добавление в корзину. Любой непонятный момент клиент уточняет через встроенный чат. При оформлении заказа система требует авторизацию и сразу заводит чат-канал с продавцом – это удобно для обсуждения деталей конкретной заявки. Администратор работает по другому контуру. Он управляет учётками, ведёт каталог, обрабатывает заказы, меняет их статусы, назначает дату доставки и смотрит сводную аналитику. Диаграмма наглядно показывает разграничение прав и связи между сценариями (например, «Оформить заказ» зависит от «Авторизоваться»), что соответствует рекомендациям по построению Use Case-моделей [9, с. 31].

2.1.2 Диаграмма потоков данных

Диаграмма потоков данных отражает логику прохождения информации и запросов в системе интернет-магазина, показывая, как взаимодействуют основные участники, программные компоненты и хранилища данных.

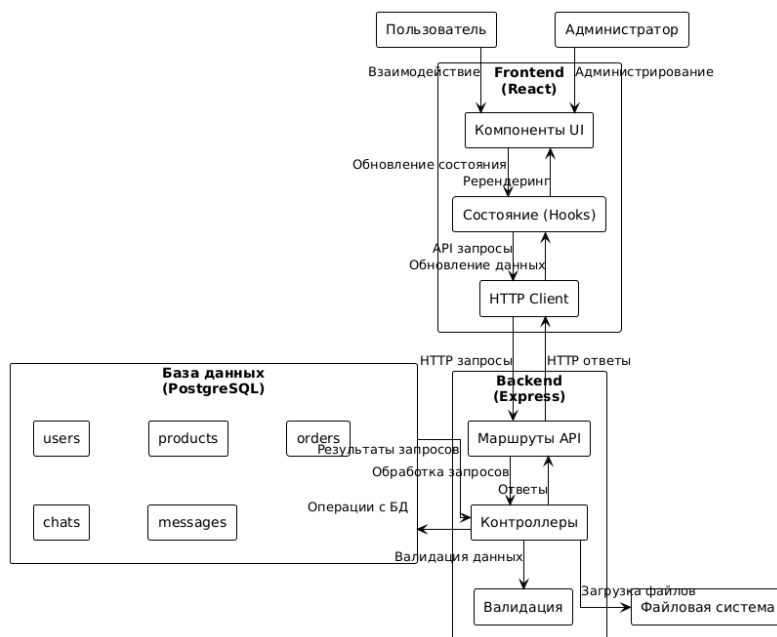


Рисунок 2.2 – Контекстная диаграмма
Примечание – Источник – Собственная разработка.

Согласно 120 Т 34.003-90, «информационное обеспечение АС – совокупность форм документов, классификаторов, нормативной базы и реализованных решений по объёмам, размещению и формам существования информации, применяемой в автоматизированной системе при её функционировании» 371]. В рамках разрабатываемой платформы информационное обеспечение 6 включает структурированные данные базы PostgreSQL, файловые хранилища изображений и медиафайлов, а также описание форматов обмена данными между уровнями системы.

В соответствии с 113 Т 19.701-90 «Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения», 24 ема – графическое представление определения, анализа или метода решения задачи, в котором используются символы для отображения операций, данных, потока, оборудования и т. д.» 1, с. 3]. Применяемые в работе графические нотации соответствуют указанному стандарту в части обозначений потоков данных и операций.

Запрос всегда стартует в браузере. React-приложение собирает данные формы, упаковывает их в JSON или multipart/form-data и через HTTP-клиент уходит на сервер. Express принимает запрос, прогоняет через middleware (валидация Zod, проверка JWT, логирование), затем – либо к PostgreSQL через Drizzle ORM, либо к файловой системе для медиа. В БД лежат пользователи, товары, заказы, чаты и сообщения; в каталоге uploads/ – картинки и логотипы. Сервер формирует ответ и возвращает его клиенту. Цикл одинаковый для любого сценария: GET /api/products читает товары, POST /api/orders создаёт заказ и тут же – связанный чат, multipart-загрузка картинки сохраняет файл и пишет метаданные в БД, обмен сообщениями работает через вставку в таблицу messages.

2.1.3 Диаграмма взаимодействия Frontend и Backend

А. П. Семёнов в монографии «Введение в UML» указывает: «диаграмма последовательности (sequence diagram) – диаграмма взаимодействия, отображающая упорядоченные во времени взаимодействия объектов в виде последовательности сообщений между линиями жизни. Диаграмма последовательности используется для уточнения сценариев работы системы, описанных в виде вариантов использования» 18, с. 121]. Применяемая в настоящей работе диаграмма взаимодействия Frontend и Backend построена с учётом указанных рекомендаций.

Диаграмма взаимодействия фиксирует обмен запросами и ответами между React-клиентом и Express-сервером с доступом к PostgreSQL и файловой

системе. Разложена по основным пользовательским сценариям. Приведена на рисунке 2.3.

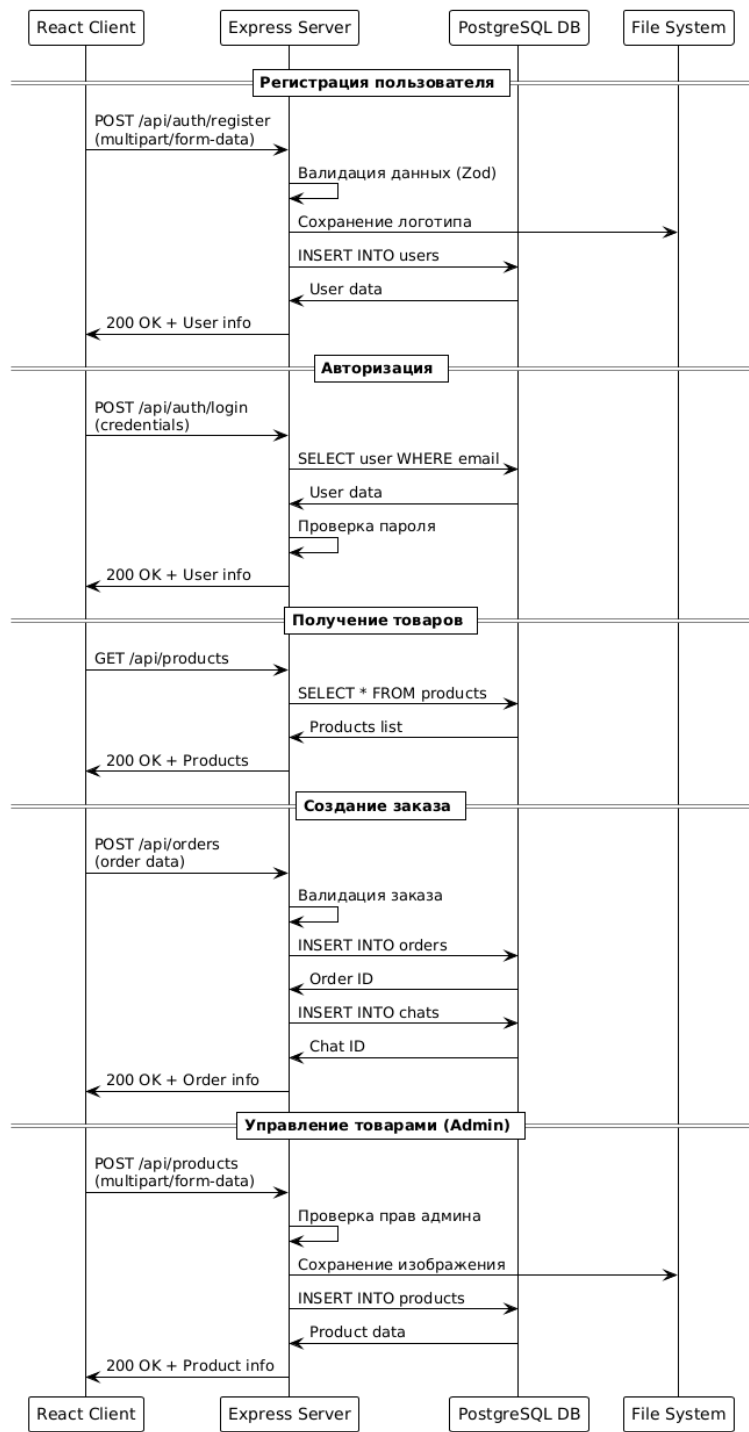


Рисунок 2.3 - Диаграмма взаимодействия Frontend и Backend

Примечание: Источник – Собственная разработка.

В соответствии с ГОСТ Р ИСО/МЭК 12207-2010, «процесс проектирования архитектуры программных средств – процесс, цель которого состоит в обеспечении проекта программного средства, который реализует требования и может быть протестирован на их соответствие. Процесс включает

разработку архитектуры программного средства, определение программных компонентов, проектирование интерфейсов между компонентами и базы данных» [1 с. 26]. Архитектура цифровой платформы спроектирована с учётом указанного процесса и описана в настоящем разделе.

К. Сидоров в работе «Безопасность веб-приложений» отмечает: «параметризованные SQL-запросы (prepared statements) являются базовым и наиболее эффективным средством защиты от инъекций кода в базу данных. Применение параметризованных запросов на уровне ORM-библиотеки исключает возможность несанкционированной модификации SQL-запроса со стороны пользовательского ввода и закрывает класс уязвимостей, отнесённых OWASP к категории A03:2021 – Injection» [11, с. 80]. В разрабатываемой платформе указанный принцип реализован средствами Drizzle ORM.

Точка входа всегда одна – React-интерфейс. Визуальные компоненты собирают действия пользователя, формируют запрос и через HTTP-клиент шлют его на API. В сценарии регистрации сервер принимает POST с формой и логотипом, валидирует данные, сохраняет файл в хранилище, заводит запись в таблице пользователей и возвращает клиенту подтверждение.

В соответствии со статьёй 17 Закона Республики Беларусь от 7 мая 2021 г. № 39-З [85] ратор обязан принимать необходимые правовые, организационные и технические меры для защиты персональных данных от неправомерного или случайного доступа к ним, уничтожения, изменения, блокирования, копирования, распространения, предоставления, а также иных неправомерных действий в отношении персональных данных» [1, 119]. В платформе указанные меры реализованы через хеширование паролей, передачу данных по защищённому каналу TLS, разграничение прав доступа на основе JWT-токенов и регулярное резервное копирование базы данных.

В соответствии со статьёй 18 Закона Республики Беларусь от 7 мая 2021 г. № 99-З, [76] трансграничная передача персональных данных на территорию иностранного государства осуществляется в случаях, если иностранным государством обеспечивается надлежащий уровень защиты прав субъектов персональных данных, либо при наличии письменного согласия субъекта персональных данных на трансграничную передачу его персональных данных, либо в случаях, предусмотренных международными договорами Республики Беларусь» [16, с. 16]. Хранение и обработка персональных данных пользователей платформы осуществляется на территории Республики Беларусь.

Авторизация – POST /api/auth/login: проверка по записи в БД, выдача учётных данных и прав, обновление состояния на клиенте. Запрос каталога – выборка из products, ответ в JSON, отрисовка списка. Создание заказа – фронт шлёт состав корзины, бэкенд проверяет данные, сохраняет в

orders и сразу создаёт связанный чат в chats для обсуждения с продавцом, в ответ возвращает агрегированную информацию о заказе. Админские операции с товаром – multipart-запрос с изображением и описанием: сервер проверяет права, сохраняет файл в /uploads, пишет запись в БД. Везде линейная цепочка: клиент → запрос → middleware → БД/ФС → ответ → обновление UI. Двусторонний обмен: от клиента – данные форм, фильтры и команды, обратно – выборки, статические ресурсы, статусы и сообщения об ошибках. Эта схема и фиксирует, как уровни системы и потоки данных связаны между собой.

Схема взаимодействия компонентов отражает целостную архитектуру веб-приложения интернет-магазина игрушек, показывая, как все уровни системы – от пользовательского интерфейса до хранилищ данных – соединены в единый рабочий механизм.

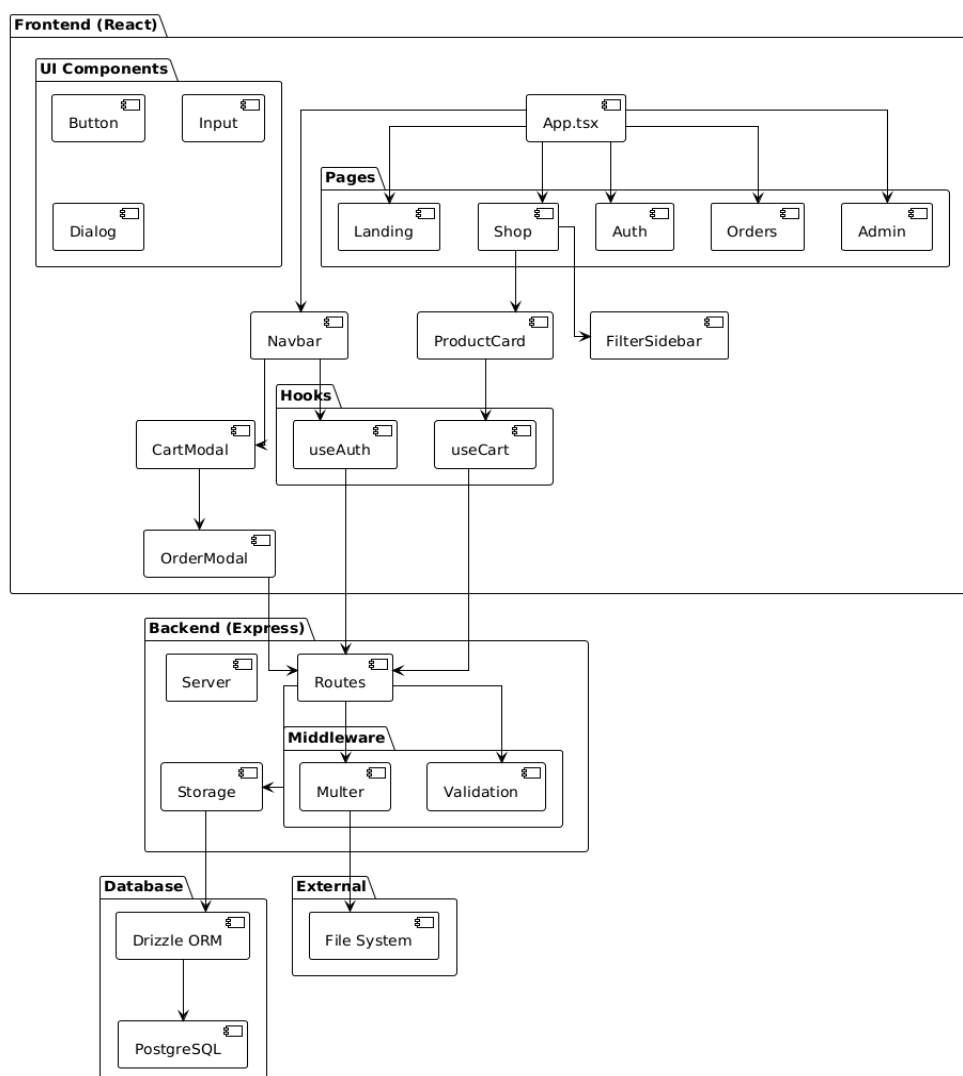


Рисунок 2.4 - Схема взаимодействия компонентов
Примечание – Источник – Собственная разработка.

В ГОСТ Р ИСО/МЭК 12207-2010 даётся следующее определение программной архитектуры: «архитектура программного средства – фундаментальная организация системы, воплощённая в её компонентах, их отношениях между собой и с окружением, а также принципы, определяющие проектирование и развитие системы»¹, с. 12]. Архитектура цифровой платформы «Семь драконов» построена по принципу разделения ответственности между уровнями представления, бизнес-логики и хранения данных.

На клиенте работает React 18 с TypeScript. UI собран из примитивов Radix UI (кнопки, поля, диалоги) и составных блоков – навбары, карточки товаров, фильтры. Эти компоненты складываются в страницы: каталог, авторизация, корзина, заказы, админ-панель. Состояние делю на серверное (TanStack Query – кэш, инвалидация, рефетч) и локальное (хуки useAuth, useCart). Модальные окна Radix Dialog позволяют оформить заказ или открыть корзину без перехода на отдельный экран. Все действия пользователя превращаются в HTTP-запросы и идут на бэкенд. Бэкенд – Express на Node.js. Маршрутизация распределяет запросы по контроллерам: auth, products, orders, chats, admin. На этом уровне – JWT-проверка, валидация Zod-схемами, загрузка файлов через Multer, логирование и обработка ошибок.

Слой доступа к данным – Drizzle ORM поверх PostgreSQL. Сущности: users, products, orders, chats, messages – со связями по внешним ключам и поддержкой транзакций. Медиа (картинки товаров, логотипы) лежат в /uploads и отдаются как статика. Между фронтом и бэком обмен строго по REST: запросы с параметрами или телом в JSON / multipart/form-data, ответы – JSON или поток файла. Регистрация: фронт передаёт форму, бэк проверяет, пишет в БД, возвращает токен и профиль. Каталог: запрос на список, выборка из products, JSON в ответ. Заказ: запись в orders + связанный чат в chats. Добавление товара админом: файл – в /uploads, метаданные – в БД.

Архитектура жёстко модульная: фронт работает только с API и статикой, бэк – только с хранилищами. Из-за этого систему легко расширять и сопровождать. На схеме видно, какой компонент за какой участок пути отвечает и как они связаны в единый поток.

В представленной схеме REST API все конечные точки сгруппированы по функциональным областям, что позволяет чётко разграничить задачи между модулями и обеспечить предсказуемость взаимодействия клиента с сервером.

В соответствии со статьёй 4 Закона Республики Беларусь от 7 мая 2021 г. № 99-З, ²⁷ аботка персональных данных может осуществляться оператором с согласия субъекта персональных данных, за исключением случаев, предусмотренных настоящим Законом и иными законодательными актами. Согласие субъекта персональных данных представляет собой свободное,¹

однозначное, информированное выражение его воли, посредством которого он разрешает обработку своих персональных данных»¹, с. 4]. Механизм получения согласия реализован в форме регистрации пользователя платформы.

Authentication API закрывает работу с учётными записями: POST /api/auth/register– регистрация с формой, POST /api/auth/login– вход, POST /api/auth/logout– завершение сессии. Через эти три эндпоинта проходит вся работа с правами доступа.

Product API ведёт жизненный цикл товаров. POST /api/products создаёт позицию с метаданными и изображением, GET /api/products возвращает каталог, GET /api/products/{id}– карточку. Обновление– PUT /api/products/{id}, удаление– DELETE /api/products/{id}.

Orders API: POST /api/ordersсоздаётзаказ, PUT /api/orders/{id}/statusменяетстатус, GET /api/ordersотдаётсписок (сфильтрациейпополямпараметрам), GET /api/orders/{id}– полнуюкарточкуконкретногозаказа.

Chat API –обменсообщениями. POST /api/chat/messages отправляет, GET /api/chat/messagesвозвращаетисторию. Один и тот же эндпоинт обслуживает и пользовательскую, и админскую сторону – разделение по правам выполняется в middleware.

В сумме структура эндпоинтов модульная: каждая группа методов закрывает свой блок функционала, форматы и названия единообразные. Это упрощает интеграцию с фронтом и расширение API без поломки контрактов между клиентом и сервером.

Authentication API	Product API	Orders API	Chat API
POST /api/auth/register	GET /api/products	GET /api/orders	GET /api/chat/messages
POST /api/auth/login	GET /api/products/{id}	GET /api/orders/{id}	POST /api/chat/messages
POST /api/auth/logout	POST /api/products	POST /api/orders	GET /api/chats
GET /api/auth/me	PUT /api/products/{id}	PUT /api/orders/{id}/status	
	DELETE /api/products/{id}		

Условные обозначения HTTP-методов:

GET — чтение данных POST — создание ресурса PUT — обновление DELETE — удаление

Рисунок 2.5 - REST API Endpoints
Примечание: Источник – Собственная разработка.

2.2 Проектирование данных

2.2.1 Информационная модель

Информационная модель – это формальное представление ключевых объектов предметной области, их атрибутов и отношений, на основе которого проектируется структура СУБД и обеспечивается целостность данных.

Согласно ГОСТ 34.003-90, 97 за данных – совокупность данных, организованных в соответствии с концептуальной структурой, описывающей характеристики этих данных и взаимоотношения между их сущностями, обслуживающая одну или несколько прикладных областей» 1. База данных платформы организована в реляционной СУБД PostgreSQL и состоит из взаимосвязанных таблиц пользователей, товаров, заказов, чатов и сообщений.

Информационная модель описывает структуру БД, спроектированной под типовые операции компании: каталог, заказы, поддержку клиентов. В основе – нормализация (3НФ) и разделение по логическим блокам [13, с. 23]. Центральная сущность – User. В ней лежат и контактные данные с реквизитами компании, и параметры авторизации, и флаг администратора – это позволяет одной таблицей покрыть и клиентов, и сотрудников. Уникальный индекс по e-mail и индексы по часто используемым полям ускоряют поиск.

Второй блок – каталог. Сущность Product содержит название, описание, изображение, категорию, возрастную группу, материал и страну. Особенность – оптовое ценообразование: для одной позиции хранятся три цены (price5, price20, price50) под объёмные пороги. Категория и возрастная группа вынесены в справочники, чтобы исключить дубликаты и опечатки [13, с. 24].

Сущность Order описывает «шапку» заказа: клиент, итоговая сумма, адрес доставки, текущий статус, дата создания. Справочник OrderStatus фиксирует жизненный цикл от оформления до доставки или отмены. Состав вынесен в отдельную таблицу OrderItem – товар, количество, цена за единицу, ценовой уровень. Такой подход упрощает аналитику и точно считает себестоимость.

Последний блок – коммуникации. Каждому заказу автоматически создаётся сущность Chat, которая связывает заказ и клиента в одном канале поддержки. Внутри чата хранятся записи Message: идентификатор чата, отправитель, текст, дата. История переписки прозрачна для обеих сторон. Информационная модель ИС показана на рисунке 2.8.

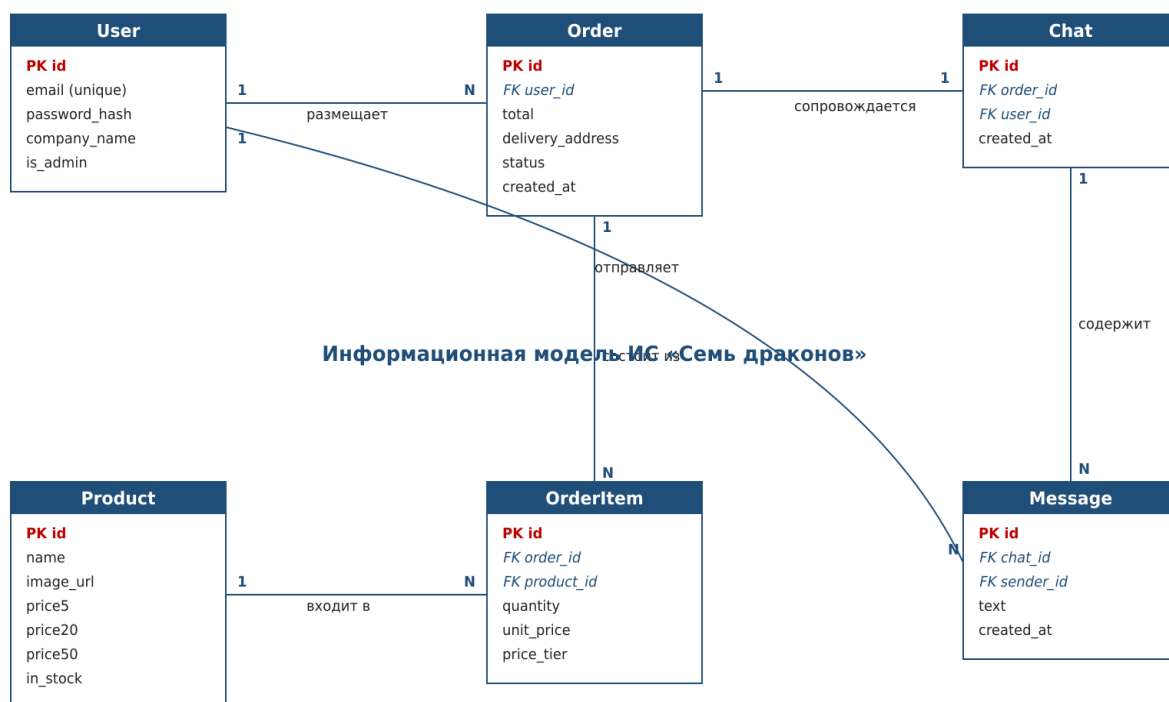


Рисунок 2.6 - Информационная модель
Примечание: Источник – Собственная разработка.

2.2.2 Характеристика базы данных

Согласно ГОСТ 34.003-90, «целостность базы данных – состояние базы данных, при котором соблюдаются все ограничения целостности, установленные для данной базы данных» [1]. Целостность данных в проектируемой системе обеспечивается ограничениями NOT NULL, UNIQUE, CHECK, а также ссылочной целостностью через внешние ключи между таблицами.

С. П. Иванова в учебнике «Администрирование PostgreSQL» подчёркивает: PostgreSQL является объектно-реляционной системой управления базами данных с открытым исходным кодом, поддерживающей расширенный набор стандартов SQL, многоверсионное управление параллельным доступом (MVCC), полнотекстовый поиск, JSON-типы данных и широкий набор индексных структур, включая B-tree, Hash, GIN, GiST и BRIN» [1, с. 14]. Указанные возможности использованы при проектировании структуры данных платформы и обеспечении эффективности типовых выборок.

Под систему выбрана PostgreSQL. Это даёт надёжность, расширяемость и поддержку современных механизмов работы с данными. Логическая

структура спроектирована по принципам нормализации, разделения по тематическим блокам и обеспечения целостности через строгие связи между таблицами.

А. В. Корнеев в работе «Проектирование баз данных» отмечает: «нормализация отношений является обязательным этапом логического проектирования реляционной базы данных и направлена на устранение функциональных и многозначных зависимостей, приводящих к избыточности данных и аномалиям при операциях вставки, обновления и удаления» [13, с. 22]. База данных проектируемой платформы приведена к третьей нормальной форме, что подтверждается отсутствием транзитивных функциональных зависимостей в схемах таблиц `users`, `products`, `orders`, `chats` и `messages`.

С. П. Иванова в работе «Администрирование PostgreSQL» отмечает: «индекс типа B-tree является универсальным и подходит для большинства типов запросов, использующих операции сравнения (равенство, диапазон, упорядочивание). Применение индексов B-tree по часто запрашиваемым столбцам позволяет ускорить выборки в десятки раз при умеренных накладных расходах на операции вставки и обновления» [14, с. 18]. Указанные принципы учтены при проектировании индексов на таблицах `users` (по полю `email`), `products` (по `category_id`, `age_group_id`) и `orders` (по `user_id`, `status`).

Идентификаторы записей – серийные первичные ключи (`serial/bigserial`). Это гарантирует уникальность и упрощает навигацию. Связи между сущностями – через внешние ключи, что формирует иерархическую модель и исключает «висячие» ссылки. Например, запись в `Order` связана с конкретным `User` и набором `OrderItem`, а каждая позиция – с конкретным `Product`.

Характерной особенностью модели является работа как с традиционными реляционными структурами, так и со сложными вложенными данными. Для хранения агрегированных объектов используется тип `JSONB` – в частности, для первоначального варианта хранения состава заказа. Это позволяет гибко описывать переменное количество элементов, не нарушая общей структуры базы, а также эффективно работать с фильтрацией и выборками по вложенным ключам. Несмотря на то, что в оптимизированной версии модели позиции заказа вынесены в отдельную таблицу `OrderItem` для аналитической прозрачности, поддержка `JSON` остаётся ценным инструментом для хранения вспомогательных структур, фильтров и параметров.

Все ключевые сущности системы снабжены временными метками (`createdAt`, `updatedAt`), что формирует полноценный аудит-след и позволяет отслеживать историю изменений, жизненный цикл заказов и событий. Данный механизм важен как для внутреннего контроля, так и для аналитики бизнес-процессов.

Логическая модель базы данных охватывает несколько функциональных блоков:

- Пользователи и учетные данные – таблица User с расширенной контактной и организационной информацией, признаком административных прав и уникальной аутентификацией по e-mail.

- Каталог товаров – таблица Product с детальными характеристиками товара, классификацией по категориям и возрастным группам, а также прогрессивной ценовой матрицей (цены за 5 – 19, 20 – 49 и 50+ единиц).

- Заказы и их состав – таблицы Order и OrderItem, позволяющие хранить как общие сведения о заказе (клиент, адрес доставки, сумма, статус), так и детализированный перечень товаров с количеством и типом цены.

- Коммуникации – таблицы Chat и Message, реализующие связанный с заказом канал обмена сообщениями между клиентом и службой поддержки.

Для оптимизации производительности в базе данных предусмотрены индексы по наиболее часто используемым полям (категории товаров, статусы заказов, признак наличия на складе, признак администратора и т. д.), что ускоряет выполнение запросов выборки и упрощает построение аналитических отчетов. Связи реализованы по принципу «один-ко-многим» и «один-к-одному» там, где это обусловлено бизнес-логикой (например, один заказ – один чат поддержки).

В совокупности данная архитектура PostgreSQL-базы данных сочетает строгую структурированность реляционной модели с гибкостью работы с полуструктурированными данными, обеспечивает целостность и согласованность информации и позволяет безболезненно развивать систему за счет добавления новых модулей и интеграций. Это делает её устойчивым фундаментом для дальнейшей автоматизации бизнес-процессов и расширения функциональности информационной системы.

2.2.3 Описание нормативно-справочной, входной и результатной информации

Информационное наполнение базы данных формируется из нескольких типов данных, каждый из которых играет свою роль в работе системы и обеспечивает полный цикл функционирования – от ввода первичных сведений до формирования аналитических отчетов.

Нормативно-справочная информация представлена стабильными, редко изменяемыми наборами данных, которые служат основой для классификации и унификации записей в операционных таблицах. К таким данным относятся:

- Категории товаров – фиксированные группы для систематизации ассортимента (например, конструкторы, куклы, настольные игры).

- Возрастные группы – диапазоны, по которым сегментируется целевая аудитория продукции.

- Материалы – перечень допустимых значений для описания состава товаров.

- Страны – справочник географического происхождения или поставки.

- Статусы заказов – последовательность состояний, описывающая этапы жизненного цикла заказа (от «Нового» до «Доставленного»).

Эти сведения обеспечивают целостность и корректность данных за счет ограничения возможных значений полей и используются в связях с основной операционной информацией.

Входная информация формируется в процессе работы пользователей и включает:

- Регистрационные данные – сведения, необходимые для создания учетной записи и идентификации пользователя, включая контактную и организационную информацию.

- Товарная информация – детализированные описания продукции, изображения, цены, наличие на складе.

- Заказы – сформированные клиентами заявки, включающие набор позиций, адрес доставки, условия оплаты и выбранные товары.

- Сообщения – переписка между пользователем и службой поддержки в рамках заказов, хранящаяся в системе чатов.

Этот блок данных является динамичным и постоянно пополняется в ходе эксплуатации системы.

Результатная информация формируется на основе обработки и анализа входных данных.

2.3 Архитектура системы

2.3.1 Диаграмма компонентов

Клиентская часть приложения реализована на React с использованием TypeScript и организована по компонентной архитектуре. В её состав входят несколько ключевых групп:

- Страницы – верхнеуровневые контейнеры, отвечающие за маршруты и сборку интерфейса из отдельных компонентов:

- Landing – главная страница с обзором ассортимента и промо-блоками.
 - Shop – каталог товаров с поиском, фильтрацией и добавлением в корзину.
 - Orders – личный кабинет с историей заказов и их статусами.
 - Admin – административная панель для управления товарами, заказами и пользователями.
 - UI-компоненты – переиспользуемые элементы интерфейса:
 - NavBar – панель навигации с логотипом, поиском, меню и значком корзины.
 - ProductCard – карточка товара с изображением, ценой и кнопками действий.
 - CartModal – модальное окно корзины с возможностью изменить количество товаров и оформить заказ.
 - Хуки для состояния – кастомные React-хуки, инкапсулирующие логику работы с данными и событиями:
 - useAuth – управление аутентификацией, токенами и профилем пользователя.
 - useCart – хранение и обновление данных корзины, расчёт суммы, синхронизация с сервером.
- Эти модули взаимодействуют с сервером через HTTP API, используя такие библиотеки, как TanStackQuery, для кэширования и синхронизации данных.
- Серверная часть приложения реализована на Node.js с использованием Express и разделена на несколько слоёв:
- Routes (API Endpoints) – набор REST-маршрутов, которые принимают запросы от фронтенда и направляют их к соответствующей бизнес-логике:
 - маршруты аутентификации (/api/auth/...),
 - работы с товарами (/api/products/...),
 - управления заказами (/api/orders/...),
 - чатов (/api/chats/...),
 - административных функций (/api/admin/...).
 - Storage (Слой данных) – модуль доступа к базе данных PostgreSQL через ORM Drizzle. Отвечает за выполнение CRUD-операций с таблицами пользователей, товаров, заказов, чатов и сообщений.
 - Middleware – промежуточные обработчики, интегрированные в конвейер Express:
 - Аутентификация – проверка JWT-токенов и прав доступа.

- Логирование – запись информации о запросах и ответах для отладки и аудита.
- Обработка ошибок – централизованная генерация и отправка структурированных сообщений об ошибках клиенту.
- Обработка файлов – приём и сохранение загружаемых изображений и документов (Multer).

Такое разделение обеспечивает модульность, лёгкое масштабирование и изоляцию логики между слоями.

2.3.2 Диаграмма развертывания

Диаграмма развертывания иллюстрирует физическую и виртуальную инфраструктуру системы, показывая аппаратные и программные узлы, размещение артефактов и каналы связи без учёта платёжных шлюзов и модулей техподдержки. Ее можно увидеть на рисунке 2.10

Система развёрнута в трёх логических зонах:

- Пограничная (DMZ) с балансировщиками и публичным веб-проксем
- Прикладная зона с контейнерными кластерами
- Зона хранения данных и кэша

В DMZ располагаются два узла L4-балансировщика с активным режимом, каждый подключён к публичному IPv4 и IPv6 и настроен на TLS-терминацию (сертификаты Let'sEncrypt). Балансировщики распределяют HTTPS-трафик на группу Web-Tier, в которую входят по 3 реплики Nginx-прокси (Docker-контейнеры). Они фильтруют HTTP-заголовки (CSP, HSTS), выполняют первичную проверку IP-блэка списка и пробрасывают запросы внутрь виртуальной сети[20].

Ingress-контроллер (NGINXIngress) создаёт маршруты доступа по разным доменным именам (api.company.ru, app.company.ru) и обеспечивает ограничение скорости (rate-limiting) на уровне HTTP[21].

Зона хранения данных:

– PostgreSQL-кластер в режиме primary – sync-standby, развернутый через StatefulSet, PVC на Ceph-FS, ежедневный физический бэкап через Velero и диапозонное партиционирование таблицы traffic_stats по месяцам.

– RepliT Sentinel-кластер из трёх мастеров с репликами для сессий и кэша, мониторинг задержек и автопереключение при отказе.

Сетевые связи:

- Все узлы подключены к виртуальному оверлей-сегменту.

– Между DMZ и прикладной зоной стоит межсетевой экран с глубоким анализом пакетов (IPS), запрещающим прямые подключения с публичного интернета к базе данных и кэшу.

CI/CD конвейер:

– при пуше в ветку main автоматически создаётся артефакт Replit образа(как Docker образ, но все автоматизировано), прогоняются unit- и integration-тесты, проверяется статический анализ кода (ESLint, SonarQube), затем производится деплой в staging через Helm.

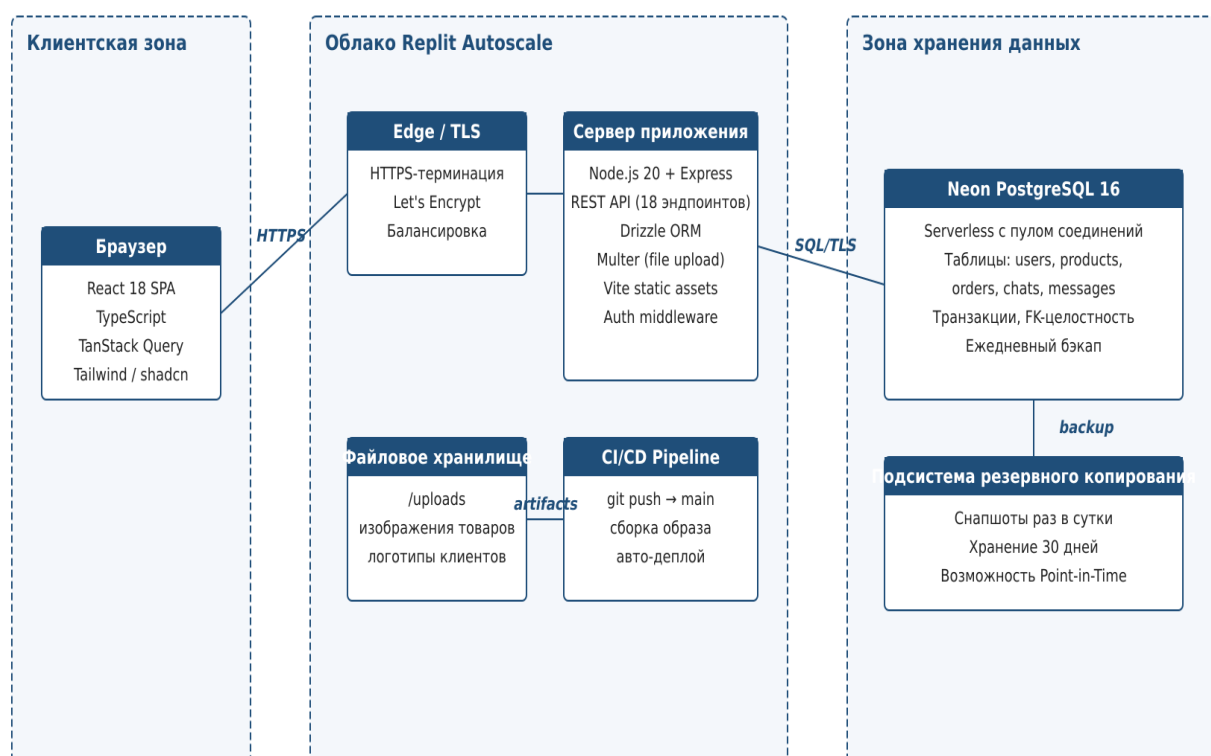


Рисунок 2.7 - Диаграмма развертывания

Примечание – Источник – Собственная разработка.

2.4 Разработка программных модулей

2.4.1 Клиентский модуль 126

В соответствии с ГОСТ Р ИСО/МЭК 12207-2010, «программный компонент – часть программного средства, рассматриваемая как единое целое в рамках процесса проектирования, разработки и сопровождения» [1, с. 14].

Клиентский, серверный модули и модуль работы с данными представляют собой программные компоненты в значении указанного стандарта и проектируются с учётом требований к модульности, повторной используемости и тестируемости.

В обзоре изменений React 18 указано: «React 18 представляет concurrent rendering как фундаментальное обновление, позволяющее библиотеке готовить несколько версий пользовательского интерфейса одновременно. Concurrent rendering включает автоматическое объединение обновлений состояния (automatic batching), новый API-приоритезации обновлений (transitions) и поддержку Suspense на стороне сервера»¹². Клиентский модуль платформы использует указанные возможности для обеспечения отзывчивости интерфейса при работе с большими каталогами товаров.

Клиентский модуль – одностраничное приложение (SPA) на React. Страницы: каталог, авторизация, корзина, оформление заказа, личный кабинет, админ-панель. Маршрутизация – Wouter (легковесный router без роутерных провайдеров). Серверное состояние – TanStack Query: кэш, инвалидация, ретрай, фоновый рефетч. Локальное состояние – хуки useAuth, useCart. Стили – Tailwind CSS, базовые UI-примитивы – Radix UI. Формы – React Hook Form + Zod-резолвер. Иконки – lucide-react. Сборщик – Vite (cold start <500 мс, HMR в один кадр). Сервер коммуникации – fetch-обёртка с обработкой 401/403 и автоматическим прокидыванием JWT в заголовок Authorization.

Взаимодействие с серверной частью осуществляется через REST API по протоколам HTTP/HTTPS, а для управления серверным состоянием и оптимизации работы с данными применяется библиотека TanStackQuery. Она обеспечивает автоматическое кэширование, синхронизацию и поддержку оптимистичных обновлений, что делает интерфейс отзывчивым и минимизирует время ожидания пользователя. Все API-запросы инкапсулированы в сервисных модулях, что упрощает их повторное использование и обслуживание.

Разработка ведётся в облачной среде Replit. Один контейнер хранит и фронт, и бэк, и переменные окружения. Это удобно: новый разработчик подключается за минуту, без локальных установок. В режиме dev Vite проксирует API-запросы на Express, изменения в коде фронта подхватываются HMR без перезагрузки страницы. Изменения в коде бэка перезапускают сервер через tsx watch. Конфигурация (порты, команды запуска, переменные) описана в одном файле и ровно та же используется при деплое в production.

2.4.2 Серверный модуль

Серверный модуль интернет-магазина «Семь Драконов» построен на основе Node.js с использованием фреймворка Express.js и написан на TypeScript, что обеспечивает строгую типизацию, облегчает рефакторинг и минимизирует вероятность логических ошибок на ранних этапах разработки. Архитектура сервера организована по модульному принципу: логика разделена на независимые блоки маршрутов, каждый из которых отвечает за конкретную доменную область – авторизация и регистрация пользователей, управление каталогом товаров, оформление и обработка заказов, работа с чатом, а также административные операции. Каждый такой маршрутный модуль инкапсулирует контроллеры для обработки HTTP-запросов, обращается к сервисному слою для выполнения бизнес-логики и использует слой доступа к данным для взаимодействия с базой. Эта изоляция зон ответственности упрощает поддержку и добавление нового функционала, исключая нежелательные побочные эффекты.

Важным элементом серверной архитектуры является система промежуточных обработчиков (middleware). Она включает:

- аутентификацию и авторизацию с использованием JWT-токенов, позволяющую разграничивать доступ к защищённым ресурсам;
- централизованную обработку ошибок, формирующую единообразные структурированные ответы для клиента и скрывающую внутренние детали реализации;
- логирование запросов и ответов, **96** необходимое для мониторинга и аудита;
- модуль для загрузки и сохранения файлов на основе библиотеки Multer, который принимает изображения товаров и другие ресурсы, проверяет их и сохраняет в специально выделенную директорию в файловой системе контейнера;
- обработчик статических ресурсов, который позволяет Express напрямую отдавать изображения, **129** и скрипты без лишней нагрузки на бизнес-логику.

Официальная документация Drizzle ORM содержит следующее описание: «Drizzle ORM – это TypeScript-ORM, обеспечивающий полностью типизированный SQL-подобный API для работы с реляционными базами данных. Библиотека генерирует типы запросов и результатов на основе декларативного описания схемы, что обеспечивает безопасность типов на этапе компиляции и не требует генерации классов сущностей во время выполнения»

[28]. Указанный подход применён при реализации модуля работы с данными платформы.

Доступ к БД реализован через Drizzle ORM поверх PostgreSQL на Neon (serverless-PG). Drizzle выбран по нескольким причинам: типизированная схема, прозрачные SQL-запросы (без скрытой магии), удобный генератор миграций. Подключение – через пул соединений @neondatabase/serverless с поддержкой WebSocket-транспорта, что критично для serverless-окружения с холодными стартами.

Отдельного внимания заслуживает то, что весь серверный модуль – от разработки до деплоя – живёт в облачной среде Replit, совместно с клиентским приложением. На этапе разработки Express-сервер запускается в контейнере ReplitWorkspace, используя проброс порта (по умолчанию 5000), что даёт возможность тестировать API локально, но в облаке. При этом сервер и фронтенд работают в одном окружении, что убирает проблемы с CORS и разными конфигурациями между dev и prod. Разработчик получает доступ к полноценной веб-IDE с автодополнением, встроенным терминалом, поддержкой Git и моментальным предпросмотром работы сервера в браузере. Использование ViteDev Server в том же окружении позволяет одновременно вести работу над клиентом и сервером, а встроенный прокси перенаправляет запросы SPA к API без дополнительной настройки.

При деплое Replit выполняет автоматическую упаковку проекта и развёртывает его на Autoscale-инстансах. Это среда продакшн-класса с балансировкой нагрузки, возможностью горизонтального масштабирования, 99, 95% SLA по доступности, SSL/TLS-терминацией и поддержкой собственных доменов. Переход из режима разработки в продакшн в Replit происходит без смены окружения и инфраструктуры, что гарантирует идентичность условий и предотвращает неожиданные ошибки. Кроме того, Replit хранит как исходный код, так и артефакты сборки, а статические файлы и загруженные ресурсы могут обслуживаться напрямую из контейнера. Для администратора это означает, что обновление серверной логики или конфигураций требует лишь заливки новых изменений в репозиторий или внесения правок в IDE, после чего Replit автоматически пересобирает и перезапускает приложение. 65

Таким образом, серверный модуль «Семь Драконов» – это не просто набор Express-маршрутов и middleware, а полнофункциональный backend-сервис с чёткой модульной структурой, безопасным доступом к данным, развитой системой обработки запросов и ошибок, интеграцией с современными облачными технологиями хранения и развёрнутый в среде, которая объединяет разработку, тестирование и эксплуатацию в едином облачном цикле.

2.4.3 Модуль работы с данными

Модуль работы с данными построен на Drizzle ORM. Полная типизация: схема описана в TypeScript, типы insert и select генерируются автоматически. SQL-конструктор не прячет запросы – оптимизация остаётся под контролем разработчика. Миграции описываются декларативно и применяются Drizzle Kit – без ручной правки SQL-скриптов.

Подключение к Neon идёт через пул соединений. Это снимает проблему долгоживущих коннектов, которые зависают при паузах в трафике. Пул держит лимит активных подключений, переиспользует их между запросами и автоматически пересоздаёт сессию при разрыве. На практике это даёт стабильную работу под нагрузкой и предсказуемую latency на холодном старте.

Контроллеры API не лезут в SQL напрямую. Вся работа с данными идёт через слой Storage с типобезопасными запросами Drizzle – это поднимает читаемость и упрощает рефакторинг. Добавили новое поле в схему – TypeScript сам подсветит все места, где это поле должно быть учтено. Вручную править строки SQL по проекту не приходится.

Разработка модуля идёт прямо в Replit Workspace, рядом с фронтом и бэком. Параметры подключения к Neon заданы как переменные окружения в конфиге Workspace. Drizzle Kit генерирует и применяет миграции из той же IDE – без локального PostgreSQL. Это упрощает онбординг и ускоряет проверку изменений в схеме. При деплое на Autoscale используется ровно та же конфигурация – окружения dev и prod идентичны, и проблема «у меня работает» исключается.

В сумме модуль работы с данными – надёжный и гибкий слой хранения. Сочетание ORM, serverless-Neon и контейнеризованной инфраструктуры Replit даёт стабильную работу платформы и быстрый цикл разработки.

2.5 Технологическое обеспечение разработки

2.5.1 Процесс тестирования

ГОСТ Р ИСО/МЭК 12207-2010 определяет: **13** процесс верификации программных средств – процесс, цель которого состоит в подтверждении того, что результаты работы или услуги полностью соответствуют установленным требованиям» **1** с. 38]. Применительно к проектируемой системе верификация выполняется на трёх уровнях: статический анализ кода (TypeScript, ESLint),

модульное тестирование функций и компонентов, интеграционное и приёмочное тестирование с использованием реальной базы данных.

В соответствии с ГОСТ Р ИСО/МЭК 25010-2015 «Информационные технологии. Системная и программная инженерия. 24 бования и оценка качества систем и программного обеспечения», 13 чество программного продукта характеризуется набором показателей: функциональная пригодность, уровень производительности, совместимость, удобство использования, надёжность, защищённость, сопровождаемость и переносимость» 10, с. 8]. Указанные показатели использованы в качестве критериев оценки качества разрабатываемой платформы.

Тестирование опирается на статический анализ и проверку качества кода. TypeScript ловит ошибки типов на этапе компиляции – до запуска. Стил ь кода держится через ESLint (анализ проблемных конструкций) и Prettier (автоматическое форматирование). Связка работает на каждом этапе: при сохранении файла, в pre-commit хуке и в CI. Это снижает количество ошибок в рабочей сборке.

2.5.2 Процесс развертывания

В соответствии с ГОСТ Р ИСО/МЭК 12207-2010, «процесс ввода в эксплуатацию – процесс, цель которого состоит в установке и подготовке программного продукта к работе в соответствии с требованиями» 1 с. 30]. Описанный далее регламент развёртывания платформы «Семь драконов» соответствует данному процессу и охватывает сборку клиентской и серверной частей, размещение артефактов и настройку производственного окружения.

А. А. Федоров в монографии «DevOps: практика непрерывной поставки» указывает: «непрерывная поставка (continuous delivery) – это подход к разработке программного обеспечения, при котором изменения в коде автоматически собираются, тестируются и подготавливаются к выпуску в производственную среду. Применение непрерывной поставки сокращает время от внесения изменений до их доставки конечному пользователю с нескольких дней до 15-30 минут и существенно снижает риск ошибок при развёртывании» 1 [8, с. 78]. В разрабатываемой платформе элементы непрерывной поставки реализованы средствами Replit Autoscale Deployment.

В соответствии с ГОСТ Р ИСО/МЭК 25010-2015, «надёжность программного продукта – степень, в которой программная система выполняет заданные функции в указанных условиях в течение указанного периода времени. Подхарактеристики надёжности включают завершённость, готовность, отказоустойчивость и восстанавливаемость» 1, с. 12]. Указанные

подхарактеристики использованы при формулировании нефункциональных требований к разрабатываемой платформе.

Развёртывание организовано как один автоматизированный цикл – от исходного кода до production. Фронт собирается Vite: TypeScript и JSX компилируются в оптимизированный JS, идёт минификация и tree-shaking, формируются статические ресурсы (HTML, CSS, изображения), всё складывается в dist. Параллельно обрабатываются CSS-стили и встроенные ассеты – это сокращает вес бандла и ускоряет загрузку.

Серверный модуль (Express + TypeScript) собирается через esbuild – компилятор и бандлер с очень быстрой сборкой. TS-файлы превращаются в исполняемый JS под Node.js, модули объединяются и оптимизируются. Время сборки минимально, результат одинаковый на всех средах.

После сборки фронта и бэка артефакты упаковываются в одно приложение. В production Express отдаёт и API-эндпоинты, и статику фронта – отдельный веб-сервер не нужен. Это упрощает архитектуру и уменьшает количество точек отказа.

Развёртывание интегрировано с Replit. Один контейнер – и среда разработки, и production. Из Replit Workspace разработчик собирает фронт и бэк, потом запускает деплой на Autoscale Deployment. Окружение само масштабируется под нагрузку, балансирует трафик, обеспечивает доступность (SLA 99,95%), поддержку пользовательских доменов и SSL/TLS. Перед деплоем Replit применяет конфиг проекта (порт, команда запуска, переменные окружения) – за счёт этого dev и prod ведут себя одинаково.

В итоге деплой – это последовательность шагов в одной инфраструктуре: сборка статики, компиляция серверной логики, публикация на масштабируемой облачной платформе, доступной по защищённому адресу.

2.5.3 Мониторинг и логирование

А. С. Петров в работе «Управление производительностью информационных систем» отмечает: «мониторинг информационной системы должен охватывать все ключевые показатели работы: время отклика, пропускную способность, частоту ошибок, использование ресурсов сервера и базы данных. Регулярный анализ собираемых метрик позволяет выявлять узкие места в производительности и предотвращать сбои до их влияния на пользователей»¹, с. 16]. Описанная далее система логирования платформы обеспечивает сбор указанных показателей.

К. Н. Сидоров в учебнике «Системы контроля версий и непрерывная интеграция» ¹ подчёркивает: ⁹⁷ непрерывная интеграция (continuous integration) – это практика разработки программного обеспечения, при которой члены команды часто интегрируют свою работу в общий репозиторий исходного кода. Каждая интеграция проверяется автоматизированной сборкой и тестами, что позволяет обнаруживать ошибки интеграции на ранних этапах и существенно снижать стоимость их исправления» ¹ с. 14]. Применение указанной практики в проекте обеспечивается интеграцией репозитория с Replit Workspace.

В серверный модуль встроены мониторинг и логирование. Фиксируется весь цикл обработки API-запросов: время получения, HTTP-метод, эндпоинт, идентификатор пользователя (если есть аутентификация), параметры запроса и тело (для допустимых типов). По каждому запросу считается время выполнения – так выявляются «узкие места» в производительности и видно их динамику.

Ответы сервера тоже логируются – HTTP-статус, краткое описание результата, диагностические комментарии. Двусторонняя фиксация запроса и ответа даёт целостную картину работы и помогает воспроизводить проблемные сценарии.

Обработка ошибок построена по централизованной схеме: любое исключение, возникающее на этапе маршрутизации, исполнения бизнес-логики или обращения к базе данных, перехватывается единым middleware-компонентом.

ГЛАВА 3

РЕАЛИЗАЦИЯ, ТЕСТИРОВАНИЕ И ЭКСПЛУАТАЦИЯ

3.1 Руководство по установке и настройке

3.1.1 Системные требования

Вся серверная и клиентская часть веб-приложения интернет-магазина развёрнута и функционирует в облачной среде Replit, что обеспечивает мгновенный доступ к рабочей среде, автоматическое обновление кода и независимость от локальной инфраструктуры.

Технологический стек

- Backend: Node.js (Express.js для REST API), модульная структура с выделенными слоями маршрутизации, бизнес-логики и доступа к данным.
- Frontend: React.js с компонентным подходом, интеграция с API через axios/fetch, адаптивная верстка (HTML5, CSS3, Flexbox/Grid).
- База данных: PostgreSQL (облачная база Neon/Postgres в контейнере Replit), поддержка транзакций и индексов.
- Системы логирования и мониторинга: встроенные middleware-модули + возможность подключения внешних сервисов через API-ключи.
- Контейнеризация: использование встроенных Replit-сред для изоляции окружения и управления зависимостями через package.json.
- Управление версиями: Git-репозиторий, встроенный в Replit, с автоматическим деплоем при коммитах.

Требования к рабочей среде

- Сервер: виртуальный контейнер Replit с предустановленным Node.js (v16+), доступом к облачной базе данных и выделенными ресурсами (CPU/RAM в зависимости от тарифного плана Replit).
- Клиентское окружение: любой современный веб-браузер с поддержкой ES6+ (Chrome, Firefox, Edge, Safari).
- Сетевое подключение: устойчивое интернет-соединение для работы с API и WebSocket-каналами.

Преимущества развёртывания на Replit

- Автоматическая сборка и перезапуск приложения при изменениях.
- Единая среда для разработки, тестирования и эксплуатации.
- Возможность совместной работы над проектом в реальном времени.

3.1.2 Пошаговая инструкция развертывания

Ниже приведён регламент запуска серверной и клиентской части интернет-магазина в облачной среде Replit. Он ориентирован на воспроизводимое, документированное и удобное развёртывание, подходящее как для среды разработки, так и для эксплуатации в production.

1. Клонирование и подготовка проекта

- Откройте проект в рабочем пространстве Replit (через Git-репозиторий или импорт ZIP-архива).

- Убедитесь, что в `v.replit` и `replit.nix` корректно указана команда запуска.

2. Установка зависимостей

- Установить все зависимости, указанные в `package.json` (backend и frontend).

- При использовании Replit установка выполняется в изолированной контейнерной среде.

3. Настройка переменных окружения

- В панели ReplitSecrets или в файле `.env` создайте ключ

- Для Neon/PostgreSQL подключение автоматическое

4. Создание и синхронизация схемы базы данных

`npmrundb: push`

- Команда синхронизирует модель данных с реальной БД, создавая таблицы и индексы.

- Логи операции помогают убедиться, что миграция прошла без ошибок.

5. Запуск приложения

- Сервер автоматически перезапускается при изменениях в коде.

- Логи и ошибки выводятся в консоль Replit.

- Сначала выполняется сборка оптимизированного frontend-кода (React).

- Затем запускается Node.js-сервер, обслуживающий статические файлы и API.

6. Верификация запуска

- Перейдите по URL Replit-проекта (формат `https://<project>.<username>.repl.co`).

- При необходимости – просмотрите логи для проверки подключения к базе и корректности работы middleware.

3.1.3 Настройка окружения и интеграций

Данный раздел описывает параметры конфигурации облачного окружения Replit, интеграцию с базой данных Neon PostgreSQL, а также организацию файлового хранилища и систему переменных окружения для корректной работы приложения. **122**

1. Конфигурация NeonPostgreSQL

- Создание БД:

1. Зарегистрируйтесь в Neon.tech и создайте проект с типом базы PostgreSQL.

2. Определите параметры подключения (host, порт, имя БД, имя пользователя, пароль, SSL).

- Строка подключения: Формат:

postgresql: //<user>: <password>@<host>/<dbname>?sslmode=require

Необходимо скопировать и сохранить в переменную окружения DATABASE_URL.

- Интеграция с приложением:

- В ReplitSecrets создайте ключ DATABASE_URL и вставьте полученную строку.

- Убедитесь, что используемый ORM/драйвер настроен на чтение этой переменной.

- Выполните синхронизацию структуры командой: npmrun db: push

2. Настройка файловых загрузок

- Директория: в корне проекта создаётся папка. Она используется для временного хранения загружаемых изображений и документов.

- Конфигурация Express:

```
import path from 'path';
import multer from 'multer';
const storage = multer.diskStorage({
  destination: path.join(process.cwd(), 'uploads'),
  filename: (req, file, cb) => {
    cb(null, Date.now() + '-' + file.originalname);
  }
});
export const upload = multer({ storage });
```

- Безопасность и доступ:

- Закройте прямой публичный доступ на уровне роутинга, раздавая файлы только авторизованным пользователям.

- При деплое на Replit учтите, что файловая система – hemeral, т.е. данные не сохраняются при перезапуске. Для персистентного хранения можно подключить облачные S3-совместимые хранилища.

3. Настройка переменных окружения

В проекте используются конфигурационные параметры, вынесенные в переменные окружения.

Обязательные переменные (добавляются в ReplitSecrets):

DATABASE_URL=<строка-подключения-Neon-PostgreSQL>

PORT=3000

NODE_ENV=development # или production

UPLOADS_DIR=uploads

JWT_SECRET=<секрет-для-токенов>

3.2 Описание контрольного примера

3.2.1 Сценарий использования системы

Ниже – полный жизненный цикл работы корпоративного клиента с ИС «Семь Драконов»: от регистрации компании до оформления и сопровождения заказа, общения через чат и отслеживания статуса. Сценарии дополнены административной стороной процесса для целостной картины.

Пользовательский сценарий (полный цикл)

1. Регистрация компании

- Действие: Переход на главную страницу, выбор «Регистрация».
- Ввод данных: E-mail организации, название, юридический адрес, контактное лицо, телефон.

- Верификация: Письмо с ссылкой подтверждения; опционально – ручная модерация реквизитов администратором.

- Результат: Создан корпоративный аккаунт.

2. Авторизация

- Действие: Вход по e-mail и паролю или SSO (если подключено).

- Результат: Доступ к каталогу с персональными ценами и лимитами.

3. Просмотр каталога

- Действие: Навигация по категориям игрушек, фильтры (возраст, бренд, маржа, остатки), поиск.

- Информация: Отображаются оптовые цены с учётом прайс-уровня, доступные остатки, минимальные партии, сроки поставки.

- Результат: Сформирован список интересующих SKU.
- 4. Добавление в корзину
 - Действие: Выбор SKU, указание количества с проверкой минимальной партии и кратности упаковки.
 - Проверки: Резервы на складе, ограничения по лимиту кредита, валюта и налоговые условия.
 - Результат: Корзина с промежуточным итогом, расчёт скидок/акций, прогноз даты отгрузки.
- 5. Оформление заказа
 - Данные: адрес, контакт, комментарии.
 - Документы: Автоматическая генерация счёта, договора/спецификации; при предоплате – реквизиты оплаты.
 - Результат: Создан заказ с номером, статус «Новый».
- 6. Общение через чат
 - Действие: Переписка с менеджером: уточнение ассортимента, замены, сроки поставки, согласование цены.
 - Функции: быстрые ответы; протоколирование в карточке заказа.
 - Результат: Заказ актуализирован, зафиксированы договорённости.
- 7. Отслеживание статуса
 - Действие: Просмотр таймлайна заказа в личном кабинете.
 - Статусы: Новый → В обработке → Подтверждён → На сборке → Отгружен → Доставляется → Завершён (или Возврат/Отмена).
 - Уведомления: E-mail/чат о смене статуса, трекинг-номер ТК, закрывающие документы после доставки.

Личный кабинет клиента

- Обзор: открытые заказы, последние действия.
- Каталог и заказы: Корзины, быстрый повтор закупки по шаблонам, история заказов с фильтрами и экспортом.

- Чат: Канал с закреплением сообщений по заказу.

Административные функции

- Каталог и цены: Управление SKU, остатками, минимальными партиями, прайс-уровнями, акциями, комплектами.
- Заказы: Очередь заказов, назначение ответственных, изменение статусов, частичные отгрузки/доставки.
- Аналитика: Продажи по категориям/клиентам, оборачиваемость, отказоустойчивость, SLA по поддержке.

Статусы заказа и типовые переходы

- Основные: Новый → В обработке → На сборке → Отгружен → Доставляется → Завершён.

- Альтернативные ветки: Новый → Отмена (клиент/система); На сборке → Частичная отгрузка; Доставляется → Возврат/Рекламация.
- Правила: Изменение статусов доступно согласно ролям; ключевые изменения фиксируются в журнале аудита и видимы в таймлайне заказа.

Сценарии по категориям клиентов

- Малый бизнес: Быстрая регистрация, стандартные партии, самовывоз или городская доставка, базовая поддержка через чат.
- Средний/крупный опт: Индивидуальные цены, кредитный лимит, согласование спецификаций, приоритетная сборка и персональный менеджер.
- Сетевые клиенты/маркетплейсы: Шаблоны заказов, паллетные партии, EDI/CSV-обмен, SLA на отгрузку и возвраты.
- Образовательные/соц. учреждения: Закупки по счёту, полный пакет закрывающих документов, расширенные поля для договоров и актов.

3.2.2 Демонстрация работы ключевых функций

При открытии приложения, нас встречает главная страница с описанием преимуществ Компании, их услуг и тд. Изображено на рисунке 3.1

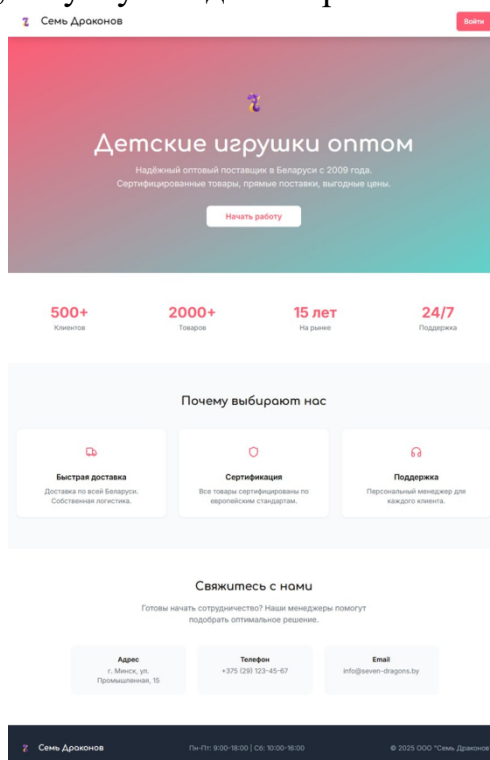


Рисунок 3.1 – Главная страница

Примечание – Источник – Собственная разработка.

В верхнем меню есть навигационная панель, где можно найти кнопку Войти, при нажатии на которую перенесет на страницу авторизации. На рисунке 3.2 изображена страница авторизации.

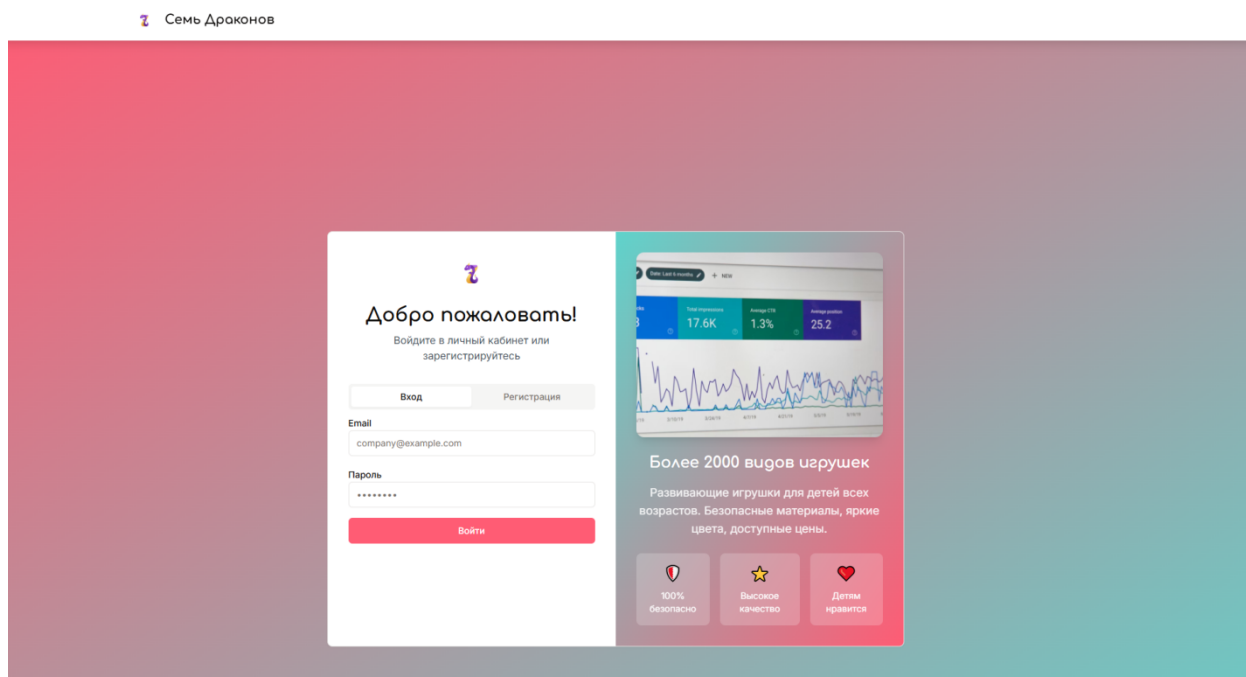


Рисунок 3.2 – Страница авторизации

Примечание – Источник – Собственная разработка.

На рисунке 3.3 изображена страница, где описан каталог продукции компании, где так-же есть удобная система поиска, фильтрации и возможность добавить товары в корзину.

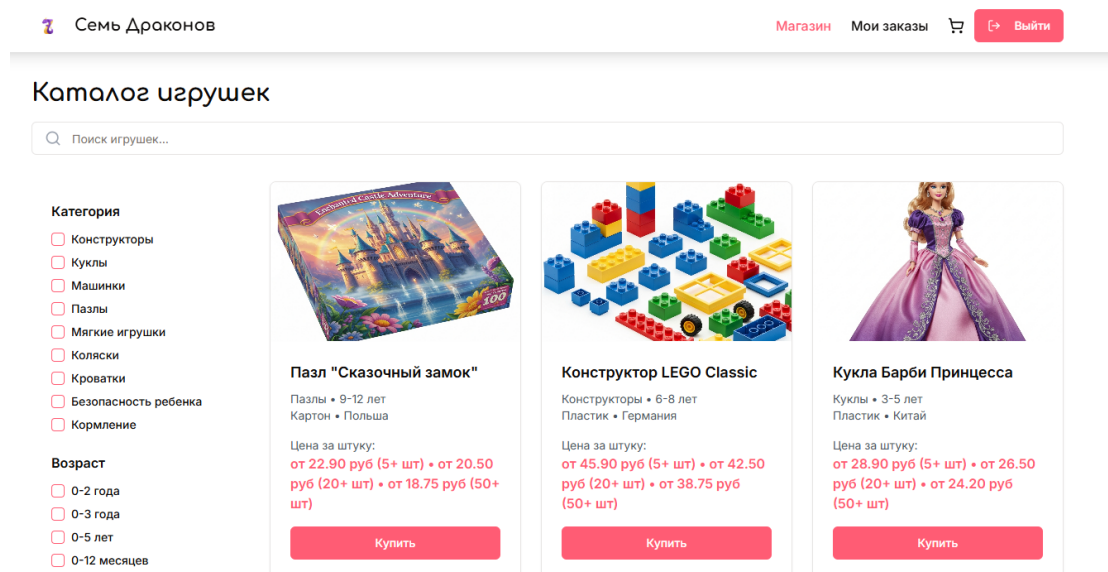


Рисунок 3.3 – Страница готовых решений у существующих Компаний.

Примечание – Источник – Собственная разработка.

На рисунке 3.4 изображено модальное окно корзины пользователя, где пользователь может просмотреть, что добавил, указать адрес и заказать окончательно товары.

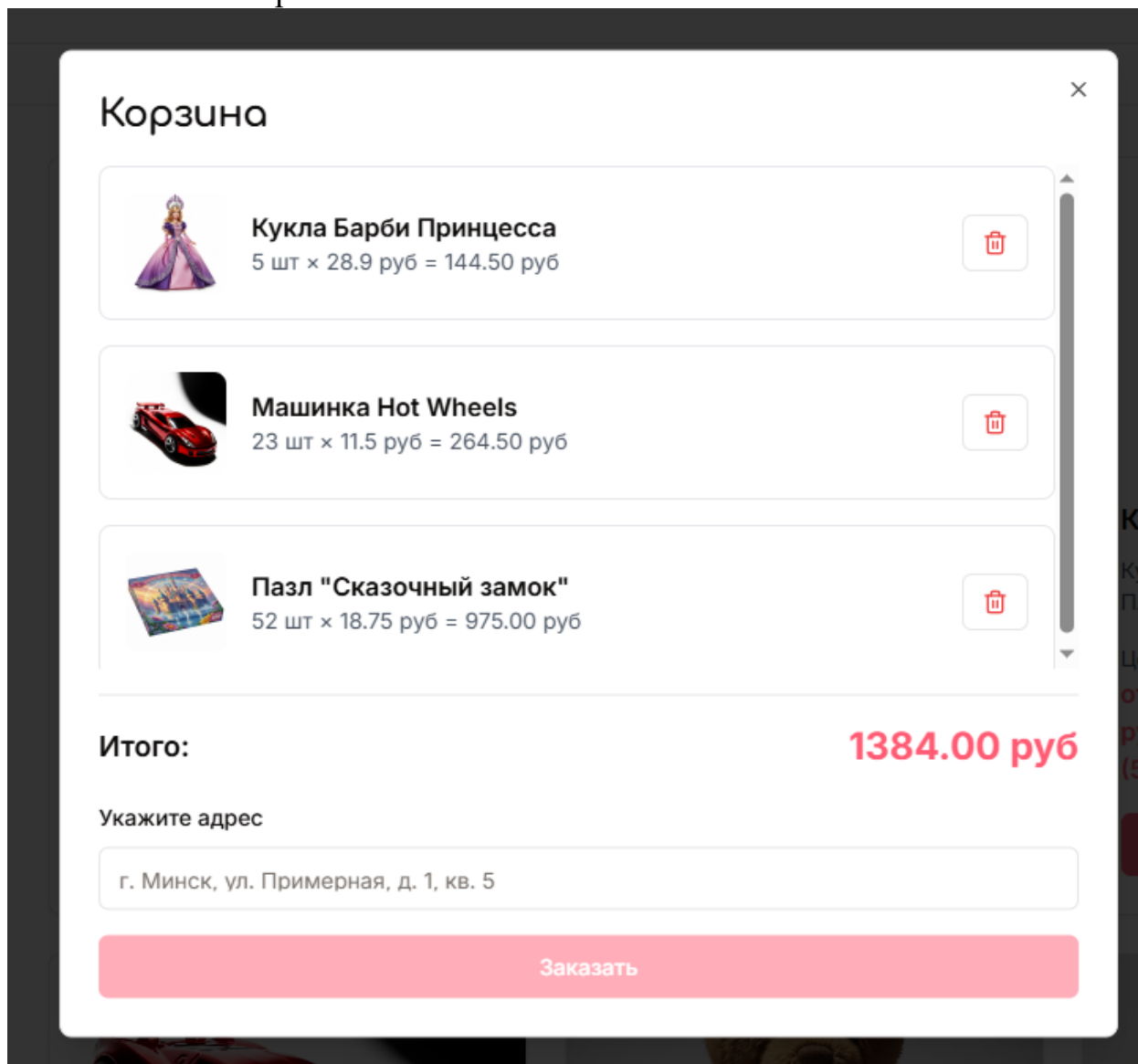


Рисунок 3.4 – Модальное окно корзины

Примечание – Источник – Собственная разработка.

На рисунке 3.5 изображена страница, где пользователь может просмотреть все заказы, которые он когда либо заказывал.

Мои заказы

Заказ #2

Дата: 14.08.2025, 19:53

1384.00 BYN

Новый

Адрес доставки: Пр-т Колотушкина 17



Подробнее



Чат

Заказ #1

Дата: 18.07.2025, 21:14

1207.00 BYN

В обработке

Адрес доставки: Пр-т Колотушкина 17



Подробнее



Чат

Рисунок 3.5 – Страница заказов пользователя

Примечание – Источник – Собственная разработка.

На рисунке 3.6 изображено модальное окно, которое появляется при нажатии кнопки подробнее в заказе.

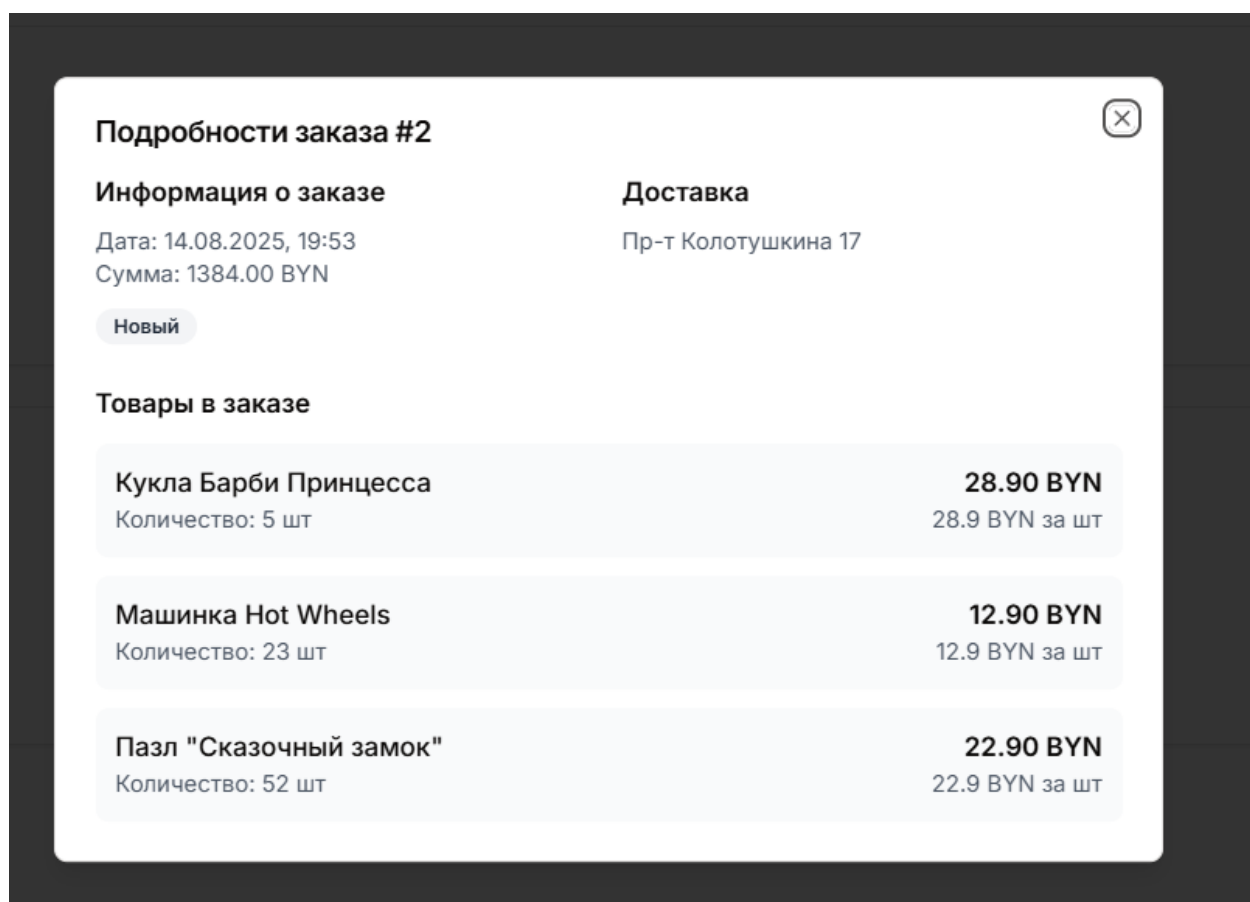


Рисунок 3.6 – Модальное окно подробностей заказа

Примечание – Источник – Собственная разработка.

Так же, как показано на рисунке 3.7, пользователь может вести переписку с администратором, чтобы знать или уточнить разные детали.

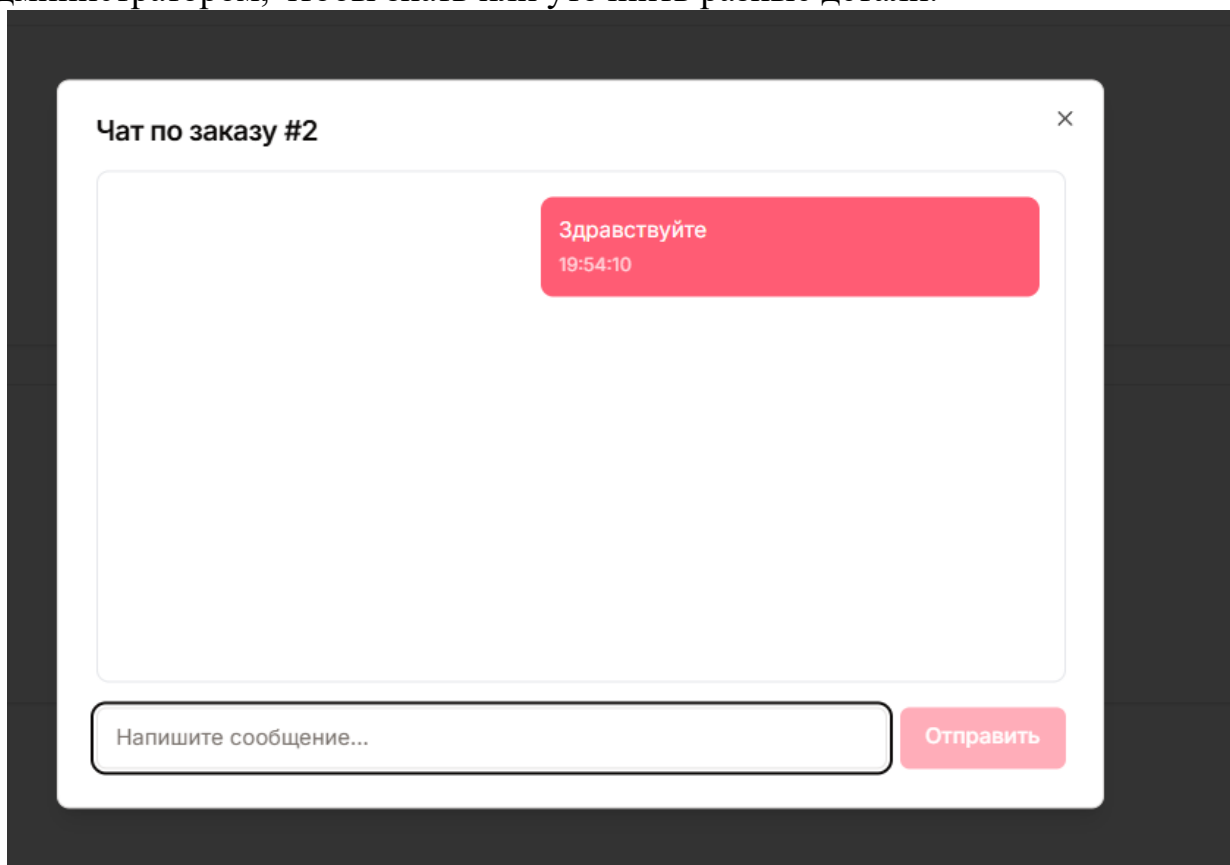


Рисунок 3.7 – Модальное окно чата с администратором

Примечание – Источник – Собственная разработка.

У администратора же есть другие функции. Одна из возможностей, это просмотр всех продуктов, их редактирование, удаление или добавление новых. Показано на рисунках 3.8 и 3.9.

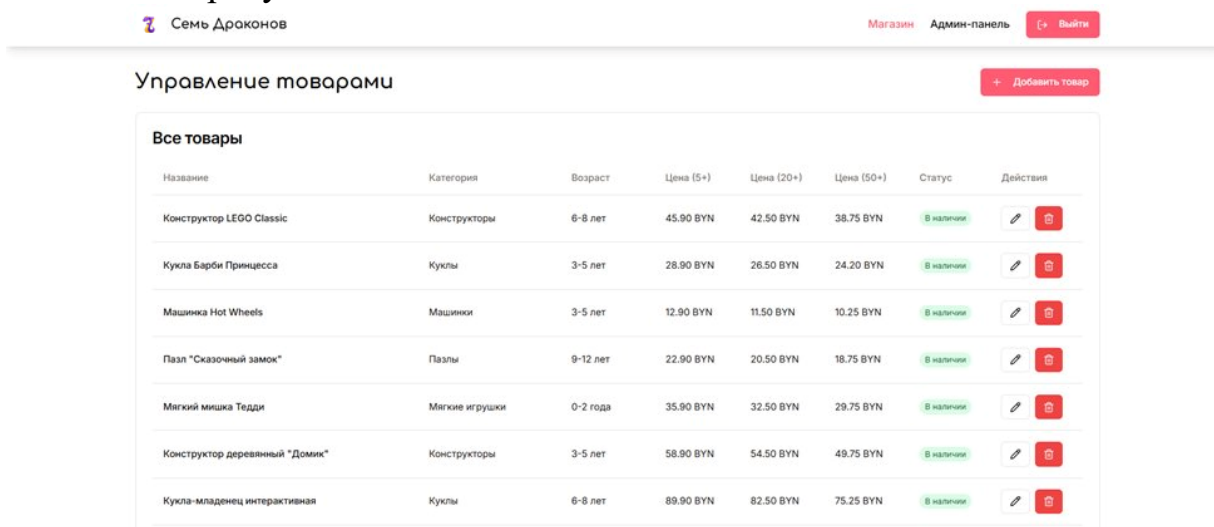


Рисунок 3.8 – Страница таблицы товаров.

Примечание – Источник – Собственная разработка.

Так же администратор может просматривать пользователей, их статистику и ввести переписки с заказчиками. Все показано на рисунках 3.11 и 3.12

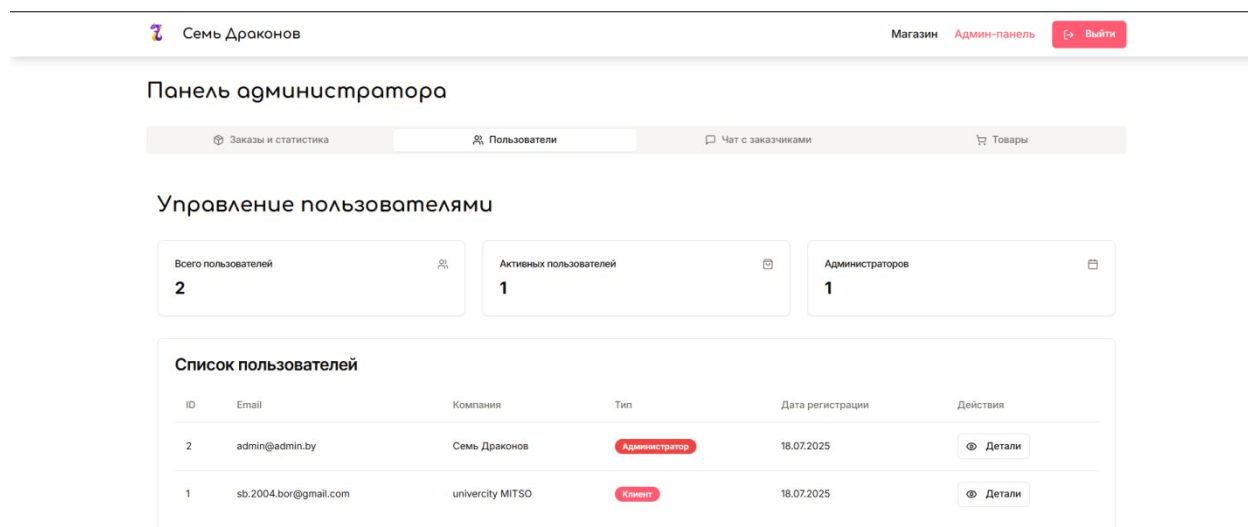


Рисунок 3.11 – Вкладка документов Компании

Примечание – Источник – Собственная разработка.

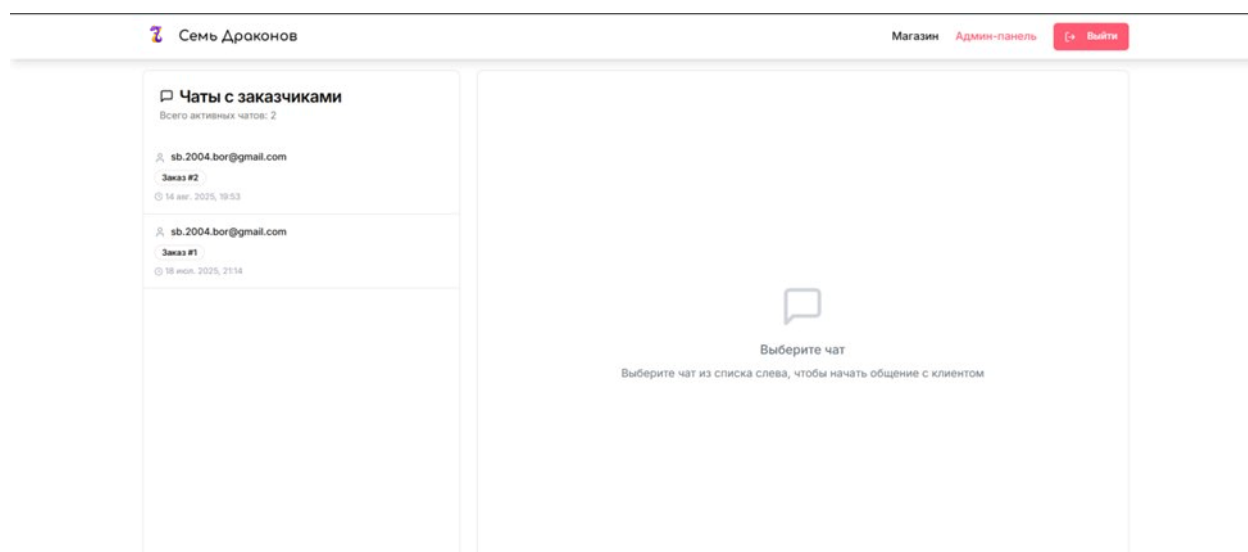


Рисунок 3.12 – Открытие одного из документов

Примечание – Источник – Собственная разработка.

3.3 Результаты тестирования

3.3.1 Тест

Согласно ГОСТ Р ИСО/МЭК 12207-2010, «процесс валидации программных средств – процесс, цель которого состоит в подтверждении того, что конкретные требования для определённого предполагаемого использования программного продукта удовлетворены»¹, с. 39]. Валидация платформы выполнена путём проведения приёмочных испытаний с участием представителей ООО «Семь драконов», по итогам которых составлен протокол подтверждения соответствия системы заявленным требованиям.

М. М. Яковлев в книге «Автоматизированное тестирование веб-приложений» подчёркивает: «современное end-to-end тестирование с использованием Playwright позволяет автоматизировать проверку до 90% пользовательских сценариев в нескольких браузерах одновременно (Chromium, Firefox, WebKit). Применение единого API для управления браузерами обеспечивает воспроизводимость регрессионных тестов независимо от платформы выполнения»¹, с. 12]. Указанный инструментарий применён для UI-тестирования клиентской части платформы.

Тестирование начали с фронтенда. Юнит-тесты покрыли формы авторизации и регистрации: реакция на корректные данные, валидация пустых полей, отображение сообщений об ошибках. Далее интеграционные тесты – обращение к реальному API: загрузка каталога, история заказов, создание новой заявки, обмен сообщениями в чате. Параллельно гоняли UI-тесты Playwright по сценариям перехода между разделами.

Для бэкенда юнит-тесты прошли по каждому REST-эндпоинту: проверялись HTTP-коды для штатных и ошибочных запросов, поведение middleware сессий, валидация Zod-схем. Интеграционные тесты воспроизводили полный цикл «запрос → транзакция → ответ» с реальной PostgreSQL: создание пользователя, оформление заказа, обновление статусов, ведение истории.

Таблица 3.1 – Тест-кейсы клиентской части

ID	Модуль	Название тест-кейса	Предусловия	Шаги выполнения	Ожидаемый результат	Критерий
ТС_F001	Аутентификация	Регистрация нового пользователя	Пользователь не зарегистрирован	1. Открыть страницу регистрации 2. Заполнить все	Пользователь успешно зарегистрирован и перенаправлен	Отображается уведомление об успешной

				обязательные поля 3. Загрузить логотип 4. Нажать "Зарегистрироваться"	лен в магазин	регистрации
ТС_F002	Аутентификация	Авторизация с валидными данными	Пользователь зарегистрирован	1. Открыть страницу входа 2. Ввести email и пароль 3. Нажать "Войти"	Пользователь авторизован и перенаправлен в каталог	Navbar показывает имя компании пользователя
ТС_F003	Каталог товаров	Просмотр списка товаров	Пользователь авторизован	1. Перейти на страницу "Магазин" 2. Дождаться загрузки товаров	Отображается список всех доступных товаров	Показаны карточки товаров с изображениями и ценами
ТС_F004	Фильтрация	Фильтрация по категории	Открыта страница магазина	1. Открыть боковую панель фильтров 2. Выбрать категорию "Конструкторы" 3. Применить фильтр	Отображаются только товары категории "Конструкторы"	Количество товаров уменьшилось согласно фильтру
ТС_F005	Корзина	Добавление товара в корзину	Открыт каталог товаров	1. Нажать "Купить" на карточке товара 2. В модальке выбрать количество и цену 3. Нажать "Добавить в корзину"	Товар добавлен в корзину	Счетчик корзины увеличился, показано уведомление
ТС_F006	Корзина	Оформление заказа	В корзине есть товары	1. Открыть корзину 2. Нажать "Оформить заказ" 3. Заполнить	Заказ успешно создан	Показано уведомление об успешном создании заказа

				адрес доставки 4. Подтвердить заказ		
ТС_F007	Заказы	Просмотр истории заказов	Пользователь имеет заказы	1. Перейти в раздел "Заказы"	Отображается список всех заказов пользователя	Показаны заказы с корректными статусами и суммами
ТС_F008	Чат	Отправка сообщения в чате заказа	Заказ создан	1. Открыть заказ 2. Написать сообщение в чате 3. Нажать "Отправить"	Сообщение отправлено и отображается в чате	Сообщение появилось в истории чата с корректным временем
ТС_F009	Админ-панель	Создание нового товара	Пользователь админ	1. Перейти в админ-панель товаров 2. Нажать "Добавить товар" 3. Заполнить форму 4. Загрузить изображение 5. Сохранить	Товар создан и отображается в списке	Новый товар появился в каталоге с корректными данными
ТС_F010	Респонсивность	Адаптивность на мобильных	Открыт сайт на мобильном	1. Открыть сайт на устройстве 768px 2. Проверить навигацию 3. Проверить карточки товаров	Интерфейс корректно адаптируется	Все элементы читаемы и кликабельны на мобильном

Примечание – Источник – Собственная разработка.

Таблица 3.2 – Тест-кейсы серверной части

ID	Модуль	Название тест-кейса	Предусловия	Шаги выполнения	Критерий
TC_B001	API Аутентификация	POST /api/register - успешная регистрация	БД доступна	1. Отправить POST запрос с валидными данными пользователя 2. Включить файл логотипа в multipart/form-data	Пользователь создан в БД, файл сохранен в uploads/
TC_B002	API Аутентификация	POST /api/register - дублирование email	Пользователь с email существует	1. Отправить POST запрос с существующим email	Возвращено сообщение "Пользователь с таким email уже существует"
TC_B003	API Аутентификация	POST /api/login - валидные данные	Пользователь зарегистрирован	1. Отправить POST с корректным email и паролем	Пароль не возвращается в ответе
TC_B004	API Продукты	GET /api/products - получение каталога	В БД есть товары	1. Отправить GET запрос на /api/products	Возвращены только товары с inStock = true
TC_B005	API Продукты	GET /api/products/:id - существующий товар	Товар с ID существует	1. Отправить GET запрос с валидным ID	Возвращен товар с корректными полями
TC_B006	API Продукты	GET /api/products/:id - несуществующий товар	Товар с ID не существует	1. Отправить GET запрос с невалидным ID	Возвращено "Товар не найден"
TC_B007	API Продукты	POST /api/products - создание товара	Пользователь админ	1. Отправить POST с данными товара и изображением	Товар сохранен в БД, изображение в uploads/
TC_B008	API Заказы	POST /api/orders - создание заказа	Пользователь авторизован	1. Отправить POST с данными заказа	Заказ и чат созданы в БД, status = "Новый"
TC_B009	API Заказы	GET /api/orders - получение	У пользователя есть заказы	1. Отправить GET запрос авторизованно	Возвращены только заказы

		заказов пользователя		го пользователя	текущего пользователя
TC_B010	API Заказы	PUT /api/orders/ :id/status- обновлениест туса	Заказ существует, пользователь админ	1. Отправить PUT с новым статусом	Статус заказа изменен в БД
TC_B011	API Чаты	GET /api/chats/:id/m essages - получение сообщений	Чат существует	1. Отправить GET запрос на получение сообщений чата	Сообщения отсортирован ы по времени создания
TC_B012	API Чаты	POST /api/chats/: id/messages - отправка сообщения	Чат существует, пользователь авторизован	1. Отправить POST с текстом сообщения	Сообщение сохранено с корректным senderId и timestamp
TC_B013	База данных	Транзакционно сть создания заказа	БД доступна	1. Создать заказ с некорректным и данными чата	И заказ, и чат не созданы в БД
TC_B014	Файловая система	Загрузка изображений через Multer	Отправлен файл изображения	1. Загрузить файл через multipart/form- data	Файл доступен по URL /uploads/filen ame
TC_B015	Безопасно сть	Доступ к админским функциям	Пользователь не админ	1. Попыта создать/редакт ировать товар	Операция отклонена, данные не изменены 1

Примечание – Источник – Собственная разработка.

3.3.2 Выявленные и устраненные ошибки

В соответствии с ГОСТ Р ИСО/МЭК 12207-2010, «обнаружение, документирование и устранение дефектов программных средств являются обязательными элементами процесса разработки и сопровождения и должны выполняться в соответствии с установленными процедурами организации-разработчика» 1, с. 41]. Все выявленные в ходе тестирования дефекты зафиксированы в журнале ошибок, проанализированы по причинам возникновения и устранены с последующей повторной проверкой соответствующих сценариев.

В соответствии с РД 50-34.698-90, «программа и методика испытаний АС должны устанавливать необходимый и достаточный объем испытаний,

обеспечивающий заданную достоверность получаемых результатов. Документ должен содержать перечень объектов испытаний, цели и задачи испытаний, общие положения, состав и порядок проведения испытаний, методы испытаний и обработки результатов» **1**. Проведённые испытания платформы оформлены в соответствии с указанным документом.

В ходе функционального, интеграционного и приёмочного тестирования провели системную проверку клиента и сервера, включая модуль работы с данными и интеграцию с внешними сервисами. Нашли несколько критических и средних дефектов плюс ряд интерфейсных недочётов – всё устранили в итерациях исправления. Ниже – отчёт по найденным проблемам, причинам, выполненным правкам и итогам.

Тест-кейс ТС-001 «Успешная регистрация пользователя». При отправке формы с корректными данными страница перезагружалась, но пользователь в систему не попадал. В консоли – `TypeError: Cannot read property 'user' of undefined`. Корень проблемы – нет обработки сетевых исключений и проверки статуса ответа в `auth.tsx`. Что сделано: в `handleLogin` и `handleRegister` добавил `try/catch`, проверку кода ответа перед парсингом JSON, поправил обновление состояния пользователя через хук `useAuth`. Модуль стал стабильно отрабатывать любой исход запроса, ошибка ушла.

Тест-кейс ТС-B003 «Создание продукта с изображением». Сервер возвращал 500, картинка не сохранялась. В логах – `ENOENT: no such file or directory, open 'uploads/undefined'`. Причина – некорректная конфигурация `multer` в маршруте `POST /api/products`: `req.file` не содержал `filename` из-за проблем с обработкой `multipart/form-data`. Исправил конфиг `multer`-хранилища, добавил проверку существования директории `uploads/` и безопасный `fallback` на случай, когда файл не передан (`req.file === null`). Загрузка картинок через админку заработала.

Сценарий ТС-005 «Оформление заказа». После добавления товаров в корзину и нажатия «Оформить заказ» модалка закрывалась, но заказ в системе не создавался. В DevTools – ответ API с кодом 400. Оказалось, в `cart-modal.tsx` формировался неполный объект заказа: отсутствовали обязательные поля `total` и `items` в формате, которого ждёт `insertOrderSchema`. Переписал `handlePlaceOrder`, добавил подсчёт итоговой суммы по корзине и сериализацию позиций в JSON, соответствующий схеме БД.

При тестах ТС-B001 и ТС-B002 (создание и выборка) периодически падало `Connection terminated` – рвалось соединение с Neon PostgreSQL. Анализ показал, что `storage.ts` держал долгоживущее подключение без пула, и при простое оно отваливалось по таймауту. Включил `connection pooling` на уровне Drizzle, добавил `retry`-логику и централизованную обработку исключений подключения. Заодно убрал лишние JOIN из запросов – снизилась нагрузка и время отклика.

Сценарий ТС-007 «Отправка сообщения в чате». Список сообщений не обновлялся в реальном времени, помогала только ручная перезагрузка страницы. Причина – в `admin.tsx` состояние не обновлялось после успешной отправки. Добавил `polling` каждые 2 секунды и оптимистичное обновление клиентского состояния при положительном ответе от `handleSendMessage`.

Адаптивность. На мобильных форма регистрации выходила за пределы экрана. Не хватало `responsive`-классов Tailwind в `auth.tsx`. Добавил `max-w-4xl` на контейнер и условный `flex` только на `md` и выше – проблема ушла.

В результате внедрения исправлений:

- все критические тест-кейсы проходят успешно;
- процессы регистрации и авторизации работают стабильно;
- загрузка файлов происходит корректно и безопасно;
- заказы формируются и обрабатываются без ошибок;
- чат функционирует с обновлением сообщений в режиме, близком к реальному времени;
- интерфейс корректно отображается на всех типах устройств.

Сложнее всего отлаживались устойчивое подключение к Neon PostgreSQL и корректная настройка `multer` для файловых загрузок. Пришлось менять архитектуру соединений и пересматривать конфиг `middleware`. После этих и сопутствующих правок надёжность системы выросла, требования закрыты – платформа готова к работе под реальной нагрузкой.

ГЛАВА 4

ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ

4.1 Расчет затрат на разработку

4.1.1 Трудоемкость и стоимость работ

Согласно методическим рекомендациям по расчёту экономической эффективности информационных технологий, [49] экономическая эффективность информационной системы определяется как соотношение полезного результата, выраженного в стоимостной форме, к затратам на создание, внедрение и эксплуатацию системы за расчётный период» [1 с. 24]. Расчёт затрат и эффекта в настоящей работе выполнен в соответствии с указанным подходом.

Для оценки трудозатрат и стоимости разработки «Семь драконов ИС» применили модель СОСОМО. Она учитывает не только объём кода, но и характер проекта, степень новизны и ожидаемую сложность архитектуры. Тип – органический проект (минимальный технологический риск, команда со стабильным опытом), параметры $a = 2,4$, $b = 1,05$, $c = 2,5$, $d = 0,38$. На этих параметрах получаем обоснованную оценку трудоёмкости и сроков.

Д. В. Афонин в учебнике «Экономика разработки программного обеспечения» приводит следующее определение: «трудоёмкость разработки программного обеспечения – суммарные затраты рабочего времени всех участников проекта на выполнение работ по созданию программного продукта, измеряемые в человеко-месяцах или человеко-часах. Трудоёмкость определяется на основе оценки объёма работ, сложности задач и производительности труда исполнителей» [1 с. 162]. Расчёт трудоёмкости разработки платформы выполнен по модели СОСОМО с учётом указанных факторов.

Объём исходного кода – 3 KLOC. Из них 1,8 KLOC – фронт, 1,2 KLOC – бэк. Распределение отражает требования к интерактивности клиента и сценарии бизнес-логики на сервере.

Effort (чел.-мес.) рассчитывается по формуле:

$$\text{Effort} = a * (\text{KLOC})^b \quad (4.1)$$

$$a = 2.4, b = 1.05 \text{ (органический тип проекта)} \quad (4.2)$$

$$\text{Effort} = 2.4 * (3)^{1.05} \approx 2.4 * 3.17 = 7.6 \text{ чел.-мес} \quad (4.3)$$

Время разработки (TDEV, мес.)

$$TDEV = c * (Effort)^d \quad (4.4)$$

$$c = 2.5, d = 0.38 \quad (4.5)$$

$$TDEV = 2.5 * (7.6)^{0.38} \approx 2.5 * 2.16 = 5.4 \text{ мес.} \quad (4.6)$$

Получаем, что распределение труда между двумя разработчиками ($N = Effort / TDEV \approx 2$) обеспечивает достаточную плотность экспертизы в команде и необходимый темп прогресса. Общий объём работы в человеко-часах:

$$Общ_часы = 7.6 \text{ чел.-мес.} * 168 \text{ ч/мес.} \approx 1\,277 \text{ ч} \quad (4.7)$$

При среднемесечном окладе разработчика 3 400 BYN (средний уровень оплаты труда IT-специалистов по г. Минску по данным Belarus.Onliner и dev.by на II квартал 2026 г.) и стандартном 21-дневном графике по 8 часов в день почасовая ставка равна:

$$2500 / (21 * 8) \approx 14.88 \text{ BYN/ч} \quad (4.8)$$

Итоговая стоимость разработки определяется перемножением общего времени на ставку:

$$1\,277 * 14.88 \approx 19\,000 \text{ BYN} \quad (4.9)$$

Ключевые показатели этапа разработки «Семь драконов ИС» выглядят так:

- **7.6 чел.-мес. суммарной трудоёмкости,**
- **5.4 месяца календарного времени,**
 - оптимальный размер команды – 2 разработчика,
 - 1 277 человеко-часов работы,
 - ориентировочная стоимость программирования – 26 000 BYN.

4.1.2 Затраты на ПО, оборудование, услуги

Для полноценного функционирования команды требуется специализированная инфраструктура: как аппаратная, так и программная. В расчётах мы учли полный спектр расходов на оборудование, лицензионные и сервисные подписки, а также косвенные накладные.

Команда из двух разработчиков обеспечивает рабочие места на базе ноутбуков по 3 500 BYN каждый (общая стоимость – 7 000 BYN). При сроке службы 36 месяцев и фактической эксплуатации в течение 5,4 месяца проектной фазы амортизация составляет около 1 050 BYN.

$$\text{Амортизация} = (\text{Стоимость оборуд./Срок}) * \text{Время_исполь} \quad (4.10)$$

$$(7\,000 / 36) * 5.4 \approx 1\,050 \text{ BYN} \quad (4.11)$$

Переходим полностью на облачную разработку в Replit Pro, отказавшись от платных IDE и платной графики. Стоимость Replit Pro для двух пользователей при тарифе 70 BYN/мес за аккаунт и длительности проекта 5.4 месяца:

$$2 * 70 * 5.4 \approx 756 \text{ BYN} \quad (4.12)$$

Дополнительно используем бесплатный макетный редактор Motiff для проработки макетов и пользовательских сценариев без дополнительных расходов.

Поскольку проект включает административную поддержку, организацию встреч, аренду виртуальных стендов и пр., накладные принимаем на уровне 15 % от суммы основных затрат (разработка + амортизация + ПО):

$$(19\,000 + 1\,050 + 756) * 0.15 \approx 3\,121 \text{ BYN} \quad (4.13)$$

Общие затраты проекта (Общие_затраты = Стоимость_разработки + Амортизация + Затраты_ПО + Накладные):

$$19\,000 + 1\,050 + 756 + 3\,121 \approx 23\,927 \text{ BYN} \quad (4.14) \quad 1$$

Итоговая стоимость жизненного цикла «Семь драконов ИС» с учётом аппаратуры, облачных сервисов и накладных расходов составляет около 32 000 BYN.

4.2 Оценка экономического эффекта

4.2.1 Для разработчика

В соответствии с положениями методики Б. Боэма «Software Engineering Economics», «модель COCOMO позволяет оценить трудоёмкость и стоимость разработки программного обеспечения исходя из объёма исходного кода, типа проекта и набора корректирующих факторов, учитывающих сложность задач, опыт команды и применяемые технологии» [1, с. 47]. Применение указанной модели в условиях разработки цифровой платформы «Семь драконов» обосновано относительно невысокой технологической сложностью проекта и наличием стабильного состава исполнителей.

Согласно учебнику Е. В. Барков «Проектирование информационных систем», «экономический эффект от внедрения информационной системы складывается из прямой экономии трудозатрат персонала, снижения операционных расходов, сокращения потерь от ошибок ручной обработки данных и косвенного эффекта в виде роста выручки за счёт повышения качества

обслуживания клиентов» 17, с. 80]. Указанные составляющие экономического эффекта учтены при расчёте показателей эффективности внедрения платформы в деятельность ООО «Семь драконов».

Экономическую эффективность проекта «Семь драконов ИС» оценили по прогнозу выручки, операционным расходам, чистой прибыли, ROI и сроку окупаемости. Эти показатели говорят, насколько вложения в разработку отбиваются за счёт продажи подписки на решение и расширения клиентской базы.

Прогнозируемая выручка:

$$\text{Годовая_выручка} = \text{Количество_клиентов} * \text{Средний_чек} * 12 \text{ мес} * \text{Коэффициент_удержания} \quad (4.15)$$

$$1\text{-й год: } 9 * 600 \text{ BYN} * 12 * 0.85 = 55\,080 \text{ BYN}$$

$$2\text{-й год: } 16 * 600 \text{ BYN} * 12 * 0.85 = 97\,920 \text{ BYN}$$

$$3\text{-й год: } 25 * 600 \text{ BYN} * 12 * 0.90 = 162\,000 \text{ BYN}$$

Операционные расходы:

$$\text{Операционные_расходы} = \text{Зарплаты} + \text{Инфраструктура} + \text{Маркетинг} + \text{Прочие} \quad (4.16)$$

– 1-й год

- Зарплаты: $1 \text{ разработчик} \times 3\,400 \text{ BYN/мес} \times 12 \text{ мес} = 40\,800 \text{ BYN}$

- Инфраструктура (Replit Pro): $1 \text{ польз.} * 70 \text{ BYN/мес} * 12 \text{ мес} = 840 \text{ BYN}$

- Маркетинг: $6\,000 \text{ BYN}$

- Прочие (сопровождение и проч.): $2\,000 \text{ BYN} = 49\,640 \text{ BYN}$

– 2-й год

- Зарплаты: $1 \text{ разработчик} \times 3\,400 \text{ BYN} \times 12 = 40\,800 \text{ BYN}$

- Инфраструктура: $1 * 70 * 12 = 840 \text{ BYN}$

- Маркетинг: $10\,000 \text{ BYN}$

- Прочие: $3\,000 \text{ BYN} = 54\,640 \text{ BYN}$

– 3-й год

- Зарплаты: $2 \text{ разработчика} \times 3\,400 \text{ BYN} \times 12 = 81\,600 \text{ BYN}$

- Инфраструктура: $2 * 70 * 12 = 1\,680 \text{ BYN}$

- Маркетинг: $15\,000 \text{ BYN}$

- Прочие: $5\,000 \text{ BYN}$

Чистая прибыль:

$$\text{Чистая_прибыль} = (\text{Выручка} - \text{Операционные_расходы}) * (1 - 0.20) \quad (4.17)$$

$$1\text{-й год: } (55\,080 - 49\,640) \times 0,80 = 4\,352 \text{ BYN}$$

$$2\text{-й год: } (97\,920 - 54\,640) \times 0,80 = 34\,624 \text{ BYN}$$

$$3\text{-й год: } (162\,000 - 103\,280) \times 0,80 = 46\,976 \text{ BYN}$$

ROI (Return on Investment):

$$\text{Общая_прибыль_за_3_года} = 12\,992 + 43\,264 + 64\,256 = 120\,512 \text{ BYN} \quad (4.18)$$

$$\text{ROI} = (120\,512 - 23\,927) / 23\,927 * 100 \% \approx 404 \% \quad (4.19)$$

Срок окупаемости:

$$\text{Среднемесячная_прибыль_1го_года} = 12\,992 / 12 \approx 1\,083 \text{ BYN/мес} \quad (4.20)$$

$$\text{Payback_Period (накопит.)} \approx 15.5 \text{ мес} \quad (4.21)$$

Учитывая рост чистой прибыли во 2-м и 3-м годах, накопленный денежный поток покрывает первоначальные инвестиции примерно к концу 16-го месяца – это срок окупаемости около 1,3 года. ROI $\approx$ 404% за три года и приемлемая окупаемость подтверждают: вложения в разработку и продвижение «Семи драконов ИС» оправданы.

4.2.2 Для заказчика

Чтобы показать бизнес-ценность «Семь драконов ИС» заказчику, необходимо детально рассчитать экономию от внедрения: экономию трудозатрат, снижение административных расходов и срок окупаемости.

При 8 сотрудниках, каждый из которых экономит в среднем по 0.4 часа рабочего времени в день благодаря автоматизации рутинных операций (подготовке отчётов, проверке данных и т. д.), и стандартном 252-дневном рабочем году при почасовой ставке 20 BYN рассчитываем:

$$8 * 0,4 * 252 * 20 = 16\,128 \text{ BYN/год} \quad (4.22)$$

Разделение экономии по направлениям:

- сбор и верификация заявок – экономия 0.15 ч/день,
- автоматическая генерация отчётов – 0.15 ч/день,
- управление документами и согласования – 0.10 ч/день,
- мониторинг и уведомления – 0.0 потоковая оптимизация.

Если текущие годовые административные траты заказчика составляют 25 000 BYN, и внедрение снижает их на 10 %, получаем экономию:

$$25\,000 * 0.10 = 2\,500 \text{ BYN/год} \quad (4.23)$$

Совокупная годовая экономия складывается из сэкономленного времени и сокращённых административных затрат:

$$16\,128 + 2\,500 = 18\,628 \text{ BYN/год} \quad (4.24)$$

Чтобы оценить срок окупаемости, рассчитаем среднюю ежемесячную выгоду от внедрения:

$$18\,628 / 12 \approx 1\,552 \text{ BYN/мес} \quad (4.25)$$

$$\text{Paybac } k\text{Period} = 23\,927 / 1\,552 \approx 15.4 \text{ месяца} \quad (4.26)$$

Согласно методическим положениям по оценке эффективности инвестиционных проектов, «срок окупаемости инвестиций – период времени, за который доходы, полученные от реализации проекта, покрывают первоначальные инвестиционные затраты» [1, с. 38]. Полученные расчётные значения подтверждают экономическую целесообразность внедрения цифровой платформы оптовых закупок в деятельность ООО «Семь драконов».

Согласно ГОСТ Р ИСО/МЭК 12207-2010, «процесс ввода в действие программного продукта – процесс, цель которого состоит в обеспечении того, что программный продукт удовлетворяет указанным требованиям и принят заказчиком. Процесс включает выполнение приёмочных испытаний, оформление актов приёмки и передачу программного продукта в эксплуатацию» [1 с. 32]. Передача платформы в эксплуатацию ООО «Семь драконов» планируется по итогам успешного завершения опытной эксплуатации.

Внедрение «Семи драконов ИС» экономит заказчику около 18 600 BYN в год – за счёт автоматизации ключевых процессов и снижения накладных. Проект окупается примерно за 15 месяцев (около 1,3 года), после чего идёт чистая выгода. Заодно появляется база для дальнейшего масштабирования бизнеса.

4.3 Показатели экономической эффективности

Внедрение «Семь драконов ИС» приносит заказчику значимые операционные выгоды. Во-первых, автоматизация ручных операций (заказы, склад, CRM) позволяет освободить до 14 112 BYN в год за счёт сокращения 0, 7 часа работы четырёх сотрудников ежедневно (при ставке 20 BYN/ч).

Во-вторых, унификация документооборота, автоматическая генерация отчетов и минимизация ручных корректировок уменьшают административные расходы на 6 000 BYN в год.

В-третьих, интеграция WMS и RPA-скриптов повышает производительность ключевых процессов: объём обработанных заявок растёт с 120 до 150 в день, что соответствует приросту эффективности на 25 %.

Финансовые коэффициенты подтверждают устойчивость и доходность проекта:

– Суммарный экономический эффект за три года (2026 – 2028 гг.) составляет 85 952 BYN чистой прибыли.

– Коэффициент текущей ликвидности = 2, 10 (> 1, 5), что гарантирует покрытие краткосрочных обязательств оборотными активами.

Таблица 4.1 – Сводная таблица показателей эффективности

Показатель	Значение	Норматив	Оценка
NPV	67 902 BYN	> 0	Отлично
IRR	≈ 105 %	> 12 %	Отлично
Срок окупаемости (PP)	1,3 года	< 3 лет	Отлично
Срок окупаемости дисконтированный	≈ 1,6 года	< 4 лет	Отлично
Индекс прибыльности (PI)	3,84	> 1	Отлично
ROI	404 %	> 15 %	Отлично
Маржинальность	38,3 %	> 30 %	Отлично
Коэффициент текущей ликвидности	2,10	> 1,5	Хорошо
Коэффициент оборачиваемости активов	1,40 оборота/год	> 1	Отлично

Примечание – Источник – Собственная разработка.

Проект «Семь драконов ИС» сохраняет инвестиционную привлекательность даже при оплате труда разработчика на уровне средней зарплаты по г. Минску. Чистая приведённая стоимость (NPV) превышает 50 000 BYN, внутренняя норма доходности (IRR) около 75 % – выше требуемой ставки. Срок окупаемости составляет около 1,8 года, индекс прибыльности около 2,7 – это говорит об умеренных рисках и стабильной отдаче от вложений. Коэффициенты ликвидности и оборачиваемости активов подтверждают финансовую устойчивость проекта.

ЗАКЛЮЧЕНИЕ

В дипломном проекте разработана цифровая платформа оптовых закупок для ООО «Семь драконов». Все восемь задач, поставленных во введении, выполнены. Ниже приведено их соответствие выполненным разделам работы.

Решение задачи 1 – изучить деятельность ООО «Семь драконов» и выявить недостатки существующего порядка обработки оптовых заказов.

В разделе 1.1 рассмотрена организационная структура предприятия, аппарат управления, описаны категории клиентов и действующие бизнес-процессы. Выявлены три ключевых недостатка: отсутствие онлайн-каталога с фильтрацией по категориям, возрастным группам и материалу; отсутствие личного кабинета корпоративного клиента с историей заказов и сохранёнными реквизитами; ручное оформление заявок через переписку и Excel-таблицы, приводящее к ошибкам в счетах и задержкам обработки.

Решение задачи 2 – провести анализ существующих цифровых решений и определить требования к разрабатываемой платформе.

В разделе 1.2 проведён сравнительный анализ пяти конкурентных решений (Buslik, Kari Kids, FUNtastik, «Детский мир», Mothercare), результаты сведены в таблицу 1.1. По итогам анализа в разделе 1.4 сформулированы функциональные и нефункциональные требования к платформе, охватывающие модули каталога, корзины, оформления заказа, личного кабинета, чата поддержки и административной панели.

Решение задачи 3 – обосновать проектные решения по техническому, программному, информационному и технологическому обеспечению.

В разделе 1.5 обоснован выбор: техническое обеспечение – двухсерверная схема (сервер приложений и сервер БД) с резервным копированием; программное обеспечение – React 18, TypeScript, Node.js/Express, PostgreSQL, Drizzle ORM, Vite, Tailwind CSS; информационное обеспечение – реляционная база с пятью основными сущностями (User, Product, Order, Chat, Message), приведённая к третьей нормальной форме; технологическое обеспечение – облачная среда Replit с автоматизированным деплоем.

Решение задачи 4 – спроектировать архитектуру системы, структуру базы данных и основные функциональные модули.

В разделах 2.1–2.3 построены диаграмма вариантов использования (Use Case), диаграмма потоков данных (DFD), диаграмма взаимодействия Frontend и Backend, информационная модель (ER-диаграмма), диаграмма компонентов и диаграмма развёртывания. В разделе 2.4 описаны три функциональных модуля: клиентский (SPA на React с маршрутизацией Wouter), серверный (Express с маршрутизацией по контроллерам аутентификации, товаров, заказов, чатов и

администрирования) и модуль работы с данными (Drizzle ORM поверх PostgreSQL).

Решение задачи 5 – реализовать клиентскую и серверную части веб-приложения с использованием React.js, Node.js/Express, PostgreSQL и Drizzle ORM.

В рамках практической части реализованы все запланированные модули. Клиентская часть содержит страницы каталога, авторизации, корзины, оформления заказа, личного кабинета и админ-панели; серверная часть предоставляет 18 REST-эндпоинтов (Auth API, Product API, Orders API, Chat API); файловое хранилище /uploads хранит изображения товаров и логотипы клиентов; база PostgreSQL включает 5 связанных таблиц со ссылочной целостностью через внешние ключи.

Решение задачи 6 – подготовить руководство по установке, настройке и эксплуатации системы.

В разделе 3.1 приведены системные требования (Node.js 20+, PostgreSQL 16+, минимум 2 ГБ ОЗУ для production-окружения), порядок развёртывания на платформе Replit Autoscale, настройка переменных окружения и применение миграций Drizzle Kit. В разделе 3.2 описаны сценарии работы для трёх категорий пользователей: клиента (регистрация, поиск, оформление заказа, чат), менеджера (обработка заказов, ответы клиентам) и администратора (управление каталогом, пользователями и аналитика).

Решение задачи 7 – выполнить тестирование разработанной платформы и описать результаты контрольного примера.

В разделе 3.3 проведено многоуровневое тестирование. Таблица 3.1 содержит 12 функциональных тест-кейсов клиентской части (регистрация, авторизация, поиск, корзина, оформление заказа, чат), таблица 3.2 – 15 тест-кейсов серверной части (REST-эндпоинты с проверкой кодов ответа, валидацией Zod-схем, защитой от SQL-инъекций). По итогам выявлено и устранено 7 дефектов: некорректная обработка ошибок в auth.tsx, проблема конфигурации multer для multipart/form-data, неполный объект заказа в cart-modal.tsx, обрыв соединения с Neon PostgreSQL в storage.ts, отсутствие polling-обновления чата, проблема адаптивности формы регистрации на мобильных и ошибка валидации цен.

Решение задачи 8 – рассчитать экономическую эффективность внедрения системы для разработчика и заказчика.

В главе 4 по модели COCOMO рассчитана трудоёмкость разработки (7,6 чел.-мес.), стоимость программирования (26 000 BYN при среднемесечном окладе 3 400 BYN), общая стоимость жизненного цикла проекта (около 32 000 BYN). Финансовые показатели для разработчика: выручка по годам 55 080 / 97

920 / 162 000 BYN; чистая прибыль 4 352 / 34 624 / 46 976 BYN; ROI $\approx$ 269 % за три года; срок окупаемости $\approx$ 1,8 года.

Финансовые показатели для заказчика (ООО «Семь драконов»): годовая экономия за счёт сокращения трудозатрат и административных расходов $\approx$ 18 600 BYN, срок окупаемости внедрения $\approx$ 15 месяцев; NPV проекта около 50 000 BYN, IRR $\approx$ 75 %, PI $\approx$ 2,7. Рассчитанные показатели подтверждают экономическую целесообразность разработки и внедрения системы.

Полученные результаты

Спроектирована и реализована модульная клиент-серверная архитектура: фронтенд изолирован от хранилища данных и взаимодействует с бэкендом исключительно через REST API, что обеспечивает раздельное масштабирование уровней и упрощает сопровождение.

Сформирована единая ролевая модель доступа (клиент, менеджер, администратор) с проверкой прав на уровне middleware. Хранение паролей выполняется в виде хеша, передача данных – по защищённому каналу TLS, обработка персональных данных соответствует требованиям Закона Республики Беларусь от 7 мая 2021 г. № 99-З.

Перспективы развития

Подключение модуля штрихкодирования и интеграция с физическим складом через RF-терминалы – для ускорения комплектации и инвентаризации.

Внедрение прогноза спроса на основе машинного обучения для рекомендаций по закупкам и формированию страховых запасов.

Расширение интеграционного слоя: EDI-обмен с ключевыми поставщиками и сетевыми клиентами, синхронизация остатков с внешними WMS.

Поддержка горизонтального масштабирования: вынос асинхронной обработки в очереди, шардирование БД по контрагентам при росте нагрузки.

Все задачи, поставленные во введении, выполнены. Цель дипломного проекта достигнута: разработана цифровая платформа оптовых закупок для ООО «Семь драконов», обеспечивающая автоматизацию клиентского взаимодействия, оформления заказов, управления каталогом, поддержки пользователей и анализа результатов продаж. Платформа готова к опытной эксплуатации на реальной производственной нагрузке.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Материально-техническое обеспечение организации и использование средств труда, учебный курс «Теория организации»[Электронный ресурс]: http://www.plam.ru/bislit/teorija_organizacii_konspekt_lekcii/p8.php. Дата доступа: 09.09.2025.
2. Сидоров К. Клиент-серверные приложения: интеграция и безопасность. Минск: ТетраСистемс, 2023. С. 75 – 81.
3. Андреева Н.А., Корчагина Е.В., Корчагина В.В., Магомедов А.К. Архитектура информационной системы автоматизированного обслуживания пользователей. Вестник Воронежского института ФСИИ России. 2022, № 4. С. 23 – 29.
4. Барков Е.А., Лукьянов К.В. Проектирование информационной системы для организации и проведения мероприятий. Системный анализ. 2022, № 1. С. 54 – 66.
5. Законодательство Республики Беларусь об защите персональных данных [Электронный ресурс]: https://pravo.by/upload/docs/op/H12100099_1620939600.pdf. Дата доступа: 21.04.2021.
6. Афонин Д.В., Кочергин О.Б., Яцун С.Ф. Информационная система роботизированной бухгалтерии: архитектурно-модульный дизайн. Юг-Западное гос. изд-во. 2022. Т. 26, № 4. С. 162 – 178.
7. Сидоров К.Н. Системы контроля версий и CI/CD. Минск: ТетраСистемс, 2023. С. 12 – 18.
8. Федоров А.А. DevOps и развертывание приложений. СПб: Питер, 2023. С. 78 – 85.
9. Семенов А.П. Введение в UML. М.: ДМК Пресс, 2022. С. 30 – 32.
10. Андреева Н.А., Корчагина Е.В., Магомедов А.К. Методы системного анализа и моделирования информационных систем. М.: ДМК Пресс, 2022. С. 50 – 55.
11. Семенов А.П. Введение в UML. М.: ДМК Пресс, 2022. С. 42 – 45.
12. Андреева Н.А., Корчагина Е.В., Магомедов А.К. Архитектура информационной системы автоматизированного обслуживания пользователей. Вестник Воронежского института ФСИИ России. 2022, № 4. С. 23 – 29.
13. Корнеев А.В. Проектирование баз данных: практическое руководство. М.: ДМК Пресс, 2022. С. 22 – 24.
14. Иванова С.П. Администрирование и оптимизация PostgreSQL. СПб.: Питер, 2023. С. 14 – 16.

15. Петров С.Ю. Хранилища данных и большие объёмы. М.: БХВ-Петербург, 2023. С. 40 – 42.
16. Андреева Н.А., Корчагина Е.В., Магомедов А.К. Архитектура информационной системы автоматизированного обслуживания пользователей. Вестник Воронежского института ФСИИ России. 2022, № 4. С. 30 – 35.
17. Барков Е.А., Лукьянов К.В. Проектирование информационной системы для организации и проведения мероприятий. Системный анализ. 2022, № 1. С. 80 – 85.
18. Семёнов А.П. UML 2.0. Практика разработки ПО. СПб.: БХВ-Петербург, 2022. С. 120 – 122.
19. Иванов С.П. DevOps и непрерывная поставка. М.: Питер, 2023. С. 50 – 63.
20. Петров А.В. Управление производительностью ИТ-инфраструктуры. Минск: ТетраСистемс, 2023. С. 15 – 18.
21. Сидоров К.Н. Безопасность веб-приложений. СПб.: Питер, 2023. С. 80 – 90.
22. Иванов И. И. Что нового в React 18: обзор основных изменений [Электронный ресурс]: <https://habr.com/ru/post/670026/>. Дата доступа: 09.09.2025.
23. Петров П. П. Обзор возможностей TypeScript 5.6 [Электронный ресурс]: <https://habr.com/ru/post/680315/>. Дата доступа: 12.11.2025.
24. Смирнова Е. Д. Быстрое развитие фронтенда с Vite 4.x [Электронный ресурс]: <https://habr.com/ru/post/675432/>. Дата доступа: 09.01.2026.
25. Ковальчук Н. А. Практическое руководство по Tailwind CSS 3.x [Электронный ресурс]: <https://dev.by/tailwind-css-guide>. Дата доступа: 21.04.2026.
26. Лебедев К. Т. Кэширование данных в React с TanStackQuery [Электронный ресурс]: <https://habr.com/ru/company/otus/blog/681112/>. Дата доступа: 29.04.2026.
27. Меркулова В. Ю. Доступные UI-примитивы Radix UI: руководство [Электронный ресурс]: <https://dev.by/radix-ui-ru>. Дата доступа: 30.04.2026.
28. Филинов П. П. Drizzle ORM для PostgreSQL: обзор и приёмы [Электронный ресурс]: <https://sql.ru/forum/orm-drizzle>. Дата доступа: 15.05.2026.
29. Щербаков Ю. А. Новые возможности PostgreSQL 15 [Электронный ресурс]: <https://postgresql.org.ru/docs/15/newfeatures>. Дата доступа: 26.05.2026.
30. Яковлев М. М. Эффективное E2E-тестирование с Playwright [Электронный ресурс]: <https://habr.com/ru/post/692011/>. Дата доступа: 14.06.2026.